Overview of Kalman Filter Theory and Navigation Applications Part I

Michael L. Carroll

October 3, 2018

©2018 by Michael L. Carroll

Table of Contents

1 Course Overview

2 Kalman Filter Overview

3 Heuristic Introduction

4 Equation Drilldown

5 State Space Concepts

Course Overview

Course Overview

- Topic Overview

Course Overview

- Topic Overview
- Course Structure

Course Overview

- Topic Overview
- Course Structure
- Detailed Topic List

Course Overview

- Topic Overview
- Course Structure
- Detailed Topic List
- Instructor Background

Course Overview: Topic Overview

Course Overview: Topic Overview

- Kalman Filtering

Course Overview: Topic Overview

- Kalman Filtering
  - General Theory Overview w/o proofs

Course Overview: Topic Overview

- Kalman Filtering
  - General Theory Overview w/o proofs
  - Practical Aspects of Application

Course Overview: Topic Overview

- Kalman Filtering
  - General Theory Overview w/o proofs
  - Practical Aspects of Application
  - Matlab and Simulink Examples

Course Overview: Topic Overview

- Kalman Filtering
  - General Theory Overview w/o proofs
  - Practical Aspects of Application
  - Matlab and Simulink Examples
- Applications to GPS Control Segment

Course Overview: Course Structure

Course Overview: Course Structure

- Originally 10 segments (derived from author's 5-day engineering short course)

Course Overview: Course Structure

- Originally 10 segments (derived from author's 5-day engineering short course)
- Approximately 1 hour per segment

Course Overview: Course Structure

- Originally 10 segments (derived from author's 5-day engineering short course)
- Approximately 1 hour per segment
- Informational Course

Course Overview: Course Structure

- Originally 10 segments (derived from author's 5-day engineering short course)
- Approximately 1 hour per segment
- Informational Course
  - Concepts

Course Overview: Course Structure

- Originally 10 segments (derived from author's 5-day engineering short course)
- Approximately 1 hour per segment
- Informational Course
  - Concepts
  - Examples

Course Overview: Course Structure

- Originally 10 segments (derived from author's 5-day engineering short course)
- Approximately 1 hour per segment
- Informational Course
  - Concepts
  - Examples
  - Exercises (optional)

Course Overview: Detailed Topic List

Course Overview: Detailed Topic List

- Part I: The Basic Kalman Equations, Part I – Heuristic Overview, Equation Drilldown, State Space Concepts

Course Overview: Detailed Topic List

- Part I: The Basic Kalman Equations, Part I – Heuristic Overview, Equation Drilldown, State Space Concepts
- Part II: The Basic Kalman Equations, Part II – Probability, Statistics and Random Processes

Course Overview: Detailed Topic List

- Part I: The Basic Kalman Equations, Part I – Heuristic Overview, Equation Drilldown, State Space Concepts
- Part II: The Basic Kalman Equations, Part II – Probability, Statistics and Random Processes
- Part III: Implementation Issues and Remedies, Applications to Navigation

Course Overview: Detailed Topic List

- Part I: The Basic Kalman Equations, Part I – Heuristic Overview, Equation Drilldown, State Space Concepts
- Part II: The Basic Kalman Equations, Part II – Probability, Statistics and Random Processes
- Part III: Implementation Issues and Remedies, Applications to Navigation
- Part IV: Applications to GPS Control Segment Kalman Filters

Course Overview: Instructor Background

Course Overview: Instructor Background

- Michael Carroll

Course Overview: Instructor Background

- Michael Carroll
- Masters Degree in Mathematics

Course Overview: Instructor Background

- Michael Carroll
- Masters Degree in Mathematics
- Raytheon Navigation Mission Lead on GPS OCX Program

Course Overview: Instructor Background

- Michael Carroll
- Masters Degree in Mathematics
- Raytheon Navigation Mission Lead on GPS OCX Program
- 37 years in Aerospace and Defense Industry

Course Overview: Instructor Background

- Michael Carroll
- Masters Degree in Mathematics
- Raytheon Navigation Mission Lead on GPS OCX Program
- 37 years in Aerospace and Defense Industry
- Navigation Systems and Algorithms

Kalman Filter Overview

What is a Kalman Filter?

Kalman Filter Overview

What is a Kalman Filter?

- Optimal state estimator of a linear, stochastic dynamic system

Kalman Filter Overview

What is a Kalman Filter?

- Optimal state estimator of a linear, stochastic dynamic system
  - Estimation based on noisy, indirect measurements of state

Kalman Filter Overview

What is a Kalman Filter?

- Optimal state estimator of a linear, stochastic dynamic system
  - Estimation based on noisy, indirect measurements of state
- Essentially, it's a Recursive Weighted Least Squares (RWLS) estimator
- B.L.U.E. = Best Linear Unbiased Estimator

Kalman Filter Overview

History

Kalman Filter Overview

History

- Rudolf E. Kalman
  Seminal paper in 1960: "A New Approach to Linear Filtering and Prediction Problems"

Kalman Filter Overview

History

- Rudolf E. Kalman
  Seminal paper in 1960: "A New Approach to Linear Filtering and Prediction Problems"
- Actually preceded by work of Peter Swerling at RAND Corp in 1950s

Kalman Filter Overview

History

- Rudolf E. Kalman
  Seminal paper in 1960: "A New Approach to Linear Filtering and Prediction Problems"
- Actually preceded by work of Peter Swerling at RAND Corp in 1950s
- Least squares goes back to Coates(1722), Boscovich(1757), Laplace(1799), Legendre(1805), Gauss(1809)

Kalman Filter Overview

History

- Rudolf E. Kalman
  Seminal paper in 1960: "A New Approach to Linear Filtering and Prediction Problems"
- Actually preceded by work of Peter Swerling at RAND Corp in 1950s
- Least squares goes back to Coates(1722), Boscovich(1757), Laplace(1799), Legendre(1805), Gauss(1809)
- Rigorous mathematical foundation built on work of mathematician David Hilbert in 1920s

Kalman Filter Overview

History

- Rudolf E. Kalman
  Seminal paper in 1960: "A New Approach to Linear Filtering and Prediction Problems"
- Actually preceded by work of Peter Swerling at RAND Corp in 1950s
- Least squares goes back to Coates(1722), Boscovich(1757), Laplace(1799), Legendre(1805), Gauss(1809)
- Rigorous mathematical foundation built on work of mathematician David Hilbert in 1920s
  - Kalman filtering now recognized as orthogonal projection in infinite dimensional Hilbert Space

Kalman Filter Overview

History

Kalman Filter Overview

History

- Kalman built upon work by the Russian Pontrayagin (optimal control theory)

Kalman Filter Overview

History

- Kalman built upon work by the Russian Pontrayagin (optimal control theory)
  - Kalman showed the duality between Controllability and Observability

Kalman Filter Overview

History

- Kalman built upon work by the Russian Pontrayagin (optimal control theory)
  - Kalman showed the duality between Controllability and Observability
  - Worked in **time domain** favorable MIMO (multiple input multiple output) in contrast to Norbert Wiener's filtering in **frequency domain**

Kalman Filter Overview

History

- Kalman built upon work by the Russian Pontrayagin (optimal control theory)
  - Kalman showed the duality between Controllability and Observability
  - Worked in **time domain** favorable MIMO (multiple input multiple output) in contrast to Norbert Wiener's filtering in **frequency domain**
- State space concepts known to physicists: Gibbs coined term "phase space" in 1901

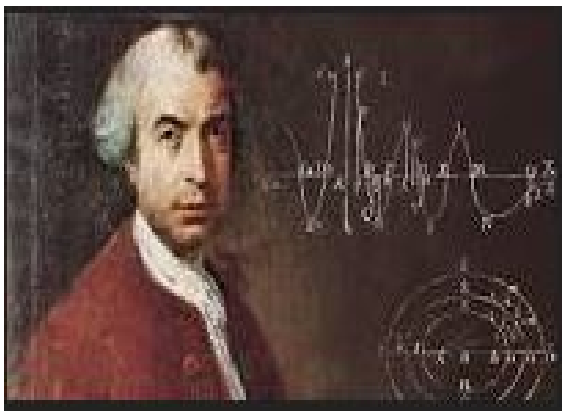
Kalman Filter Overview

History

- Kalman built upon work by the Russian Pontrayagin (optimal control theory)
  - Kalman showed the duality between Controllability and Observability
  - Worked in **time domain** favorable MIMO (multiple input multiple output) in contrast to Norbert Wiener's filtering in **frequency domain**
- State space concepts known to physicists: Gibbs coined term "phase space" in 1901
- Elie Cartan: 1869-1951: "espace des etats"

Kalman Filter Overview

Least Squares Luminaries

Roger Boscovich 

Karl F. Gauss 

Least Squares Luminaries

Peter Swerling 

Rudolf Kalman 

Kalman Filter Overview

History

Kalman Filter Overview

History

- Used in early 1960s by Schmidt on Apollo program

Kalman Filter Overview

History

- Used in early 1960s by Schmidt on Apollo program
- Kalman filters are now ubiquitous, especially in navigation

Kalman Filter Overview

KF uses in GPS

Kalman Filter Overview

KF uses in GPS

- All three segments have Kalman filters

Kalman Filter Overview

KF uses in GPS

- All three segments have Kalman filters
- CS: ECEST = Ephemeris and Clock Estimator (legacy term) and RTGx (JPL's Real-Time GPSY)

Kalman Filter Overview

KF uses in GPS

- All three segments have Kalman filters
- CS: ECEST = Ephemeris and Clock Estimator (legacy term) and RTGx (JPL's Real-Time GPSY)
- Estimates inertial positions and velocities of SVs, solar pressure parameters, SV and MS clock parameters, and troposphere state at each MS

Kalman Filter Overview

KF uses in GPS

- All three segments have Kalman filters
- CS: ECEST = Ephemeris and Clock Estimator (legacy term) and RTGx (JPL's Real-Time GPSY)
- Estimates inertial positions and velocities of SVs, solar pressure parameters, SV and MS clock parameters, and troposphere state at each MS
- In UE receivers, satellite nav software, and offline computations

Kalman Filter Overview

KF uses in GPS

- All three segments have Kalman filters
- CS: ECEST = Ephemeris and Clock Estimator (legacy term) and RTGx (JPL's Real-Time GPSY)
- Estimates inertial positions and velocities of SVs, solar pressure parameters, SV and MS clock parameters, and troposphere state at each MS
- In UE receivers, satellite nav software, and offline computations
- Used in aircraft Operational Flight Programs (OFPs) and Inertial Navigation Systems (INSs) to integrate GPS and other external navaids to optimally estimate INS error states

Kalman Filter Overview

KF uses in GPS

- All three segments have Kalman filters
- CS: ECEST = Ephemeris and Clock Estimator (legacy term) and RTGx (JPL's Real-Time GPSY)
- Estimates inertial positions and velocities of SVs, solar pressure parameters, SV and MS clock parameters, and troposphere state at each MS
- In UE receivers, satellite nav software, and offline computations
- Used in aircraft Operational Flight Programs (OFPs) and Inertial Navigation Systems (INSs) to integrate GPS and other external navaids to optimally estimate INS error states
- 1990s: Paradigm shift in navigation:
  Was: "INS/GPS"
  Is: "GPS/INS"

Kalman Filter Overview

Kalman Filter Overview

- As a branch of applied mathematics, Kalman filtering uses techniques and results from many mathematical subjects: linear algebra, differential equations, measure theory, probability, statistics, functional analysis (Hilbert space)

Kalman Filter Overview

- As a branch of applied mathematics, Kalman filtering uses techniques and results from many mathematical subjects: linear algebra, differential equations, measure theory, probability, statistics, functional analysis (Hilbert space)
- But its essentials are very intuitive and can be understood with minimal math background (matrix algebra, calculus, probability and statistics)

Kalman Filter Overview

Some Notational Conventions

Kalman Filter Overview

Some Notational Conventions

- Time-dependent scalar or vector quantities: $x(k)$ (discrete), $x(t)$ (continuous)

Kalman Filter Overview

Some Notational Conventions

- Time-dependent scalar or vector quantities: $x(k)$ (discrete), $x(t)$ (continuous)
- Corresponding vector components: $x_i(k)$ or $x_i(t)$

Kalman Filter Overview

Some Notational Conventions

- Time-dependent scalar or vector quantities:
  $x(k)$ (discrete), $x(t)$ (continuous)
- Corresponding vector components:
  $x_i(k)$ or $x_i(t)$
- Time-dependent matrix quantities (capital letters):
  $P(k)$ (discrete), $P(t)$ (continuous)

Kalman Filter Overview

Some Notational Conventions

- Time-dependent scalar or vector quantities:
  $x(k)$ (discrete), $x(t)$ (continuous)
- Corresponding vector components:
  $x_i(k)$ or $x_i(t)$
- Time-dependent matrix quantities (capital letters):
  $P(k)$ (discrete), $P(t)$ (continuous)
- Corresponding matrix components (elements):
  $p_{ij}(k)$ or $p_{ij}(t)$

Kalman Filter Overview

Some Notational Conventions

- Time-dependent scalar or vector quantities:
  $x(k)$ (discrete), $x(t)$ (continuous)
- Corresponding vector components:
  $x_i(k)$ or $x_i(t)$
- Time-dependent matrix quantities (capital letters):
  $P(k)$ (discrete), $P(t)$ (continuous)
- Corresponding matrix components (elements):
  $p_{ij}(k)$ or $p_{ij}(t)$
- Use hats for estimated quantities and special superscripts to indicate pre- and post-update values:
  $\hat{x}^+(k)$ and $P^-(k)$

Part I

The Basic Kalman Equations

Topics

Part I

The Basic Kalman Equations

Topics

- Understanding the Equations: Heuristic Introduction

Part I

The Basic Kalman Equations

Topics

- Understanding the Equations: Heuristic Introduction
- Taking the Equations Apart

Part I

The Basic Kalman Equations

Topics

- Understanding the Equations: Heuristic Introduction
- Taking the Equations Apart
- State Space Concepts

The Basic Kalman Equations

Understanding the Equations: Heuristic Introduction

Subtopics

- Recursive Predictor-Corrector Algorithms

The Basic Kalman Equations

Understanding the Equations: Heuristic Introduction

Subtopics

- Recursive Predictor-Corrector Algorithms
- Running Averages

The Basic Kalman Equations

Understanding the Equations: Heuristic Introduction

Subtopics

- Recursive Predictor-Corrector Algorithms
- Running Averages
- Extrapolation (Prediction)

The Basic Kalman Equations

Understanding the Equations: Heuristic Introduction

Subtopics

- Recursive Predictor-Corrector Algorithms
- Running Averages
- Extrapolation (Prediction)
- Gain Computation

The Basic Kalman Equations

Understanding the Equations: Heuristic Introduction

Subtopics

- Recursive Predictor-Corrector Algorithms
- Running Averages
- Extrapolation (Prediction)
- Gain Computation
- Update

Recursive Predictor-Corrector Algorithm

What kind of problems are we trying to solve?

Recursive Predictor-Corrector Algorithm

What kind of problems are we trying to solve?

- To estimate quantities associated with a noisy dynamic process using imperfect, noisy measurements

Recursive Predictor-Corrector Algorithm

What kind of problems are we trying to solve?

- To estimate quantities associated with a noisy dynamic process using imperfect, noisy measurements
  - Generally a vector quantity: the **state** of the system or process

Recursive Predictor-Corrector Algorithm

What kind of problems are we trying to solve?

- To estimate quantities associated with a noisy dynamic process using imperfect, noisy measurements
  - Generally a vector quantity: the **state** of the system or process
  - Dynamic process is only modeled; not known perfectly and may be stochastic: i.e., slightly different each time it happens

Recursive Predictor-Corrector Algorithm

What kind of problems are we trying to solve?

- To estimate quantities associated with a noisy dynamic process using imperfect, noisy measurements
  - Generally a vector quantity: the **state** of the system or process
  - Dynamic process is only modeled; not known perfectly and may be stochastic: i.e., slightly different each time it happens
  - We never really know the true state at any given time – that's why we need an estimator!

Recursive Predictor-Corrector Algorithm

What kind of problems are we trying to solve?

- To estimate quantities associated with a noisy dynamic process using imperfect, noisy measurements
  - Generally a vector quantity: the **state** of the system or process
  - Dynamic process is only modeled; not known perfectly and may be stochastic: i.e., slightly different each time it happens
  - We never really know the true state at any given time – that's why we need an estimator!
- Measurements may be from multiple sensors simultaneously and/or sequentially

Recursive Predictor-Corrector Algorithm

What kind of problems are we trying to solve?

- To estimate quantities associated with a noisy dynamic process using imperfect, noisy measurements
  - Generally a vector quantity: the **state** of the system or process
  - Dynamic process is only modeled; not known perfectly and may be stochastic: i.e., slightly different each time it happens
  - We never really know the true state at any given time – that's why we need an estimator!
- Measurements may be from multiple sensors simultaneously and/or sequentially
  - Measurements don't necessarily measure the states directly

Recursive Predictor-Corrector Algorithm

Recursive Predictor-Corrector Algorithm

- Goal: Devise a method to combine several measurements and produce the best estimate of some quantity of interest (the state of some system)

Recursive Predictor-Corrector Algorithm

- Goal: Devise a method to combine several measurements and produce the best estimate of some quantity of interest (the state of some system)
- Measurements don't always happen simultaneously. Often sequential, especially if multiple, independent measurement sources are available.

Recursive Predictor-Corrector Algorithm

- Goal: Devise a method to combine several measurements and produce the best estimate of some quantity of interest (the state of some system)
- Measurements don't always happen simultaneously. Often sequential, especially if multiple, independent measurement sources are available.
  - Hence we need an algorithm.

Recursive Predictor-Corrector Algorithm

- Goal: Devise a method to combine several measurements and produce the best estimate of some quantity of interest (the state of some system)
- Measurements don't always happen simultaneously. Often sequential, especially if multiple, independent measurement sources are available.
  - Hence we need an algorithm.
  - Preferably a recursive algorithm whereby we update (correct) our last estimate with each new piece of measurement data

Recursive Predictor-Corrector Algorithm

- Goal: Devise a method to combine several measurements and produce the best estimate of some quantity of interest (the state of some system)
- Measurements don't always happen simultaneously. Often sequential, especially if multiple, independent measurement sources are available.
  - Hence we need an algorithm.
  - Preferably a recursive algorithm whereby we update (correct) our last estimate with each new piece of measurement data
- But the system whose state we want to estimate is often dynamic, i.e., changing with time. So we need to take that into account (prediction).

Recursive Predictor-Corrector Algorithm

- Goal: Devise a method to combine several measurements and produce the best estimate of some quantity of interest (the state of some system)
- Measurements don't always happen simultaneously. Often sequential, especially if multiple, independent measurement sources are available.
  - Hence we need an algorithm.
  - Preferably a recursive algorithm whereby we update (correct) our last estimate with each new piece of measurement data
- But the system whose state we want to estimate is often dynamic, i.e., changing with time. So we need to take that into account (prediction).
- Bottom line: We seek a recursive predictor-corrector algorithm.

Recursive Predictor-Corrector Algorithm

Recursive Predictor-Corrector Algorithm

- Prediction:

Recursive Predictor-Corrector Algorithm

- Prediction:
  - **Extrapolate** solution from previous iteration to present time

Recursive Predictor-Corrector Algorithm

- Prediction:
  - **Extrapolate** solution from previous iteration to present time (This is how we take system dynamics into account!)

Recursive Predictor-Corrector Algorithm

- Prediction:
  - **Extrapolate** solution from previous iteration to present time (This is how we take system dynamics into account!)
- Correction:

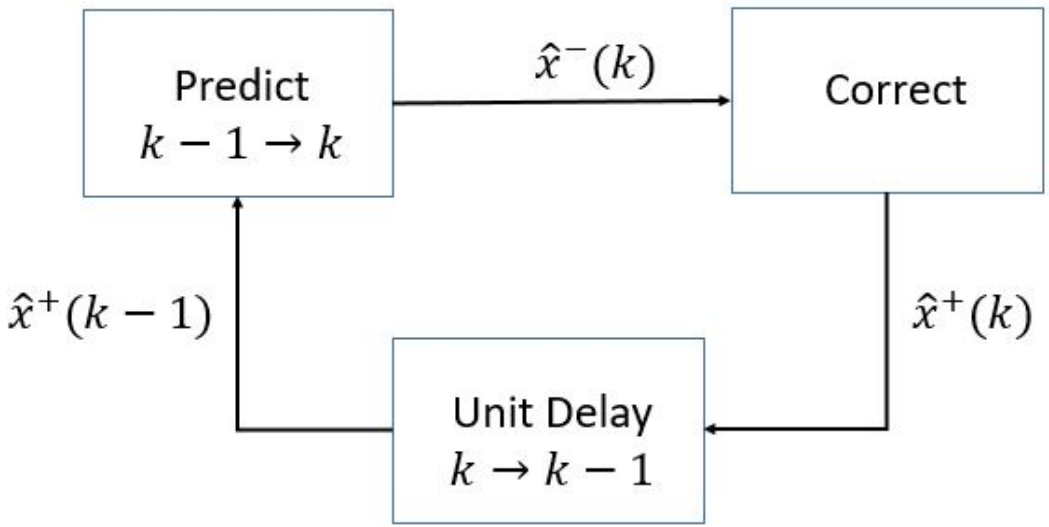
Recursive Predictor-Corrector Algorithm

- Prediction:
  - **Extrapolate** solution from previous iteration to present time (This is how we take system dynamics into account!)
- Correction:
  - Use measurement information to **update** or **correct** the prediction

Recursive Predictor-Corrector Algorithm

- Prediction:
  - **Extrapolate** solution from previous iteration to present time (This is how we take system dynamics into account!)
- Correction:
  - Use measurement information to **update** or **correct** the prediction
- And the cycle recursively begins anew!

Recursive Predictor-Corrector Algorithm (Discrete System)



Recursive Predictor-Corrector Algorithm

Recursive Predictor-Corrector Algorithm

- Algorithm

Recursive Predictor-Corrector Algorithm

- Algorithm
  - Computational procedure

Recursive Predictor-Corrector Algorithm

- Algorithm
  - Computational procedure
  - Given initial values of some quantities, new quantities are computed

Recursive Predictor-Corrector Algorithm

- Algorithm
  - Computational procedure
  - Given initial values of some quantities, new quantities are computed
- Recursive

Recursive Predictor-Corrector Algorithm

- Algorithm
  - Computational procedure
  - Given initial values of some quantities, new quantities are computed
- Recursive
  - The new quantities are the same as the old quantities

Recursive Predictor-Corrector Algorithm

- Algorithm
  - Computational procedure
  - Given initial values of some quantities, new quantities are computed
- Recursive
  - The new quantities are the same as the old quantities
  - New quantities become initial values for next cycle

Simple Example: Running Averages

Simple Example: Running Averages

- Suppose we want to estimate some quantity like the resistance of a single resistor. We have ...

Simple Example: Running Averages

- Suppose we want to estimate some quantity like the resistance of a single resistor. We have ...
  - a voltmeter with which to make measurements

Simple Example: Running Averages

- Suppose we want to estimate some quantity like the resistance of a single resistor. We have ...
  - a voltmeter with which to make measurements
  - a model for the resistance: the "true" value x of the resistance is assumed to be constant in time: $x(k+1) = x(k)$

Simple Example: Running Averages

- Suppose we want to estimate some quantity like the resistance of a single resistor. We have ...
  - a voltmeter with which to make measurements
  - a model for the resistance: the "true" value x of the resistance is assumed to be constant in time: $x(k+1) = x(k)$
- We wish to take repeated measurements z of the resistance using the voltmeter

Simple Example: Running Averages

- Suppose we want to estimate some quantity like the resistance of a single resistor. We have ...
  - a voltmeter with which to make measurements
  - a model for the resistance: the "true" value x of the resistance is assumed to be constant in time: $x(k+1) = x(k)$
- We wish to take repeated measurements z of the resistance using the voltmeter $z(1), z(2), \ldots$

Simple Example: Running Averages

Simple Example: Running Averages

- The **state** that we are trying to estimate is the resistance value of the resistor

Simple Example: Running Averages

- The **state** that we are trying to estimate is the resistance value of the resistor
- We don't know what the state is. (Otherwise, we wouldn't need to take measurements)

Simple Example: Running Averages

- The **state** that we are trying to estimate is the resistance value of the resistor
- We don't know what the state is. (Otherwise, we wouldn't need to take measurements)
- Our **state dynamics** or **process model** is very simple: The state is 1-dimensional and does not change with time. So the model is:

$$x(k+1) = x(k)$$

Simple Example: Running Averages

Simple Example: Running Averages

- What is the best way to estimate the state of our system?

Simple Example: Running Averages

- What is the best way to estimate the state of our system?
- In the absence of any other information, we should probably just average the measurements

Simple Example: Running Averages

- What is the best way to estimate the state of our system?
- In the absence of any other information, we should probably just average the measurements
- We assume that the averages are unbiased, i.e., the measurement errors have zero mean

Simple Example: Running Averages

Simple Example: Running Averages

- Simple running average is like a Kalman filter with trivial dynamics ($x(k+1) = x(k)$)

Simple Example: Running Averages

- Simple running average is like a Kalman filter with trivial dynamics ($x(k+1) = x(k)$)

Given a sequence of random measurements,$z(1), z(2), \ldots$, compute at each stage the best estimate of the true value of the state: At the $n+1^{\text{st}}$ step, your best guess would be the average of all previous values:

Simple Example: Running Averages

- Simple running average is like a Kalman filter with trivial dynamics ($x(k+1) = x(k)$)

Given a sequence of random measurements,$z(1), z(2), \ldots$, compute at each stage the best estimate of the true value of the state: At the $n+1^{\text{st}}$ step, your best guess would be the average of all previous values:

$$\hat{x}(k+1) = \frac{1}{k}\sum_{j=1}^{k} z(j) = \frac{1}{k}[\, z(1) + z(2) + \cdots + z(k)\,] \qquad (1)$$

Simple Example: Running Averages

Simple Example: Running Averages

- Q: Why is the average of all previous measurements the "best" estimate?

Simple Example: Running Averages

- Q: Why is the average of all previous measurements the "best" estimate?
- A: The dynamics tell us that the quantity to be estimated is not changing, and the only variability stems from the measurements, i.e., the measurement noise

Simple Example: Running Averages

- Q: Why is the average of all previous measurements the "best" estimate?
- A: The dynamics tell us that the quantity to be estimated is not changing, and the only variability stems from the measurements, i.e., the measurement noise
- Hence, we should try to eliminate the measurement variability by averaging the measurements

Recursive Running Average

Recursive Running Average

- Therefore, as much as possible, we want to rid ourselves of the noise by averaging

Recursive Running Average

- Therefore, as much as possible, we want to rid ourselves of the noise by averaging
- Assume that noise is zero-mean, i.e., its average tends to zero over the long run

Recursive Running Average

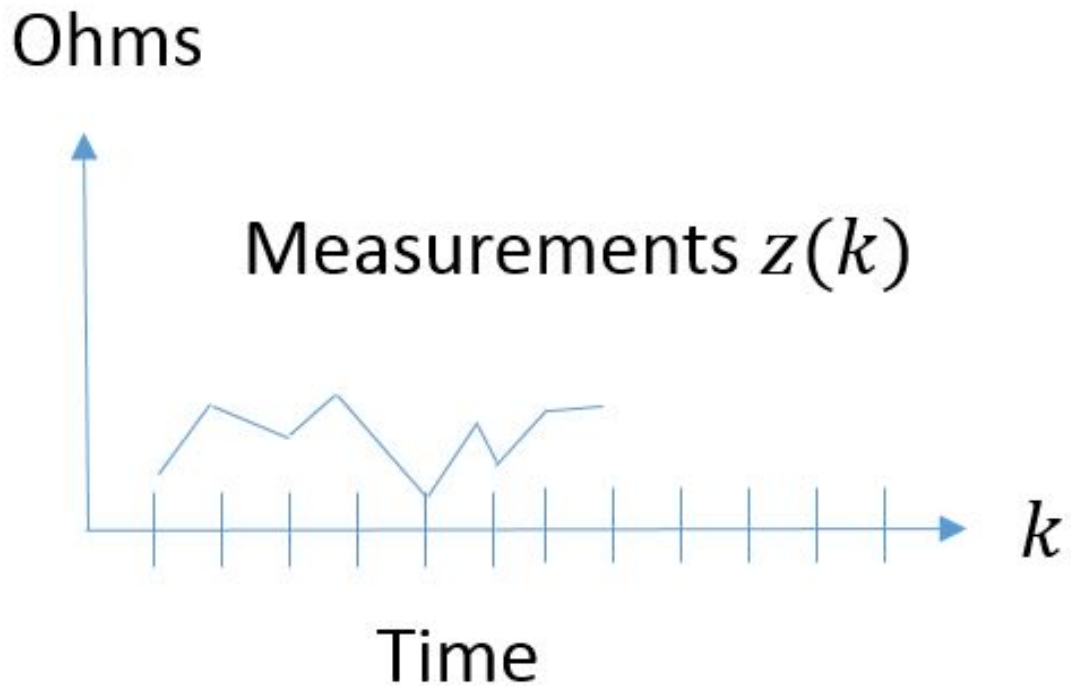
- Therefore, as much as possible, we want to rid ourselves of the noise by averaging
- Assume that noise is zero-mean, i.e., its average tends to zero over the long run
- If we had a bias, we wouldn't call that noise; that would be a systematic error on the part of our voltmeter. We'd have to calibrate it out.

Recursive Running Average

- Therefore, as much as possible, we want to rid ourselves of the noise by averaging
- Assume that noise is zero-mean, i.e., its average tends to zero over the long run
- If we had a bias, we wouldn't call that noise; that would be a systematic error on the part of our voltmeter. We'd have to calibrate it out.
- Noise can't be calibrated out; we have to average it out over time

Recursive Running Average

Recursive Running Average



Recursive Running Average

Recursive Running Average

(Eq.(1))

$$\hat{x}(k+1) = \frac{1}{k}\sum_{j=1}^{k} z(j) = \frac{1}{k}[\, z(1) + z(2) + \cdots + z(k)\,]$$

Recursive Running Average

(Eq.(1))

$$\hat{x}(k+1) = \frac{1}{k}\sum_{j=1}^{k} z(j) = \frac{1}{k}[\, z(1) + z(2) + \cdots + z(k)\,]$$

- Now suppose our estimating task is going to continue for a long time and we don't want to fill up our computer memory with a record of all the measurements taken since the start

Recursive Running Average

(Eq.(1))

$$\hat{x}(k+1) = \frac{1}{k}\sum_{j=1}^{k} z(j) = \frac{1}{k}[\, z(1) + z(2) + \cdots + z(k)\,]$$

- Now suppose our estimating task is going to continue for a long time and we don't want to fill up our computer memory with a record of all the measurements taken since the start
- We just want to use the previously calculated value and add a small adjustment

Step-by-Step Derivation:

Step-by-Step Derivation:

$$\hat{x}(k+1) = \frac{1}{k}\sum_{j=1}^{k} z(j) \qquad \text{by def. Eq. (1)}$$

Step-by-Step Derivation:

$$\hat{x}(k+1) = \frac{1}{k}\sum_{j=1}^{k} z(j) \qquad \text{by def. Eq. (1)}$$

$$= \frac{1}{k}\sum_{j=1}^{k-1} z(j) + \frac{1}{k} z(k) \qquad \text{separate last term from sum}$$

Step-by-Step Derivation:

$$\hat{x}(k+1) = \frac{1}{k}\sum_{j=1}^{k} z(j) \qquad \text{by def. Eq. (1)}$$

$$= \frac{1}{k}\sum_{j=1}^{k-1} z(j) + \frac{1}{k}z(k) \qquad \text{separate last term from sum}$$

$$= \frac{1}{k}\cdot\frac{k-1}{k-1}\sum_{j=1}^{k-1} z(j) + \frac{1}{k}z(k) \qquad \text{multiply by 1}$$

Step-by-Step Derivation:

$$\hat{x}(k+1) = \frac{1}{k}\sum_{j=1}^{k} z(j) \qquad \text{by def. Eq. (1)}$$

$$= \frac{1}{k}\sum_{j=1}^{k-1} z(j) + \frac{1}{k}z(k) \qquad \text{separate last term from sum}$$

$$= \frac{1}{k}\cdot\frac{k-1}{k-1}\sum_{j=1}^{k-1} z(j) + \frac{1}{k}z(k) \qquad \text{multiply by 1}$$

$$= \frac{k-1}{k}\cdot\frac{1}{k-1}\sum_{j=1}^{k-1} z(j) + \frac{1}{k}z(k) \qquad \text{rearrange}$$

Step-by-Step Derivation:

$$\hat{x}(k+1) = \frac{1}{k}\sum_{j=1}^{k} z(j) \qquad \text{by def. Eq. (1)}$$

$$= \frac{1}{k}\sum_{j=1}^{k-1} z(j) + \frac{1}{k}z(k) \qquad \text{separate last term from sum}$$

$$= \frac{1}{k}\cdot\frac{k-1}{k-1}\sum_{j=1}^{k-1} z(j) + \frac{1}{k}z(k) \qquad \text{multiply by 1}$$

$$= \frac{k-1}{k}\cdot\frac{1}{k-1}\sum_{j=1}^{k-1} z(j) + \frac{1}{k}z(k) \qquad \text{rearrange}$$

$$= \frac{k-1}{k}\hat{x}(k) + \frac{1}{k}z(k) \qquad \text{by def. Eq. (1) again}$$

Step-by-Step Derivation:

$$\hat{x}(k+1) = \frac{1}{k}\sum_{j=1}^{k} z(j) \qquad \text{by def. Eq. (1)}$$

$$= \frac{1}{k}\sum_{j=1}^{k-1} z(j) + \frac{1}{k}z(k) \qquad \text{separate last term from sum}$$

$$= \frac{1}{k}\cdot\frac{k-1}{k-1}\sum_{j=1}^{k-1} z(j) + \frac{1}{k}z(k) \qquad \text{multiply by 1}$$

$$= \frac{k-1}{k}\cdot\frac{1}{k-1}\sum_{j=1}^{k-1} z(j) + \frac{1}{k}z(k) \qquad \text{rearrange}$$

$$= \frac{k-1}{k}\hat{x}(k) + \frac{1}{k}z(k) \qquad \text{by def. Eq. (1) again}$$

$$= \hat{x}(k) - \frac{1}{k}\hat{x}(k) + \frac{1}{k}z(k) \qquad \text{distribute and simplify}$$

Step-by-Step Derivation:

$$\hat{x}(k+1) = \frac{1}{k}\sum_{j=1}^{k} z(j) \qquad \text{by def. Eq. (1)}$$

$$= \frac{1}{k}\sum_{j=1}^{k-1} z(j) + \frac{1}{k}z(k) \qquad \text{separate last term from sum}$$

$$= \frac{1}{k}\cdot\frac{k-1}{k-1}\sum_{j=1}^{k-1} z(j) + \frac{1}{k}z(k) \qquad \text{multiply by 1}$$

$$= \frac{k-1}{k}\cdot\frac{1}{k-1}\sum_{j=1}^{k-1} z(j) + \frac{1}{k}z(k) \qquad \text{rearrange}$$

$$= \frac{k-1}{k}\hat{x}(k) + \frac{1}{k}z(k) \qquad \text{by def. Eq. (1) again}$$

$$= \hat{x}(k) - \frac{1}{k}\hat{x}(k) + \frac{1}{k}z(k) \qquad \text{distribute and simplify}$$

$$= \hat{x}(k) + \frac{1}{k}\left[\, z(k) - \hat{x}(k) \,\right]. \qquad \text{rearrange and factor out } 1/k$$

Recursive Running Average

Recursive Running Average

- We now have a recursive algorithm for computing a running average.
  Recipe:

Recursive Running Average

- We now have a recursive algorithm for computing a running average.
  Recipe:
  - Take new measurement, subtract off old average and weight the resulting residual by a gain of $1/k$

Recursive Running Average

- We now have a recursive algorithm for computing a running average.
  Recipe:
  - Take new measurement, subtract off old average and weight the resulting residual by a gain of $1/k$
  - Add this weighted residual to the old average and, voilà, you have the new average!

Recursive Running Average

Recursive Running Average

- Note that we don't have to remember all the previous values or averages, only the last average; we use the latest measurement to **correct** the last average

Recursive Running Average

- Note that we don't have to remember all the previous values or averages, only the last average; we use the latest measurement to **correct** the last average
- Note also that as k goes to ∞, the gain $1/k$ goes to 0. (The filter is said to be going to sleep!)

Recursive Running Average

- Note that we don't have to remember all the previous values or averages, only the last average; we use the latest measurement to **correct** the last average
- Note also that as k goes to ∞, the gain $1/k$ goes to 0. (The filter is said to be going to sleep!)

$$\hat{x}(k+1) = \hat{x}(k) + \frac{1}{k}\left[z(k) - \hat{x}(k)\right] \tag{2}$$

Recursive Running Average

Recursive Running Average

- The **residual** $z(k) - \hat{x}(k)$ is the difference between the actual new measurement and what we expected or predicted the new measurement to be

Recursive Running Average

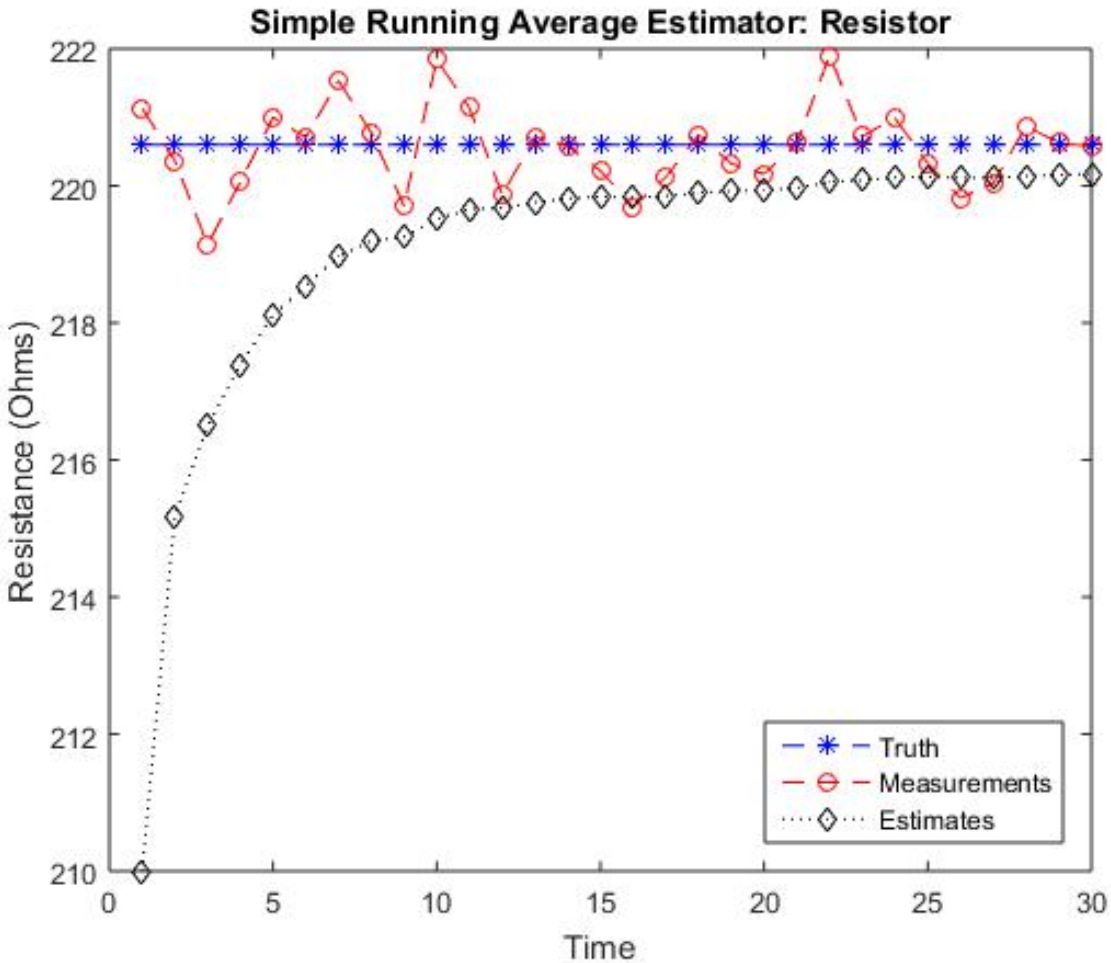
- The **residual** $z(k) - \hat{x}(k)$ is the difference between the actual new measurement and what we expected or predicted the new measurement to be
- Why is $\hat{x}(k)$ our predicted measurement?

Recursive Running Average

- The **residual** $z(k) - \hat{x}(k)$ is the difference between the actual new measurement and what we expected or predicted the new measurement to be
- Why is $\hat{x}(k)$ our predicted measurement?
- Answer: In the absence of new information, the previous estimate is the only thing we have to go on!

Recursive Running Average
MATLAB Scripts: runavgdemo.m, recursive_avg.m

Recursive Running Average
MATLAB Scripts: runavgdemo.m, recursive_avg.m



Extrapolation (Prediction)

Extrapolation (Prediction)

- In the running average example, the dynamics were trivial, i.e., $x(k+1) = x(k)$

Extrapolation (Prediction)

- In the running average example, the dynamics were trivial, i.e., $x(k+1) = x(k)$
- This means that the state never really changes. It's always equal to its previous value.

Extrapolation (Prediction)

- In the running average example, the dynamics were trivial, i.e., $x(k+1) = x(k)$
- This means that the state never really changes. It's always equal to its previous value.
- We just don't know what the exact value is.

Extrapolation (Prediction)

- In the running average example, the dynamics were trivial, i.e., $x(k+1) = x(k)$
- This means that the state never really changes. It's always equal to its previous value.
- We just don't know what the exact value is.
- Example: Measuring the same resistor over and over again with a voltmeter to estimate the resistor's resistance value. Or getting on and off a scale repeatedly.

Extrapolation (Prediction)

Extrapolation (Prediction)

- In more general Kalman filtering, we complicate the situation by adding two more elements:

Extrapolation (Prediction)

- In more general Kalman filtering, we complicate the situation by adding two more elements:
  - State dynamics with time dependence and process noise:

$$x(k+1) = f\left[k, x(k), w(k)\right]$$

Extrapolation (Prediction)

- In more general Kalman filtering, we complicate the situation by adding two more elements:
  - State dynamics with time dependence and process noise:
$$x(k+1) = f\left[k, x(k), w(k)\right]$$
  - Typically, the process noise is additive:
$$x(k+1) = f\left[k, x(k)\right] + w(k)$$

Extrapolation (Prediction)

Extrapolation (Prediction)

- Form of function f can be simple or complicated, linear or nonlinear

Extrapolation (Prediction)

- Form of function f can be simple or complicated, linear or nonlinear
- Independent variable k indicates possible explicit time dependence

Extrapolation (Prediction)

- Form of function f can be simple or complicated, linear or nonlinear
- Independent variable k indicates possible explicit time dependence
- Process noise assumed to be Gaussian white noise (to be defined later)

Extrapolation (Prediction)

Extrapolation (Prediction)

- General Kalman filtering further complicates things by allowing the state variable to be vector-valued

Extrapolation (Prediction)

- General Kalman filtering further complicates things by allowing the state variable to be vector-valued
- This turns the function f into a vector-valued function, often a matrix (possibly time-dependent)

Extrapolation (Prediction)

- General Kalman filtering further complicates things by allowing the state variable to be vector-valued
- This turns the function f into a vector-valued function, often a matrix (possibly time-dependent)
- Process noise must therefore also become a vector-valued variable

Extrapolation (Prediction)

Extrapolation (Prediction)

- Note that presence of a random forcing function like Gaussian white noise makes the state vector process a stochastic process (aka random process)

Extrapolation (Prediction)

- Note that presence of a random forcing function like Gaussian white noise makes the state vector process a stochastic process (aka random process)
  - If white noise were zero, problem would be deterministic

Extrapolation (Prediction)

- Note that presence of a random forcing function like Gaussian white noise makes the state vector process a stochastic process (aka random process)
  - If white noise were zero, problem would be deterministic
- We'll see what this means when we take a look at random variables and random processes

Extrapolation (Prediction)

- Note that presence of a random forcing function like Gaussian white noise makes the state vector process a stochastic process (aka random process)
  - If white noise were zero, problem would be deterministic
- We'll see what this means when we take a look at random variables and random processes
- Even for the same initial conditions the sample trajectory will be different each time, due to randomly changing forcing functions

Extrapolation (Prediction)

Extrapolation (Prediction)

- Note that we did not mention a control vector u that could also be part of the forcing function

Extrapolation (Prediction)

- Note that we did not mention a control vector u that could also be part of the forcing function
  - Control vectors usually considered to be deterministic although in reality you can't control things perfectly!

Extrapolation (Prediction)

- Note that we did not mention a control vector u that could also be part of the forcing function
  - Control vectors usually considered to be deterministic although in reality you can't control things perfectly!
  - E.g., aircraft actuator (e.g., elevator or flaps) deflection angle might be a control quantity yet its precise value may not be known!)

Extrapolation (Prediction)

- Note that we did not mention a control vector u that could also be part of the forcing function
  - Control vectors usually considered to be deterministic although in reality you can't control things perfectly!
  - E.g., aircraft actuator (e.g., elevator or flaps) deflection angle might be a control quantity yet its precise value may not be known!)
- We are going to ignore the presence of deterministic control and focus solely on state estimation

Measurement Update (Correction)

Measurement Update (Correction)

- In running average example, the update implicitly assumed that measurements were corrupted by white noise

Measurement Update (Correction)

- In running average example, the update implicitly assumed that measurements were corrupted by white noise
- Math model for this would be:
  $z(k) = x(k) + v(k)$,
  where $v(k)$ is Gaussian white measurement noise sequence

Measurement Update (Correction)

- In running average example, the update implicitly assumed that measurements were corrupted by white noise
- Math model for this would be:
  $z(k) = x(k) + v(k)$,
  where $v(k)$ is Gaussian white measurement noise sequence
- Measurement processing is also often called Observation processing

Measurement Update (Correction)

Measurement Update (Correction)

- Because we assumed that measurement noise was white Gaussian and zero mean, our best guess was simply the average value of all the measurements

Measurement Update (Correction)

- Because we assumed that measurement noise was white Gaussian and zero mean, our best guess was simply the average value of all the measurements
- Remember the resistor example: our voltmeter is not perfect, and each measurement will differ slightly from the previous one, even though we are measuring the same resistor each time

Measurement Update (Correction)

Measurement Update (Correction)

- General Kalman filtering complicates the measurement model by adding two new elements:

Measurement Update (Correction)

- General Kalman filtering complicates the measurement model by adding two new elements:
  - The measurement may have a more complicated relationship to the underlying system state:

$$z(k) = h\left[k, x(k)\right] + v(k),$$

where h could be a non-linear function with possible explicit time dependence

Measurement Update (Correction)

- General Kalman filtering complicates the measurement model by adding two new elements:
  - The measurement may have a more complicated relationship to the underlying system state:

$$z(k) = h\left[k, x(k)\right] + v(k),$$

    where h could be a non-linear function with possible explicit time dependence
  - Just as the state x is allowed to be a vector variable, so the measurement z could also be a vector variable

Kalman Gain Computation

Kalman Gain Computation

- Even though we've discussed the measurement update processing step, there is an important calculation make just prior the update

Kalman Gain Computation

- Even though we've discussed the measurement update processing step, there is an important calculation make just prior the update
- Before we can incorporate the information contained in both the old estimate and the new measurement, we have to first calculate the **Kalman gain** K_k at the kth step

Kalman Gain Computation

- Even though we've discussed the measurement update processing step, there is an important calculation make just prior the update
- Before we can incorporate the information contained in both the old estimate and the new measurement, we have to first calculate the **Kalman gain** K_k at the kth step
- $K_k = \frac{1}{k}$ in our running average example

Kalman Gain Computation

- Even though we've discussed the measurement update processing step, there is an important calculation make just prior the update
- Before we can incorporate the information contained in both the old estimate and the new measurement, we have to first calculate the **Kalman gain** K_k at the kth step
- $K_k = \frac{1}{k}$ in our running average example
- If we have k pieces of information *of equal quality*, it seems intuitive that we weight each piece by $1/k$

Kalman Gain Computation

Kalman Gain Computation

- What we are weighting is the measurement residual, also called the **innovation**: $z(k) - \hat{x}(k)$

Kalman Gain Computation

- What we are weighting is the measurement residual, also called the **innovation**: $z(k) - \hat{x}(k)$
- The innovation contains all the new information, i.e., the new measurement, minus what we thought we already knew, i.e., the previous estimate

Kalman Gain Computation

- What we are weighting is the measurement residual, also called the **innovation**: $z(k) - \hat{x}(k)$
- The innovation contains all the new information, i.e., the new measurement, minus what we thought we already knew, i.e., the previous estimate
- The Kalman gain computation occurs after the extrapolation but before we update the estimate with measurement information

Kalman Gain Computation

- What we are weighting is the measurement residual, also called the **innovation**: $z(k) - \hat{x}(k)$
- The innovation contains all the new information, i.e., the new measurement, minus what we thought we already knew, i.e., the previous estimate
- The Kalman gain computation occurs after the extrapolation but before we update the estimate with measurement information
- The Kalman gain is also sometimes referred to as the 'Innovation gain'

Exercises

1. Let's re-familiarize ourselves with vectors and matrices. Addition:

$$\begin{bmatrix} x_2 \\ x_n \end{bmatrix} + \begin{bmatrix} y_2 \\ y_n \end{bmatrix} = \begin{bmatrix} x_1 + y_1 \\ x_2 + y_2 \\ x_n + y_n \end{bmatrix}$$

For vectors in three space, this would be:

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = x_3 + y_3$$

Compute:

$$\begin{bmatrix} -1 \\ 3 \\ 0 \end{bmatrix} + \begin{bmatrix} 7 \\ -1 \\ 5 \end{bmatrix} =$$

Exercises

$$\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 2 \\ 2 \\ 1 \end{bmatrix} =$$

$$\begin{bmatrix} 0.75 \\ -3 \\ 100 \end{bmatrix} + \begin{bmatrix} 0.75 \\ 0 \\ -1000 \end{bmatrix} =$$

Exercises

$$\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} =$$

$$\begin{bmatrix} 1.1 \\ -3.2 \\ 9.7 \end{bmatrix} + \begin{bmatrix} -0.8 \\ 3.2 \\ 10 \end{bmatrix} =$$

Exercises

2 Multiplying a row vector times a column vector:

$$\begin{bmatrix} a_1 & a_2 & a_3 \end{bmatrix} \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} = a_1 b_1 + a_2 b_2 + a_3 b_3$$

Now you try it:

$$\begin{bmatrix} 0 & 1 & 3 \end{bmatrix} \begin{bmatrix} 4 \\ 3 \\ 0 \end{bmatrix} =$$

$$\begin{bmatrix} -1 & 3 & -2 \end{bmatrix} \begin{bmatrix} 1 \\ -2 \\ 2 \end{bmatrix} =$$

Exercises

$$\begin{bmatrix} x & y & z \end{bmatrix} \begin{bmatrix} u \\ v \\ w \end{bmatrix} =$$

$$\begin{bmatrix} 2x & .5x & -7x \end{bmatrix} \begin{bmatrix} -10 \\ 4 \\ -1 \end{bmatrix} =$$

Exercises

3 Multiplying a matrix times a column vector:

$$\begin{bmatrix} 1 & 0 & 1 \\ 2 & 7 & 0 \\ 3 & 2 & 5 \end{bmatrix} \begin{bmatrix} 7 \\ -1 \\ 5 \end{bmatrix} =$$

Hint: Multiply the first row of the matrix times the column vector to get the first element of the new column vector. Then multiply by the second row of the matrix, and so on.

$$\begin{bmatrix} 2 & 1 & 0 \\ 0 & 1 & 1 \\ 2 & 2 & 2 \end{bmatrix} \begin{bmatrix} 3 \\ -3 \\ 1 \end{bmatrix} =$$

$$\begin{bmatrix} 3 & 1 & 1 \\ -1 & 1 & -1 \\ 0 & 2 & 2 \end{bmatrix} \begin{bmatrix} 3 \\ -3 \\ 1 \end{bmatrix} =$$

Exercises

4. What happens if you multiply a column on the left by a row vector on the right?
   item What is the transpose of the following matrix?

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

Exercises

5. Given the following scalar measurements of some constant scalar process, calculate the average value at each stage k by two different methods: 1) by averaging all past values and 2) by using the recursive algorithm in Eq. (2):
    1. $z_1 = 0.76$
    2. $z_2 = 0.67$
    3. $z_3 = 0.79$
    4. $z_4 = 0.81$
    5. $z_5 = 0.78$
    6. $z_6 = 0.79$
    7. $z_7 = 0.81$
    8. $z_8 = 0.82$
    9. $z_9 = 0.77$

Exercises

6. You are traveling in a car at 45 mph in a 35 mph speed zone. Your friendly neighborhood police officer is watching you with her radar gun. She is measuring your speed, of course, but needs to report as well your position as a function of time. Write down a dynamics and measurement model for this problem in which the state variable is your positional displacement from some arbitrary initial position. Assume that there is no process noise in your dynamics and that your velocity is not changing. Assume the policeman makes a measurement with her radar gun every 2 seconds.
   Hint: You have two variables of interest!
7. How is this problem different from the resistor problem?

Part 1

Equation Drilldown

Topics

- Understanding the Notation

Part 1

Equation Drilldown

Topics

- Understanding the Notation
- Examples

Part 1

Equation Drilldown

Topics

- Understanding the Notation
- Examples
- Exercises

Understanding the Notation

Kalman Filter Problem Summary

Understanding the Notation

Kalman Filter Problem Summary

- Given two models:

Understanding the Notation

Kalman Filter Problem Summary

- Given two models:
  - State Dynamics Model
    (sometimes called the System or Process Model)

Understanding the Notation

Kalman Filter Problem Summary

- Given two models:
  - State Dynamics Model
    (sometimes called the System or Process Model)
  - Measurement Model
    (sometimes called the Observation Model)

Understanding the Notation

Kalman Filter Problem Summary

- Given two models:
  - State Dynamics Model
    (sometimes called the System or Process Model)
  - Measurement Model
    (sometimes called the Observation Model)
- Find the optimal linear gain $K(k)$ with which to weight the measurement residual (or innovation); the weighted residual is then used to additively update the state estimate and the estimation error covariance (estimation uncertainty)

Understanding the Notation

Kalman Filter Problem Summary

General, Non-Linear, Time-Invariant, Discrete-Time Formulation

Given:

Understanding the Notation

Kalman Filter Problem Summary

General, Non-Linear, Time-Invariant, Discrete-Time Formulation

Given:

- State Dynamics Model: $x(k) = f[x(k-1)] + w(k)$

Understanding the Notation

Kalman Filter Problem Summary

General, Non-Linear, Time-Invariant, Discrete-Time Formulation

Given:

- State Dynamics Model: $x(k) = f[x(k-1)] + w(k)$
  - x=state, f=true dynamics function, w=process noise

Understanding the Notation

Kalman Filter Problem Summary

General, Non-Linear, Time-Invariant, Discrete-Time Formulation

Given:

- State Dynamics Model: $x(k) = f[x(k-1)] + w(k)$
  - x=state, f=true dynamics function, w=process noise
- Measurement Model: $z(k) = h[x(k)] + v(k)$

Understanding the Notation

Kalman Filter Problem Summary

General, Non-Linear, Time-Invariant, Discrete-Time Formulation

Given:

- State Dynamics Model: $x(k) = f[x(k-1)] + w(k)$
  - x=state, f=true dynamics function, w=process noise
- Measurement Model: $z(k) = h[x(k)] + v(k)$
  - z=measurement, h=measurement function, v=measurement noise

Understanding the Notation

Kalman Filter Problem Summary

General, Non-Linear, Time-Invariant, Discrete-Time Formulation

Given:

- State Dynamics Model: $x(k) = f[x(k-1)] + w(k)$
  - x=state, f=true dynamics function, w=process noise
- Measurement Model: $z(k) = h[x(k)] + v(k)$
  - z=measurement, h=measurement function, v=measurement noise

Find:

Understanding the Notation

Kalman Filter Problem Summary

General, Non-Linear, Time-Invariant, Discrete-Time Formulation

Given:

- State Dynamics Model: $x(k) = f[x(k-1)] + w(k)$
  - x=state, f=true dynamics function, w=process noise
- Measurement Model: $z(k) = h[x(k)] + v(k)$
  - z=measurement, h=measurement function, v=measurement noise

Find:

- Optimal gain $K(k)$ that allows us to update our state estimate in the following way:
  $\hat{x}(k) = f[\hat{x}(k-1)] + K(k)\,[\,z(k) - h(f[\hat{x}(k-1)])\,]$

Understanding the Notation

Kalman Filter Problem Summary

General, Non-Linear, Time-Invariant, Discrete-Time Formulation

Given:

- State Dynamics Model: $x(k) = f[x(k-1)] + w(k)$
  - x=state, f=true dynamics function, w=process noise
- Measurement Model: $z(k) = h[x(k)] + v(k)$
  - z=measurement, h=measurement function, v=measurement noise

Find:

- Optimal gain $K(k)$ that allows us to update our state estimate in the following way:
  $\hat{x}(k) = f[\hat{x}(k-1)] + K(k)\,[\,z(k) - h(f[\hat{x}(k-1)])\,]$
- We'll discuss the form of the covariance update later!

Understanding the Notation

Kalman Filter Problem Summary

Understanding the Notation

Kalman Filter Problem Summary

- Kalman filtering only deals with linear (or linearized) system dynamics

Understanding the Notation

Kalman Filter Problem Summary

- Kalman filtering only deals with linear (or linearized) system dynamics
  - Assume from now on that vector-valued function $f[\hat{x}(k-1)]$ is simply matrix multiplication, something like Φ
    (which we assume for now does not vary with time)

Understanding the Notation

Kalman Filter Problem Summary

- Kalman filtering only deals with linear (or linearized) system dynamics
  - Assume from now on that vector-valued function $f[\hat{x}(k-1)]$ is simply matrix multiplication, something like Φ
    (which we assume for now does not vary with time)
  - Also assume hat h is a linear, vector-valued function, i.e. a matrix H

Understanding the Notation

Kalman Filter Problem Summary

- Kalman filtering only deals with linear (or linearized) system dynamics
  - Assume from now on that vector-valued function $f[\hat{x}(k-1)]$ is simply matrix multiplication, something like Φ (which we assume for now does not vary with time)
  - Also assume hat h is a linear, vector-valued function, i.e. a matrix H
- Find a gain $K(k)$ in form of multiplier (i.e., a gain matrix)

Understanding the Notation

Kalman Filter Problem Summary

(Linear Discrete Formulation)

Given:

Understanding the Notation

Kalman Filter Problem Summary
(Linear Discrete Formulation)

Given:

- State Dynamics Model: $x(k) = \Phi x(k-1) + w(k)$

Understanding the Notation

Kalman Filter Problem Summary
(Linear Discrete Formulation)
Given:

- State Dynamics Model: $x(k) = \Phi x(k-1) + w(k)$
  - $x(k)$=state,Φ=state transition matrix, $w(k)$=process noise

Understanding the Notation

Kalman Filter Problem Summary
(Linear Discrete Formulation)

Given:

- State Dynamics Model: $x(k) = \Phi x(k-1) + w(k)$
  - $x(k)$=state,Φ=state transition matrix, $w(k)$=process noise
- Measurement Model: $z(k) = Hx(k) + v(k)$
  - $z(k)$=measurement,H=meas. matrix (assumed constant), $v(k)$=meas. noise

Understanding the Notation

Kalman Filter Problem Summary
(Linear Discrete Formulation)

Given:

- State Dynamics Model: $x(k) = \Phi x(k-1) + w(k)$
  - $x(k)$=state,Φ=state transition matrix, $w(k)$=process noise
- Measurement Model: $z(k) = Hx(k) + v(k)$
  - $z(k)$=measurement,H=meas. matrix (assumed constant), $v(k)$=meas. noise

Find:

Understanding the Notation

Kalman Filter Problem Summary
(Linear Discrete Formulation)

Given:

- State Dynamics Model: $x(k) = \Phi x(k-1) + w(k)$
  - $x(k)$=state,Φ=state transition matrix, $w(k)$=process noise
- Measurement Model: $z(k) = Hx(k) + v(k)$
  - $z(k)$=measurement,H=meas. matrix (assumed constant), $v(k)$=meas. noise

Find:

- State estimate in the form
  $\hat{x}(k) = \Phi\hat{x}(k-1) + K(k)[z(k) - H\Phi\hat{x}(k-1)]$

Understanding the Notation

Kalman Filter Problem Summary
(Linear Discrete Formulation)

Given:

- State Dynamics Model: $x(k) = \Phi x(k-1) + w(k)$
  - $x(k)$=state,Φ=state transition matrix, $w(k)$=process noise
- Measurement Model: $z(k) = Hx(k) + v(k)$
  - $z(k)$=measurement,H=meas. matrix (assumed constant), $v(k)$=meas. noise

Find:

- State estimate in the form $\hat{x}(k) = \Phi\hat{x}(k-1) + K(k)[z(k) - H\Phi\hat{x}(k-1)]$
- Note that $\Phi\hat{x}(k-1)$ extrapolates of the previous estimate to the current time

Understanding the Notation

Kalman Filter Problem Summary

Slight Notational Change

Understanding the Notation

Kalman Filter Problem Summary

Slight Notational Change

- The term $\Phi\hat{x}(k-1)$ is the extrapolated state estimate prior to making the measurement update or correction

Understanding the Notation

Kalman Filter Problem Summary

Slight Notational Change

- The term $\Phi\hat{x}(k-1)$ is the extrapolated state estimate prior to making the measurement update or correction
- To distinguish prior (extrapolated) estimate from updated estimate, we use the following notation:

$$\hat{x}^{-}(k) = \Phi\hat{x}^{+}(k-1) = \text{predicted estimate prior to update}$$

$$\hat{x}^{+}(k) = \text{updated state estimate}$$

Understanding the Notation

Kalman Filter Problem Summary

Understanding the Notation

Kalman Filter Problem Summary

- Thus, the solution we seek looks like this:

Understanding the Notation

Kalman Filter Problem Summary

- Thus, the solution we seek looks like this:

$$\hat{x}^+(k) = \hat{x}^-(k) + K(k)[z(k) - H\hat{x}^-(k)] \tag{3}$$

Understanding the Notation

Kalman Filter Problem Summary

- Thus, the solution we seek looks like this:

$$\hat{x}^+(k) = \hat{x}^-(k) + K(k)[z(k) - H\hat{x}^-(k)] \tag{3}$$

where $\hat{x}^-(k) = \Phi\hat{x}^+(k-1)$

Understanding the Notation

Kalman Filter Problem Summary

- Thus, the solution we seek looks like this:

$$\hat{x}^+(k) = \hat{x}^-(k) + K(k)[z(k) - H\hat{x}^-(k)] \tag{3}$$

  where $\hat{x}^-(k) = \Phi\hat{x}^+(k-1)$

- This now looks a lot like the running average problem!

Understanding the Notation

Kalman Filter Problem Summary

Understanding the Notation

Kalman Filter Problem Summary

- We've formulated the discrete problem

Understanding the Notation

Kalman Filter Problem Summary

- We've formulated the discrete problem
- We've indicated the form of the solution: the recursive estimating algorithm with the Kalman gain

Understanding the Notation

Kalman Filter Problem Summary

- We've formulated the discrete problem
- We've indicated the form of the solution: the recursive estimating algorithm with the Kalman gain
- But we haven't yet presented the general solution; only in the case of a simple, constant, scalar system

Understanding the Notation

Kalman Filter Problem Summary

- We've formulated the discrete problem
- We've indicated the form of the solution: the recursive estimating algorithm with the Kalman gain
- But we haven't yet presented the general solution; only in the case of a simple, constant, scalar system
- When we arrive at the general solution, we'll get an added benefit: While trying to come up with $\widehat{x}^+(k)$ and $K(k)$, we'll also end up with a recursive estimate of our estimation uncertainty! This is the so-called error covariance matrix.

Understanding Kalman Filter Notation

Drilldown on State Dynamics

Understanding Kalman Filter Notation

Drilldown on State Dynamics

- State Dynamics Model: $x(k) = \Phi x(k-1) + w(k)$

Understanding Kalman Filter Notation

Drilldown on State Dynamics

- State Dynamics Model: $x(k) = \Phi x(k-1) + w(k)$
- $x(k)$ is the **state vector** with n elements, written as a column vector:

Understanding Kalman Filter Notation

Drilldown on State Dynamics

- State Dynamics Model: $x(k) = \Phi x(k-1) + w(k)$
- $x(k)$ is the **state vector** with n elements, written as a column vector:
- $x(k) = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} (k) = \begin{bmatrix} x_1(k) \\ \vdots \\ x_n(k) \end{bmatrix}$

Understanding Kalman Filter Notation
Drilldown State Dynamics

Understanding Kalman Filter Notation
Drilldown State Dynamics

- Discrete State Dynamics

Understanding Kalman Filter Notation
Drilldown State Dynamics

- Discrete State Dynamics
- KF doesn't work for nonlinear dynamics function $f[x(k-1)]$

Understanding Kalman Filter Notation
Drilldown State Dynamics

- Discrete State Dynamics
- KF doesn't work for nonlinear dynamics function $f[x(k-1)]$
- Restriction to matrices: $x(k) = \Phi(k)x(k-1) + w(k)$

Understanding Kalman Filter Notation
Drilldown State Dynamics

- Discrete State Dynamics
- KF doesn't work for nonlinear dynamics function $f[x(k-1)]$
- Restriction to matrices: $x(k) = \Phi(k)x(k-1) + w(k)$
- $\Phi(k-1)$ is called the **state transition matrix**. It is $n \times n$ in size.

Understanding Kalman Filter Notation
Drilldown on State Dynamics

Understanding Kalman Filter Notation
Drilldown on State Dynamics

- State Transition Matrix: $\Phi(k-1) = \Phi(k, k-1)$

Understanding Kalman Filter Notation
Drilldown on State Dynamics

- State Transition Matrix: $\Phi(k-1) = \Phi(k, k-1)$
- Extrapolates state vector from time step $k-1$ to time step k (pulling from past to present)

Understanding Kalman Filter Notation
Drilldown on State Dynamics

- State Transition Matrix: $\Phi(k-1) = \Phi(k, k-1)$
- Extrapolates state vector from time step $k-1$ to time step k (pulling from past to present)
- Tells you how the state would evolve in the absence of forcing functions

Understanding Kalman Filter Notation
Drilldown on State Dynamics

- State Transition Matrix: $\Phi(k-1) = \Phi(k, k-1)$
- Extrapolates state vector from time step $k-1$ to time step k (pulling from past to present)
- Tells you how the state would evolve in the absence of forcing functions
- Remember: this is still just a model. We don't really know what the true state is, because the random forcing function (process noise) generates disturbances!

Understanding Kalman Filter Notation
Drilldown State Dynamics

Understanding Kalman Filter Notation
Drilldown State Dynamics

- We could throw in other factors and terms

Understanding Kalman Filter Notation
Drilldown State Dynamics

- We could throw in other factors and terms
  - A matrix $\Gamma(k)$ to distribute the Gaussian white noise to the states

Understanding Kalman Filter Notation
Drilldown State Dynamics

- We could throw in other factors and terms
  - A matrix $\Gamma(k)$ to distribute the Gaussian white noise to the states
  - An additional, deterministic control variable u_k, and associated distribution matrix $\Lambda(k)$

Understanding Kalman Filter Notation
Drilldown State Dynamics

- We could throw in other factors and terms
  - A matrix $\Gamma(k)$ to distribute the Gaussian white noise to the states
  - An additional, deterministic control variable u_k, and associated distribution matrix $\Lambda(k)$
- For simplicity we'll generally leave the control variable out of the picture

Understanding Kalman Filter Notation
Drilldown State Dynamics

- We could throw in other factors and terms
  - A matrix $\Gamma(k)$ to distribute the Gaussian white noise to the states
  - An additional, deterministic control variable u_k, and associated distribution matrix $\Lambda(k)$
- For simplicity we'll generally leave the control variable out of the picture
- We are deliberately trying to keep the notation as simple as possible

The Kalman Filter Solution: State Estimate Extrapolation

The Kalman Filter Solution: State Estimate Extrapolation

- Given an estimate $\widehat{x}^+(k-1)$ at time $k-1$, we extrapolate it forward in time by the state transition matrix:

$$\widehat{x}^-(k) = \Phi(k)\widehat{x}^+(k-1) \tag{4}$$

The Kalman Filter Solution: State Estimate Extrapolation

- Given an estimate $\widehat{x}^+(k-1)$ at time $k-1$, we extrapolate it forward in time by the state transition matrix:

$$\widehat{x}^-(k) = \Phi(k)\widehat{x}^+(k-1) \tag{4}$$

- Q: Why isn't the white noise sequence w included in this extrapolation?

The Kalman Filter Solution: State Estimate Extrapolation

- Given an estimate $\widehat{x}^+(k-1)$ at time $k-1$, we extrapolate it forward in time by the state transition matrix:

$$\widehat{x}^-(k) = \Phi(k)\widehat{x}^+(k-1) \tag{4}$$

- Q: Why isn't the white noise sequence w included in this extrapolation?
  A: Because the white noise forcing function is a **zero-mean** process. Thus, on average, the equation is homogeneous and we should therefore use the homogeneous solution, i.e., the state transition matrix.

The Kalman Filter Solution: Covariance Extrapolation

The Kalman Filter Solution: Covariance Extrapolation

- Q: How certain are we that our solution is right?

The Kalman Filter Solution: Covariance Extrapolation

- Q: How certain are we that our solution is right?
  A: The answer is given by the **estimation error covariance matrix** or often just called the covariance matrix $P(k)$

The Kalman Filter Solution: Covariance Extrapolation

- Q: How certain are we that our solution is right?
  A: The answer is given by the **estimation error covariance matrix** or often just called the covariance matrix $P(k)$
- Just like the state estimate, it is propagated forward in time before being corrected:

The Kalman Filter Solution: Covariance Extrapolation

- Q: How certain are we that our solution is right?
  A: The answer is given by the **estimation error covariance matrix** or often just called the covariance matrix $P(k)$
- Just like the state estimate, it is propagated forward in time before being corrected:

$$P^{-}(k) = \Phi(k)P^{+}(k)\Phi(k)^{\top} + Q(k) \tag{5}$$

  where Q_k is a matrix called the **process noise covariance matrix**

The Kalman Filter Solution: Covariance Extrapolation

- Q: How certain are we that our solution is right?
  A: The answer is given by the **estimation error covariance matrix** or often just called the covariance matrix $P(k)$
- Just like the state estimate, it is propagated forward in time before being corrected:

$$P^{-}(k) = \Phi(k)P^{+}(k)\Phi(k)^{\top} + Q(k) \tag{5}$$

  where Q_k is a matrix called the **process noise covariance matrix**
- Q_k measures the amount of dispersion in the white noise forcing function $w(k)$

The Kalman Filter Solution: Covariance Extrapolation

- Q: How certain are we that our solution is right?
  A: The answer is given by the **estimation error covariance matrix** or often just called the covariance matrix $P(k)$
- Just like the state estimate, it is propagated forward in time before being corrected:

$$P^{-}(k) = \Phi(k)P^{+}(k)\Phi(k)^{\top} + Q(k) \tag{5}$$

  where Q_k is a matrix called the **process noise covariance matrix**
- Q_k measures the amount of dispersion in the white noise forcing function $w(k)$
- Later we'll learn how to define $P(k)$ in terms of the expected error in the estimation $\tilde{x}(k) = x(k) - \widehat{x}^{+}(k)$

The Kalman Filter Solution: Covariance Extrapolation

The Kalman Filter Solution: Covariance Extrapolation

- What does Eq. (5) really mean?

The Kalman Filter Solution: Covariance Extrapolation

- What does Eq. (5) really mean?
- The term $\Phi(k)P^{+}(k)\Phi(k)^{\top}$ represents the effect that the state dynamics has on the estimation error covariance

The Kalman Filter Solution: Covariance Extrapolation

- What does Eq. (5) really mean?
- The term $\Phi(k)P^{+}(k)\Phi(k)^{\top}$ represents the effect that the state dynamics has on the estimation error covariance
  - P(k) is positive definite matrix and Φ tends to increase its norm

The Kalman Filter Solution: Covariance Extrapolation

- What does Eq. (5) really mean?
- The term $\Phi(k)P^{+}(k)\Phi(k)^{\top}$ represents the effect that the state dynamics has on the estimation error covariance
  - P(k) is positive definite matrix and Φ tends to increase its norm
- The other term Q_k represents the increased uncertainty added into each step due to the process noise inherent in the system

The Kalman Filter Solution: The Kalman Gain

The Kalman Filter Solution: The Kalman Gain

- The heart of the solution is the **Kalman Gain**: $K(k)$

The Kalman Filter Solution: The Kalman Gain

- The heart of the solution is the **Kalman Gain**: $K(k)$

$$K(k) = P^{-}(k)H(k)^{\top}\left[H(k)P^{-}(k)H(k)^{\top} + R(k)\right]^{-1} \quad (6)$$

where R_k is the measurement noise covariance matrix (to be defined later) governing the white measurement noise v_k in the measurement model:

$$z(k) = H(k)x(k) + v(k)$$

The Kalman Filter Solution: State Update Equation

The Kalman Filter Solution: State Update Equation

- Using the gain, the state update equation (which we've already seen) is

$$\widehat{x}^{+}(k) = \widehat{x}^{-}(k) + K(k)\left[z(k) - H(k)\widehat{x}^{-}(k)\right] \tag{7}$$

where z_k is the measurement and H_k is the measurment matrix at time k.

The Kalman Filter Solution: The Covariance Update Equation

The Kalman Filter Solution: The Covariance Update Equation

- Likewise, the error covariance $P(k)$ is corrected using the gain:

$$P^{+}(k) = [I - K(k)H(k)]\, P^{-}(k) \tag{8}$$

The Kalman Filter Solution: The Covariance Update Equation

- Likewise, the error covariance $P(k)$ is corrected using the gain:

$$P^{+}(k) = [I - K(k)H(k)]\, P^{-}(k) \tag{8}$$

- This is not a good form to use in numerical work!

The Kalman Filter Solution: The Covariance Update Equation

- Likewise, the error covariance $P(k)$ is corrected using the gain:

$$P^{+}(k) = [I - K(k)H(k)]\, P^{-}(k) \tag{8}$$

- This is not a good form to use in numerical work!
- Better to use the Joseph form:

The Kalman Filter Solution: The Covariance Update Equation

- Likewise, the error covariance $P(k)$ is corrected using the gain:

$$P^{+}(k) = [I - K(k)H(k)]\, P^{-}(k) \tag{8}$$

- This is not a good form to use in numerical work!
- Better to use the Joseph form:

$$P^{+}(k) = [I - K(k)H(k)]\, P^{-}(k)\, [I - K(k)H(k)]^{\top} + K(k)R(k)K(k)^{\top}$$

The Kalman Filter Solution: The Covariance Update Equation

- Likewise, the error covariance $P(k)$ is corrected using the gain:

$$P^{+}(k) = [I - K(k)H(k)]\, P^{-}(k) \tag{8}$$

- This is not a good form to use in numerical work!
- Better to use the Joseph form:

$$P^{+}(k) = [I - K(k)H(k)]\, P^{-}(k)\, [I - K(k)H(k)]^{\top} + K(k)R(k)K(k)^{\top}$$

  - Better preserves symmetry of P

The Five Kalman Equations

State Extrapolation: $\widehat{x}^{-}(k) = \Phi(k)\widehat{x}^{+}(k-1)$ (9)

The Five Kalman Equations

State Extrapolation: $\widehat{x}^{-}(k) = \Phi(k)\widehat{x}^{+}(k-1)$ (9)

Covariance Extrapolation: $P^{-}(k) = \Phi(k)P^{+}(k)\Phi(k)^{\top} + Q(k)$ (10)

The Five Kalman Equations

State Extrapolation: $\widehat{x}^{-}(k) = \Phi(k)\widehat{x}^{+}(k-1)$ (9)

Covariance Extrapolation: $P^{-}(k) = \Phi(k)P^{+}(k)\Phi(k)^{\top} + Q(k)$ (10)

Kalman Gain: $K(k) = P^{-}(k)H(k)^{\top}\left[H(k)P^{-}(k)H(k)^{\top} + R(k)\right]^{-1}$ (11)

The Five Kalman Equations

State Extrapolation: $\widehat{x}^{-}(k) = \Phi(k)\widehat{x}^{+}(k-1)$ (9)

Covariance Extrapolation: $P^{-}(k) = \Phi(k)P^{+}(k)\Phi(k)^{\top} + Q(k)$ (10)

Kalman Gain: $K(k) = P^{-}(k)H(k)^{\top}\left[H(k)P^{-}(k)H(k)^{\top} + R(k)\right]^{-1}$ (11)

State Update: $\widehat{x}^{+}(k) = \widehat{x}^{-}(k) + K(k)\left[z(k) - H(k)\widehat{x}^{-}(k)\right]$ (12)

The Five Kalman Equations

State Extrapolation: $\widehat{x}^-(k) = \Phi(k)\widehat{x}^+(k-1)$ (9)

Covariance Extrapolation: $P^-(k) = \Phi(k)P^+(k)\Phi(k)^\top + Q(k)$ (10)

Kalman Gain: $K(k) = P^-(k)H(k)^\top \left[H(k)P^-(k)H(k)^\top + R(k)\right]^{-1}$ (11)

State Update: $\widehat{x}^+(k) = \widehat{x}^-(k) + K(k)\left[z(k) - H(k)\widehat{x}^-(k)\right]$ (12)

Covariance Update: $P^+(k) = [I - K(k)H(k)]\,P^-(k)$ (13)

or

$$P^+(k) = [I - K(k)H(k)]\,P^-(k)[I - K(k)H(k)]^\top + K(k)R(k)K(k)^\top$$

Definitions for the Five Equations

Definitions for the Five Equations

- $P^+(k) = \mathrm{E}\left[\tilde{x}(k)^+(k)[\tilde{x}(k)^+(k)]^\top\right]$ is the estimation error covariance matrix
  Often just referred to as the 'covariance matrix'

Definitions for the Five Equations

- $P^+(k) = \mathrm{E}\left[\tilde{x}(k)^+(k)[\tilde{x}(k)^+(k)]^\top\right]$ is the estimation error covariance matrix
  Often just referred to as the 'covariance matrix'
- $\tilde{x}(k) = x(k) - \widehat{x}^+(k)$ is the estimation error

Definitions for the Five Equations

- $P^+(k) = \mathrm{E}\left[\tilde{x}(k)^+(k)[\tilde{x}(k)^+(k)]^\top\right]$ is the estimation error covariance matrix
  Often just referred to as the 'covariance matrix'
- $\tilde{x}(k) = x(k) - \widehat{x}^+(k)$ is the estimation error
- $Q(k) := \mathrm{E}\left[w(k)w(k)^\top\right]$ is the process noise covariance matrix

Definitions for the Five Equations

- $P^+(k) = \mathrm{E}\left[\tilde{x}(k)^+(k)[\tilde{x}(k)^+(k)]^\top\right]$ is the estimation error covariance matrix
  Often just referred to as the 'covariance matrix'
- $\tilde{x}(k) = x(k) - \widehat{x}^+(k)$ is the estimation error
- $Q(k) := \mathrm{E}\left[w(k)w(k)^\top\right]$ is the process noise covariance matrix
- $R(k) := \mathrm{E}\left[v(k)v(k)^\top\right]$ is the measurement noise covariance matrix

Examples

Examples

- The MATLAB files

Examples

- The MATLAB files
- Scalar Example: Resistor Revisited

Examples

- The MATLAB files
- Scalar Example: Resistor Revisited
- Non-accelerated Motion (constant velocity)

Examples

- The MATLAB files
- Scalar Example: Resistor Revisited
- Non-accelerated Motion (constant velocity)
- Damped Harmonic Oscillator

About the Examples

About the Examples

- Discrete-time systems

About the Examples

- Discrete-time systems
- Gradually increasing complexity:

About the Examples

- Discrete-time systems
- Gradually increasing complexity:
  - Resistor: 1-state, static (no dynamics)

About the Examples

- Discrete-time systems
- Gradually increasing complexity:
  - Resistor: 1-state, static (no dynamics)
  - Non-accelerated Motion: 2-state, but one of the state static

About the Examples

- Discrete-time systems
- Gradually increasing complexity:
  - Resistor: 1-state, static (no dynamics)
  - Non-accelerated Motion: 2-state, but one of the state static
  - Damped Harmonic Oscillator: 2-state, with both states dynamic

The MATLAB files:
`kf_resistor_demo.m`, `constant_velocity.m`, `damposc.m`

The MATLAB files: kf_resistor_demo.m, constant_velocity.m, damposc.m

- File located here: TBD

The MATLAB files: `kf_resistor_demo.m`, `constant_velocity.m`, `damposc.m`

- File located here: TBD
- Copy them to your MATLAB folder and play with them!

The MATLAB files: `kf_resistor_demo.m`, `constant_velocity.m`, `damposc.m`

- File located here: TBD
- Copy them to your MATLAB folder and play with them!
  - Modify the initial conditions, number of samples

The MATLAB files: `kf_resistor_demo.m`, `constant_velocity.m`, `damposc.m`

- File located here: TBD
- Copy them to your MATLAB folder and play with them!
  - Modify the initial conditions, number of samples
  - Modify the R and Q noise parameters

Scalar Example: Resistor Revisited

Scalar Example: Resistor Revisited

- Number of states: 1

Scalar Example: Resistor Revisited

- Number of states: 1
- System Dynamics: $x(k) = x(k-1)$

Scalar Example: Resistor Revisited

- Number of states: 1
- System Dynamics: $x(k) = x(k-1)$
- Measurement Model: $z(k) = x(k) + v(k)$, where $v(k)$ is zero mean, Gaussian white noise with variance σ^2

Scalar Example: Resistor Revisited

- Number of states: 1
- System Dynamics: $x(k) = x(k-1)$
- Measurement Model: $z(k) = x(k) + v(k)$, where $v(k)$ is zero mean, Gaussian white noise with variance σ^2
- Thus, $\Phi = 1$, $H_k = 1$, $Q_k = 0$, $R_k = \sigma^2$

Scalar Example: Resistor Revisited

Scalar Example: Resistor Revisited

- State extrapolation and covariance extrapolation are simple:

$$\widehat{x}^{-}(k) = \widehat{x}^{+}(k-1)$$

$$P^{-}(k) = P^{+}(k)$$

where P is a 1×1 matrix or scalar.

Scalar Example: Resistor Revisited

- State extrapolation and covariance extrapolation are simple:

$$\widehat{x}^{-}(k) = \widehat{x}^{+}(k-1)$$

$$P^{-}(k) = P^{+}(k)$$

  where P is a 1×1 matrix or scalar.

- The Kalman gain equation
  $K(k) = P^{-}(k)H(k)^{\top}\left[H(k)P^{-}(k)H(k)^{\top} + R(k)\right]^{-1}$
  becomes:

$$K_k = P^{-}(k)\left(P^{-}(k) + \sigma^2\right)^{-1}$$

Scalar Example: Resistor Revisited

Scalar Example: Resistor Revisited

- State update equation
  $\widehat{x}^+(k) = \widehat{x}^-(k) + K(k)\left[z(k) - H(k)\widehat{x}^-(k)\right]$ becomes

$$\widehat{x}^+(k) = \widehat{x}^-(k) + \frac{P^-(k)}{P^-(k) + \sigma^2}\left(z(k) - \widehat{x}^-(k)\right)$$

Scalar Example: Resistor Revisited

- State update equation
  $\widehat{x}^+(k) = \widehat{x}^-(k) + K(k)\left[z(k) - H(k)\widehat{x}^-(k)\right]$ becomes

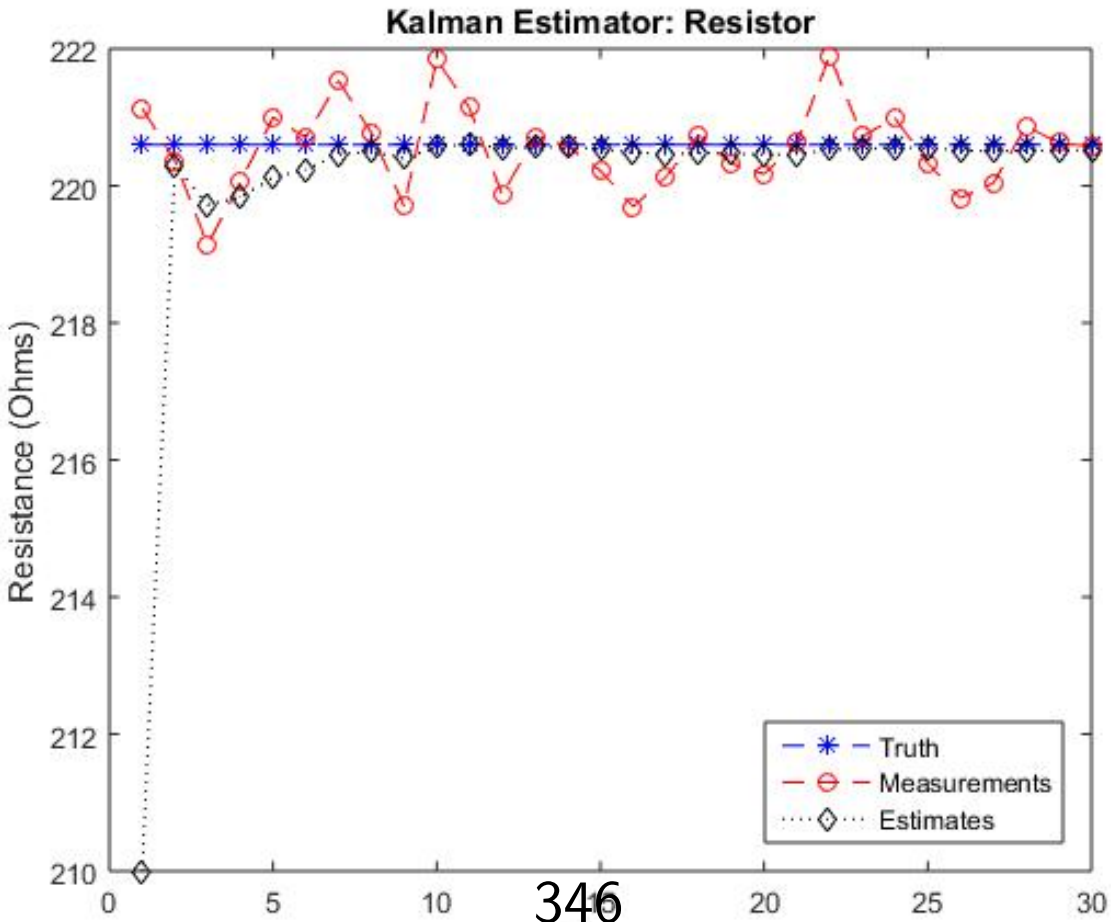
$$\widehat{x}^+(k) = \widehat{x}^-(k) + \frac{P^-(k)}{P^-(k) + \sigma^2}\left(z(k) - \widehat{x}^-(k)\right)$$

- Covariance update equation $P^+(k) = \left[I - K(k)H(k)\right]P^-(k)$ becomes

$$P^+(k) = \left(1 - \frac{P^-(k)}{P^-(k) + \sigma^2}\right)P^-(k)$$

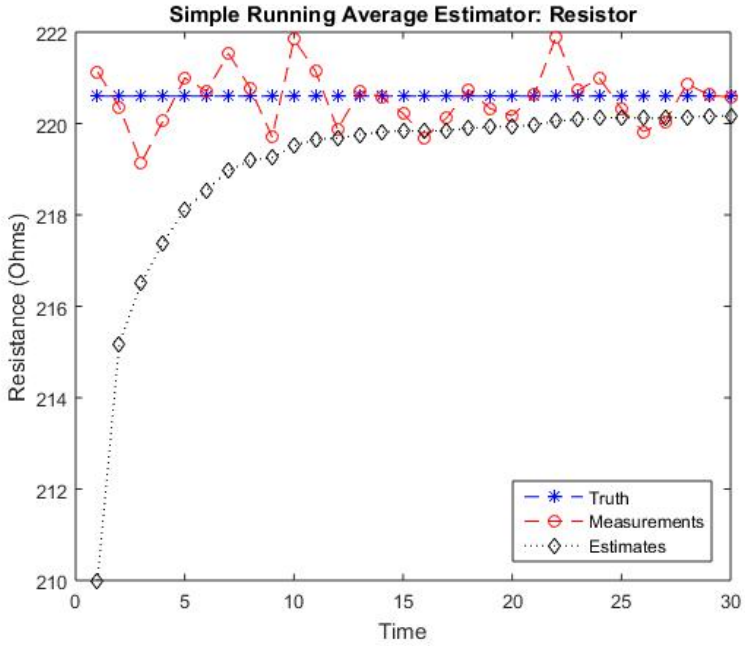
Scalar Example Resistor Revisited: Same problem and parameters,
but using Kalman filter instead of a running average:

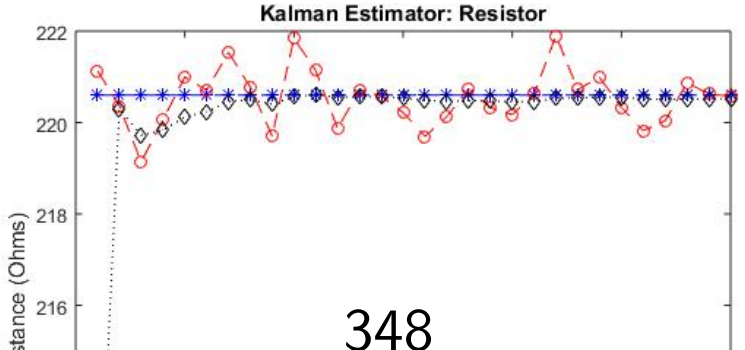
Scalar Example Resistor Revisited: Same problem and parameters,
but using Kalman filter instead of a running average:



Static, Scalar Example: Resistor Revisited: Comparing KF and Running Average

Static, Scalar Example: Resistor Revisited: Comparing KF and Running Average



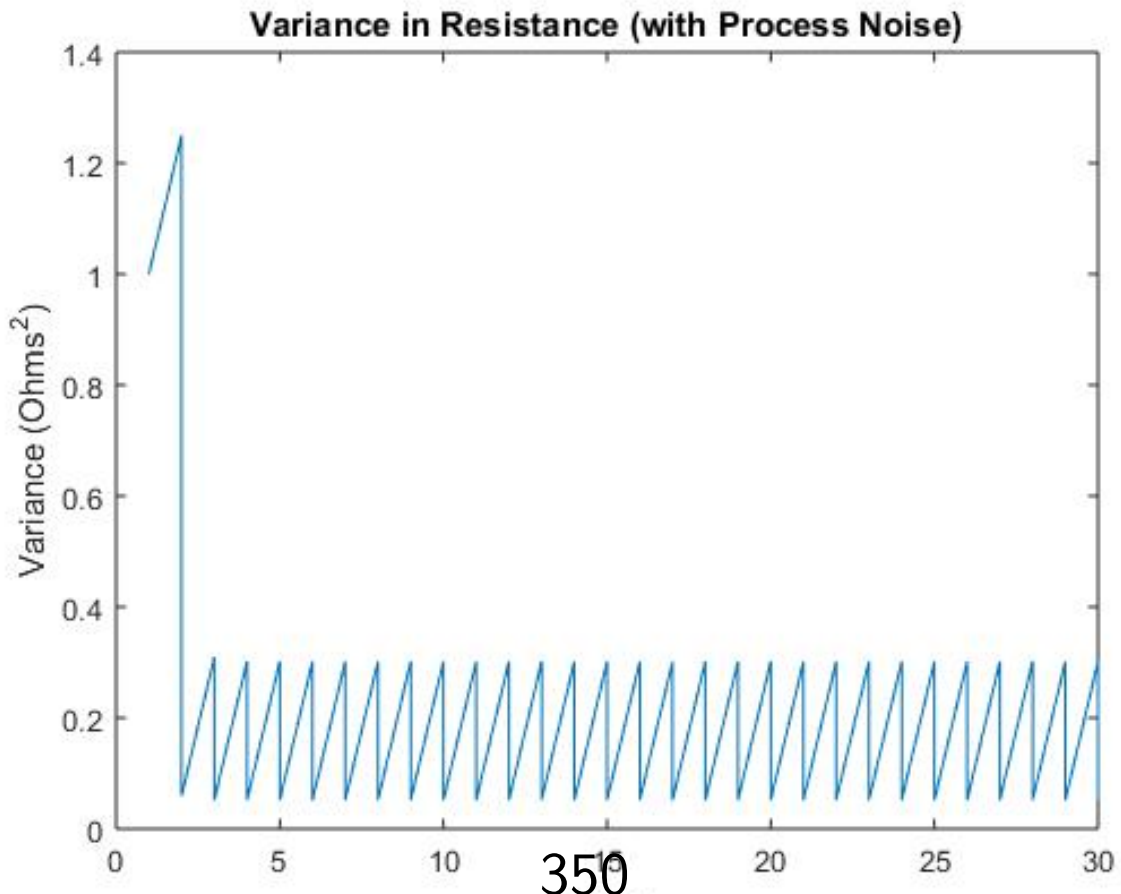


Kalman Sawtooth Plots

By interleaving the pre- and post-update elements of P over a run (typically only the diagonal elements), we can obtain Kalman sawtooth plots.

Kalman Sawtooth Plots

By interleaving the pre- and post-update elements of P over a run (typically only the diagonal elements), we can obtain Kalman sawtooth plots.

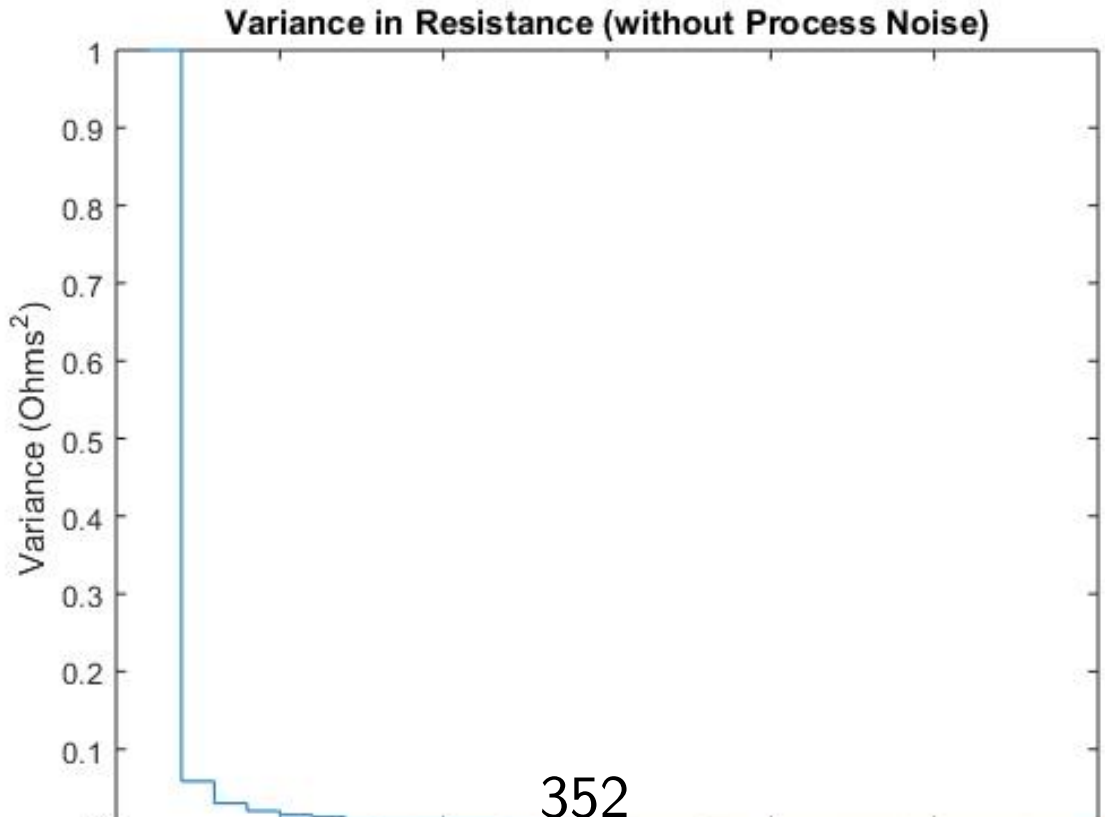


Kalman Sawtooth Plots

These plots show both the growth in uncertainty (upticks) upon extrapolation of dynamics and the reduction in uncertainty (downticks) after measurement updates. Without process noise, there are no upticks:

Kalman Sawtooth Plots

These plots show both the growth in uncertainty (upticks) upon extrapolation of dynamics and the reduction in uncertainty (downticks) after measurement updates. Without process noise, there are no upticks:



Non-Accelerated Motion: Constant Velocity

Non-Accelerated Motion: Constant Velocity

- Number of states: 2

Non-Accelerated Motion: Constant Velocity

- Number of states: 2
- System Dynamics:

$$x_1(k) = x_1(k-1) + x_2(k-1)\Delta t$$
$$x_2(k) = x_2(k-1)$$

Non-Accelerated Motion: Constant Velocity

- Number of states: 2
- System Dynamics:

$$x_1(k) = x_1(k-1) + x_2(k-1)\Delta t$$
$$x_2(k) = x_2(k-1)$$

- Measurement Model: $z(k) = x_1(k) + v(k)$, where $v(k)$ is zero mean, Gaussian white noise with variance σ^2

Non-Accelerated Motion: Constant Velocity

- Number of states: 2
- System Dynamics:

$$x_1(k) = x_1(k-1) + x_2(k-1)\Delta t$$
$$x_2(k) = x_2(k-1)$$

- Measurement Model: $z(k) = x_1(k) + v(k)$, where $v(k)$ is zero mean, Gaussian white noise with variance σ^2
- Thus, $\Phi = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}$, $\Delta t = t_k - t_{k-1}$ (assumed constant), $H_k = \begin{bmatrix} 1 & 0 \end{bmatrix}$, $Q_k = 0$, $R_k = \sigma^2$

Non-Accelerated Motion: Constant Velocity

- Number of states: 2
- System Dynamics:

$$x_1(k) = x_1(k-1) + x_2(k-1)\Delta t$$
$$x_2(k) = x_2(k-1)$$

- Measurement Model: $z(k) = x_1(k) + v(k)$, where $v(k)$ is zero mean, Gaussian white noise with variance σ^2
- Thus, $\Phi = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}$, $\Delta t = t_k - t_{k-1}$ (assumed constant), $H_k = \begin{bmatrix} 1 & 0 \end{bmatrix}$, $Q_k = 0$, $R_k = \sigma^2$
- Note that we are assuming the acceleration is truly zero. Any uncertainty in that assumption could be dealt with by adding a very small process noise to the velocity component.

Non-accelerated Motion (constant velocity): Covariance Extrapolation

Non-accelerated Motion (constant velocity): Covariance Extrapolation

- $P^-(k) = \Phi(k)P^+(k)\Phi(k)^\top + Q(k)$ becomes:

Non-accelerated Motion (constant velocity): Covariance Extrapolation

- $P^-(k) = \Phi(k)P^+(k)\Phi(k)^\top + Q(k)$ becomes:

$$P^-(k) = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} p_{11} & p_{12} \\ p_{21} & p_{22} \end{bmatrix} \begin{bmatrix} 1 & 0 \\ \Delta t & 1 \end{bmatrix}$$

Non-accelerated Motion (constant velocity): Covariance Extrapolation

- $P^-(k) = \Phi(k)P^+(k)\Phi(k)^\top + Q(k)$ becomes:

$$
\begin{aligned}
P^-(k) &= \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} p_{11} & p_{12} \\ p_{21} & p_{22} \end{bmatrix} \begin{bmatrix} 1 & 0 \\ \Delta t & 1 \end{bmatrix} \\
&= \begin{bmatrix} p_{11} + p_{21}\Delta t & p_{12} + p_{22}\Delta t \\ p_{21} & p_{22} \end{bmatrix} \begin{bmatrix} 1 & 0 \\ \Delta t & 1 \end{bmatrix}
\end{aligned}
$$

Non-accelerated Motion (constant velocity): Covariance Extrapolation

- $P^-(k) = \Phi(k)P^+(k)\Phi(k)^\top + Q(k)$ becomes:

$$
\begin{aligned}
P^-(k) &= \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} p_{11} & p_{12} \\ p_{21} & p_{22} \end{bmatrix} \begin{bmatrix} 1 & 0 \\ \Delta t & 1 \end{bmatrix} \\
&= \begin{bmatrix} p_{11} + p_{21}\Delta t & p_{12} + p_{22}\Delta t \\ p_{21} & p_{22} \end{bmatrix} \begin{bmatrix} 1 & 0 \\ \Delta t & 1 \end{bmatrix} \\
&= \begin{bmatrix} p_{11} + (p_{12} + p_{21})\,\Delta t + p_{22}\Delta t^2 & p_{12} + p_{22}\Delta t \\ p_{12} + p_{22}\Delta t & p_{22} \end{bmatrix}
\end{aligned}
$$

Non-accelerated Motion (constant velocity): Covariance Extrapolation

- $P^-(k) = \Phi(k)P^+(k)\Phi(k)^\top + Q(k)$ becomes:

$$
\begin{aligned}
P^-(k) &= \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} p_{11} & p_{12} \\ p_{21} & p_{22} \end{bmatrix} \begin{bmatrix} 1 & 0 \\ \Delta t & 1 \end{bmatrix} \\
&= \begin{bmatrix} p_{11} + p_{21}\Delta t & p_{12} + p_{22}\Delta t \\ p_{21} & p_{22} \end{bmatrix} \begin{bmatrix} 1 & 0 \\ \Delta t & 1 \end{bmatrix} \\
&= \begin{bmatrix} p_{11} + (p_{12} + p_{21})\,\Delta t + p_{22}\Delta t^2 & p_{12} + p_{22}\Delta t \\ p_{12} + p_{22}\Delta t & p_{22} \end{bmatrix}
\end{aligned}
$$

- Note that we've decluttered by suppressing the +/- superscripts and k's on the right-hand sides

Non-accelerated Motion (constant velocity): Covariance Extrapolation

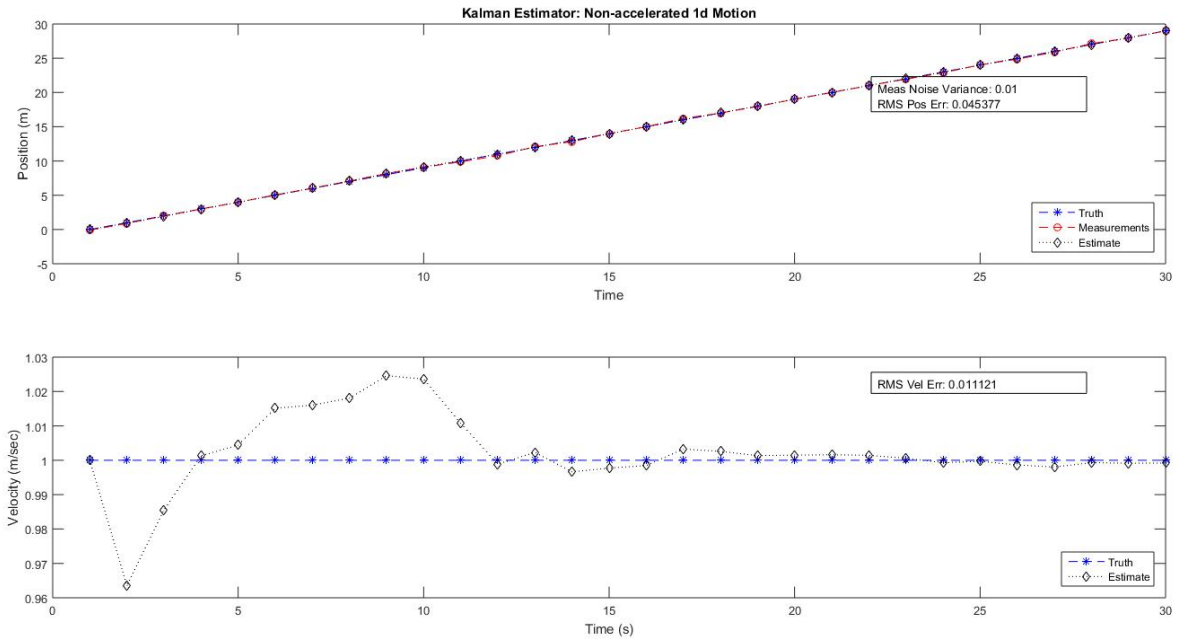
- $P^-(k) = \Phi(k)P^+(k)\Phi(k)^\top + Q(k)$ becomes:

$$
\begin{aligned}
P^-(k) &= \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} p_{11} & p_{12} \\ p_{21} & p_{22} \end{bmatrix} \begin{bmatrix} 1 & 0 \\ \Delta t & 1 \end{bmatrix} \\
&= \begin{bmatrix} p_{11} + p_{21}\Delta t & p_{12} + p_{22}\Delta t \\ p_{21} & p_{22} \end{bmatrix} \begin{bmatrix} 1 & 0 \\ \Delta t & 1 \end{bmatrix} \\
&= \begin{bmatrix} p_{11} + (p_{12} + p_{21})\,\Delta t + p_{22}\Delta t^2 & p_{12} + p_{22}\Delta t \\ p_{12} + p_{22}\Delta t & p_{22} \end{bmatrix}
\end{aligned}
$$

- Note that we've decluttered by suppressing the +/- superscripts and k's on the right-hand sides
- The Kalman gain computation is left as an exercise!

Non-accelerated Motion (constant velocity): Filter Performance (without Process Noise)

Non-accelerated Motion (constant velocity): Filter Performance (without Process Noise)



Non-accelerated Motion (constant velocity): Filter Sawtooth

Note uptick in uncertainty even without process noise, due to $\Phi \neq I$ in covariance extrapolation; causes increase in uncertainty:

Non-accelerated Motion (constant velocity): Filter Sawtooth

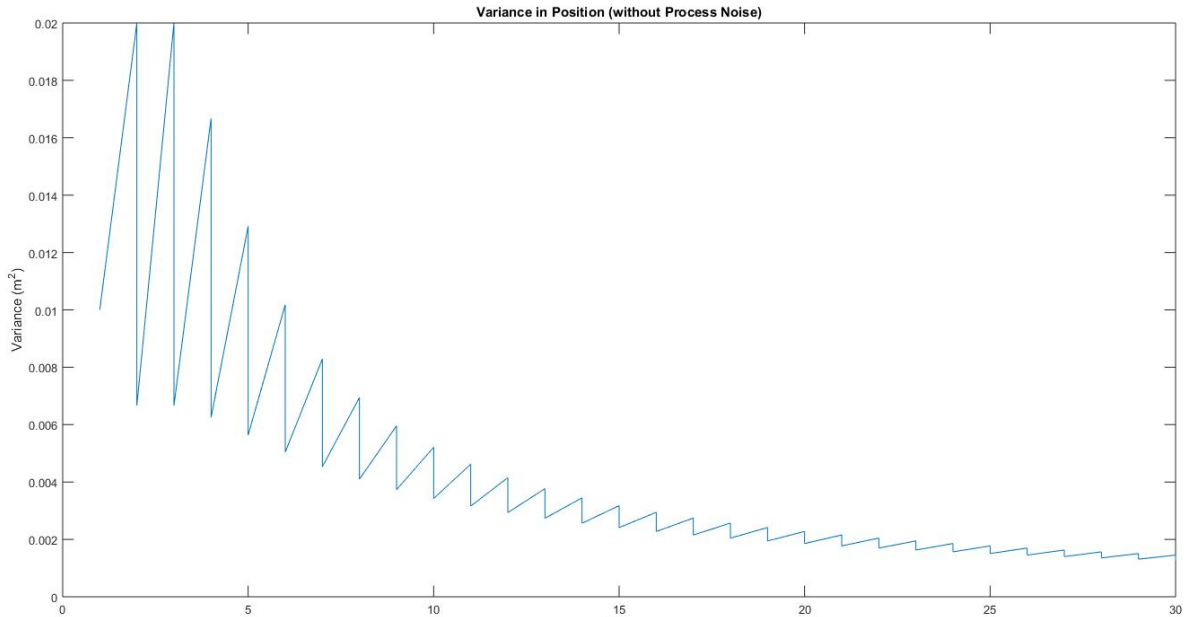
Note uptick in uncertainty even without process noise, due to $\Phi \neq I$ in covariance extrapolation; causes increase in uncertainty:

$$P^{-}(k) = \Phi(k)P^{+}(k)\Phi(k)^{\top} + Q(k)$$

Non-accelerated Motion (constant velocity): Filter Sawtooth

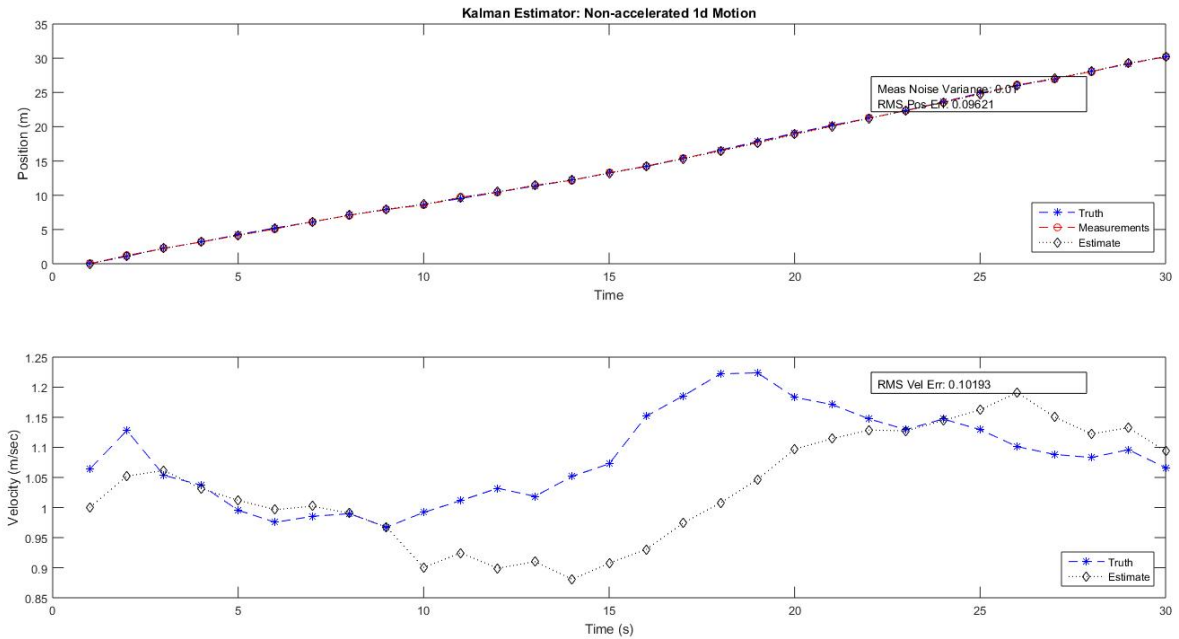
Note uptick in uncertainty even without process noise, due to $\Phi \neq I$ in covariance extrapolation; causes increase in uncertainty:

$$P^{-}(k) = \Phi(k)P^{+}(k)\Phi(k)^{\top} + Q(k)$$



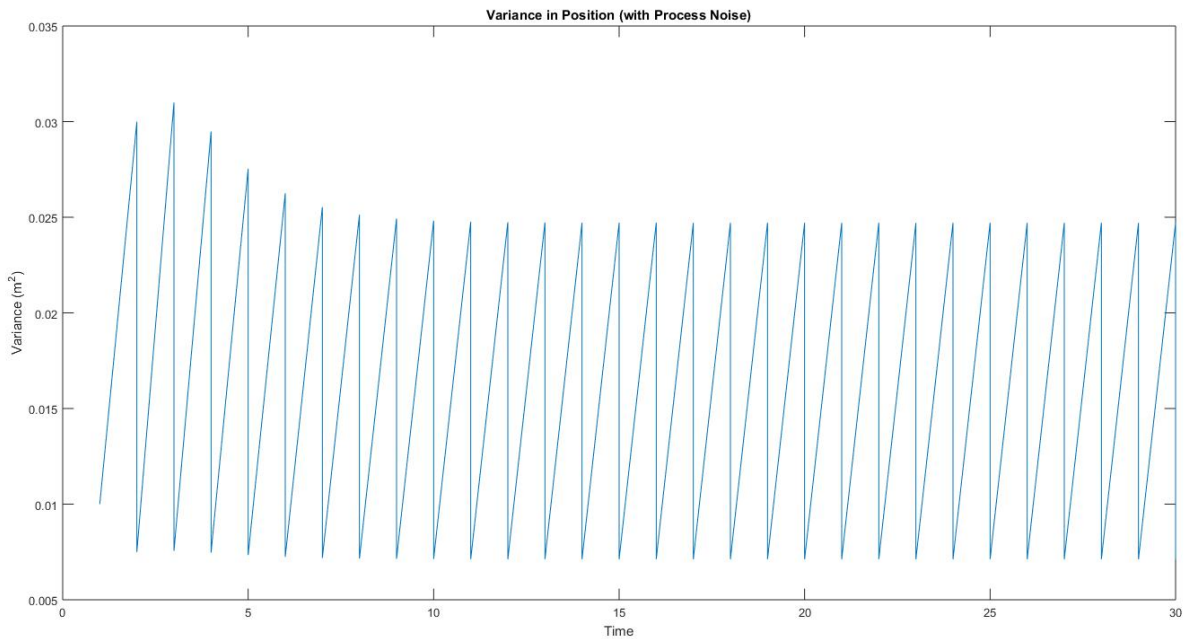
Non-accelerated Motion (constant velocity): Filter Performance (with Process Noise)

Non-accelerated Motion (constant velocity): Filter Performance (with Process Noise)



Non-accelerated Motion (constant velocity): Filter Sawtooth with Process Noise

Non-accelerated Motion (constant velocity): Filter Sawtooth with Process Noise

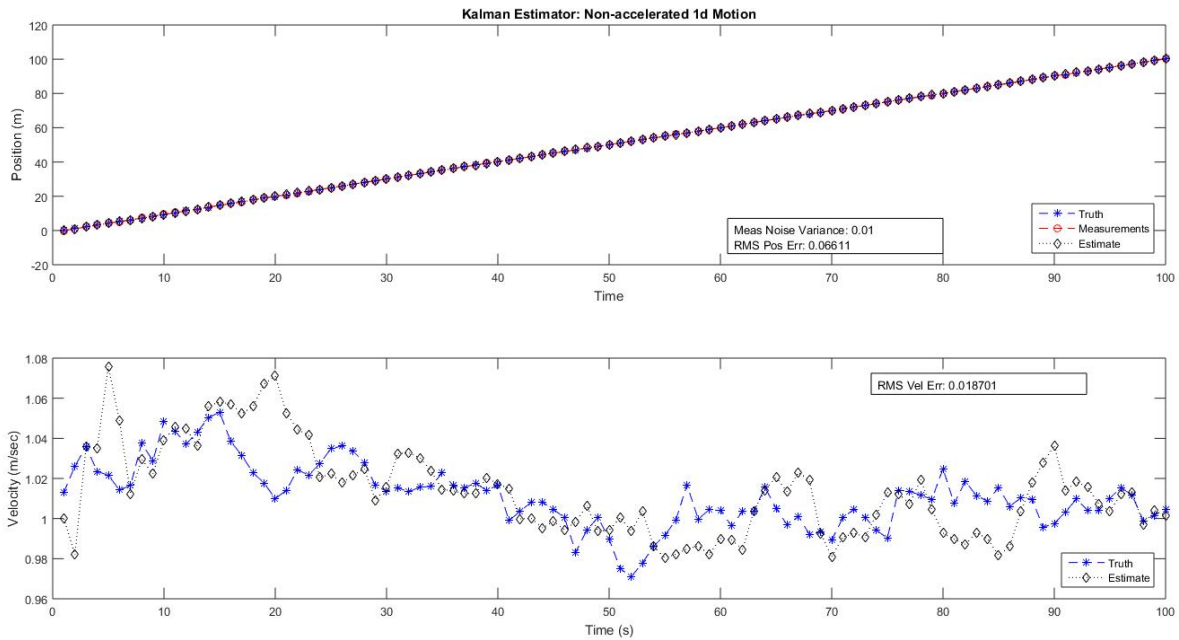


Non-accelerated Motion (constant velocity): Filter Performance (100 samples; less Process Noise)

Position Process Noise: 0.001; Velocity Process Noise: 0.0001

Non-accelerated Motion (constant velocity): Filter Performance (100 samples; less Process Noise)

Position Process Noise: 0.001; Velocity Process Noise: 0.0001

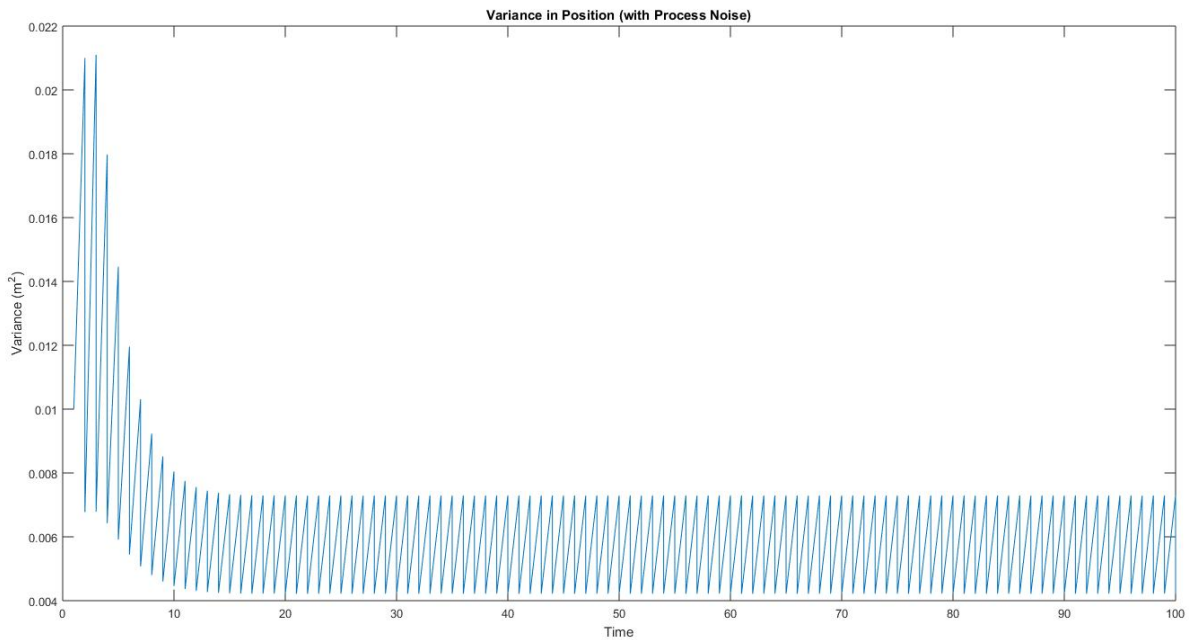


Non-accelerated Motion (constant velocity): Filter Sawtooth with Process Noise (100 samples; less Process Noise)

Position Process Noise: 0.001; Velocity Process Noise: 0.0001

Non-accelerated Motion (constant velocity): Filter Sawtooth with Process Noise (100 samples; less Process Noise)

Position Process Noise: 0.001; Velocity Process Noise: 0.0001

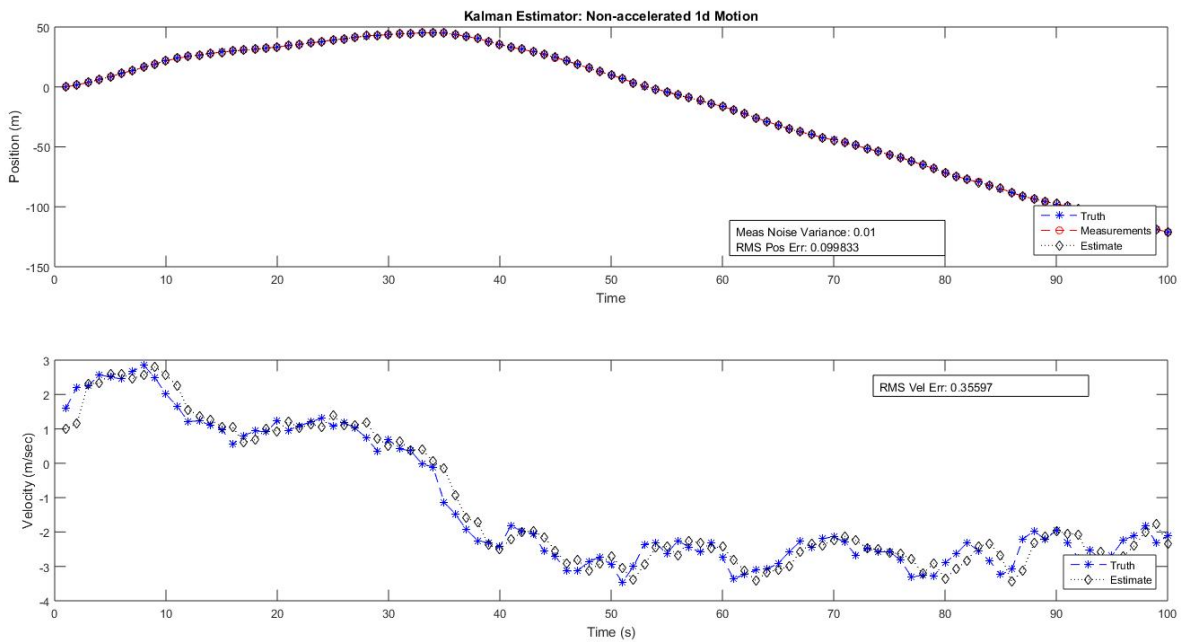


Non-accelerated Motion (constant velocity): Filter Performance (100 samples; large Process Noise)

Position Process Noise: 0.001; Velocity Process Noise: 0.1

Non-accelerated Motion (constant velocity): Filter Performance (100 samples; large Process Noise)

Position Process Noise: 0.001; Velocity Process Noise: 0.1

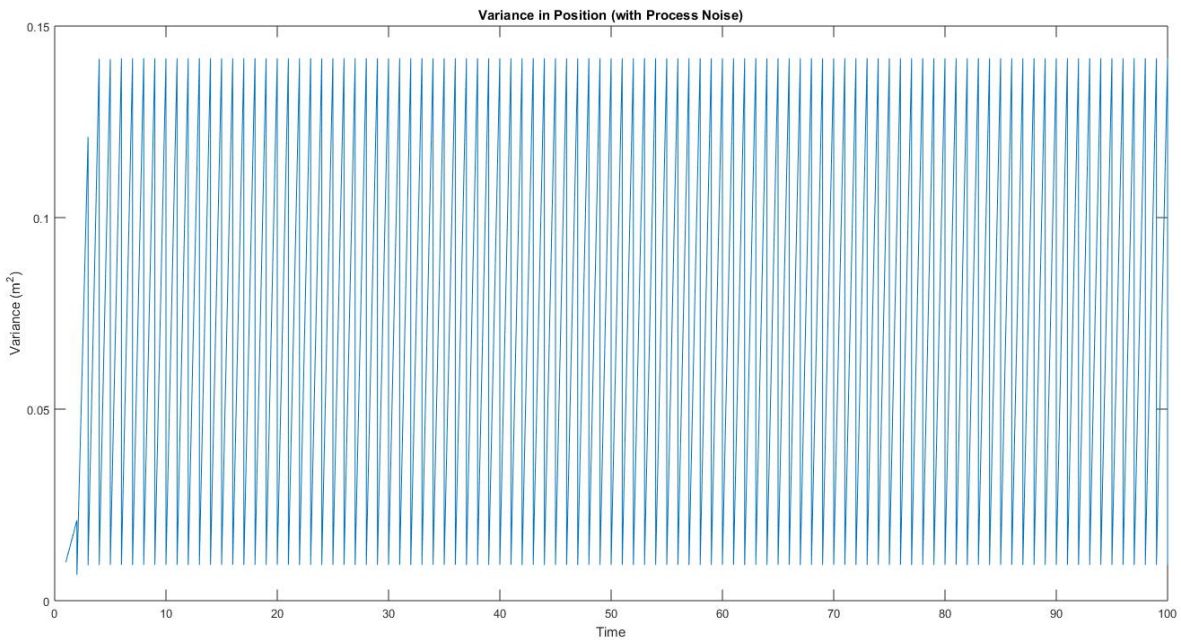


Non-accelerated Motion (constant velocity): Filter Sawtooth with Process Noise (100 samples; large Process Noise)

Position Process Noise: 0.001; Velocity Process Noise: 0.1

Non-accelerated Motion (constant velocity): Filter Sawtooth with Process Noise (100 samples; large Process Noise)

Position Process Noise: 0.001; Velocity Process Noise: 0.1

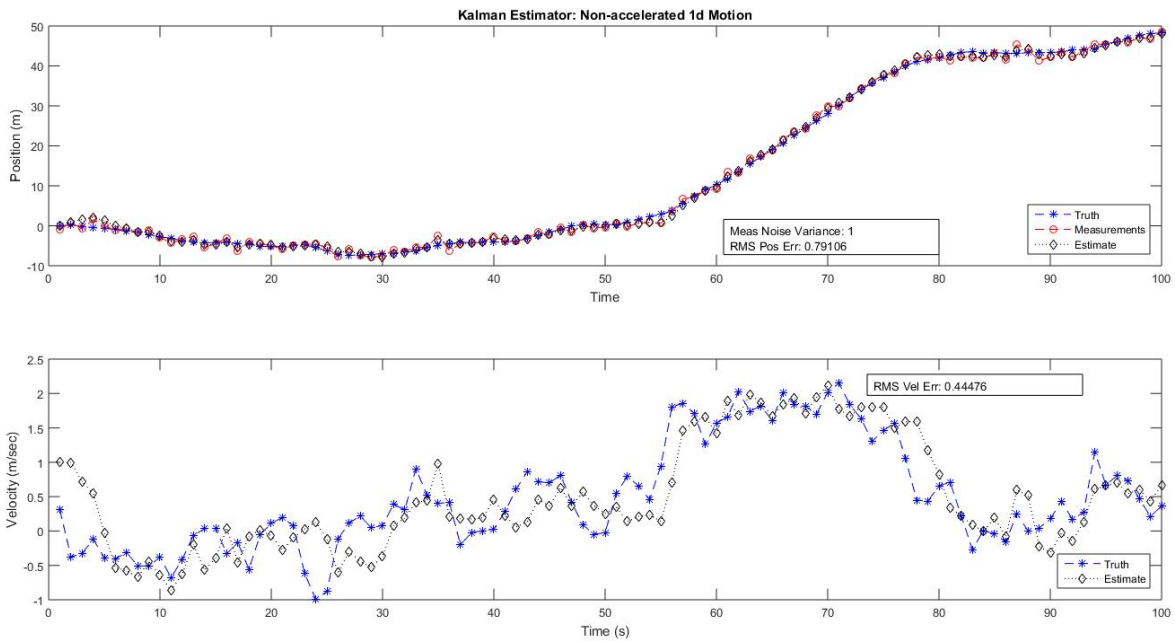


Non-accelerated Motion (constant velocity): Filter Performance

100 samples; large Process Noise; large Measurement Noise
Position Process Noise: 0.001; Velocity Process Noise: 0.1

Non-accelerated Motion (constant velocity): Filter Performance

100 samples; large Process Noise; large Measurement Noise
Position Process Noise: 0.001; Velocity Process Noise: 0.1

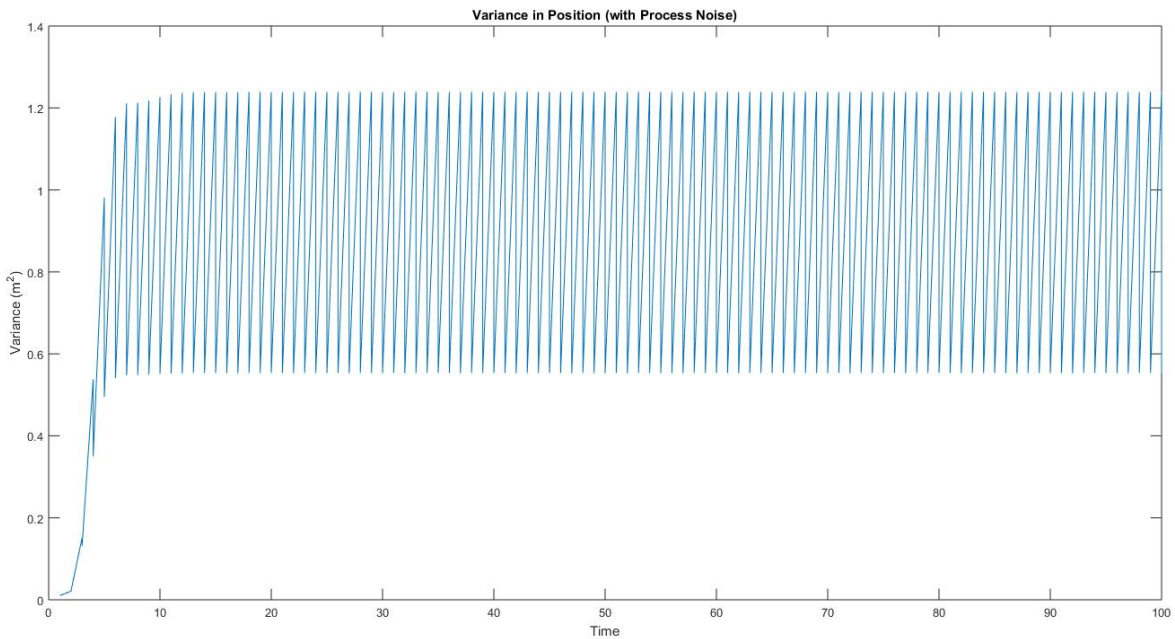


Non-accelerated Motion (constant velocity): Filter Sawtooth with Process Noise

100 samples; large Process Noise; large Measurement Noise
Position Process Noise: 0.001; Velocity Process Noise: 0.1

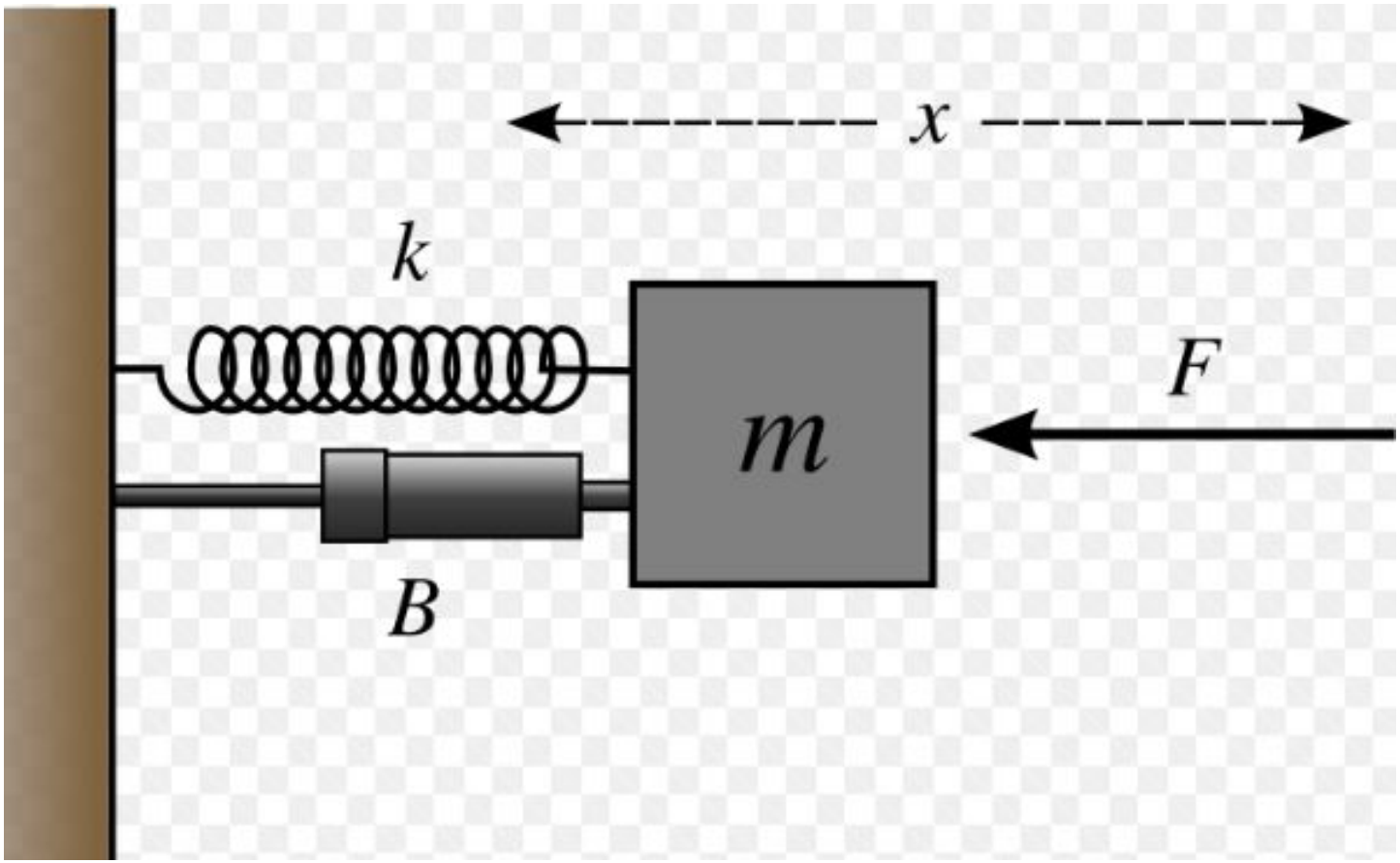
Non-accelerated Motion (constant velocity): Filter Sawtooth with Process Noise

100 samples; large Process Noise; large Measurement Noise
Position Process Noise: 0.001; Velocity Process Noise: 0.1



Damped Harmonic Oscillator: Spring, Mass, Damper System

Damped Harmonic Oscillator: Spring, Mass, Damper System



Damped Harmonic Oscillator: System Dynamics

Damped Harmonic Oscillator: System Dynamics

- Number of states = 2. System model:

Damped Harmonic Oscillator: System Dynamics

- Number of states = 2. System model:

$$\begin{bmatrix} x_1(k) \\ x_2(k) \end{bmatrix} = \exp\left(\begin{bmatrix} 0 & 1 \\ -b/m & -k/m \end{bmatrix} \Delta t \right) \begin{bmatrix} x_1(k-1) \\ x_2(k-1) \end{bmatrix} + \begin{bmatrix} w_1(k) \\ w_2(k) \end{bmatrix}$$

Damped Harmonic Oscillator: System Dynamics

- Number of states = 2. System model:

$$\begin{bmatrix} x_1(k) \\ x_2(k) \end{bmatrix} = \exp\left(\begin{bmatrix} 0 & 1 \\ -b/m & -k/m \end{bmatrix} \Delta t \right) \begin{bmatrix} x_1(k-1) \\ x_2(k-1) \end{bmatrix} + \begin{bmatrix} w_1(k) \\ w_2(k) \end{bmatrix}$$

- Thus, $\Phi(k) = \exp\left(\begin{bmatrix} 0 & 1 \\ -b/m & -k/m \end{bmatrix} \Delta t \right)$. We'll explain later how to derive state transition matrices from the underlying differential equations.

Damped Harmonic Oscillator: System Dynamics

- Number of states = 2. System model:

$$\begin{bmatrix} x_1(k) \\ x_2(k) \end{bmatrix} = \exp\left(\begin{bmatrix} 0 & 1 \\ -b/m & -k/m \end{bmatrix} \Delta t \right) \begin{bmatrix} x_1(k-1) \\ x_2(k-1) \end{bmatrix} + \begin{bmatrix} w_1(k) \\ w_2(k) \end{bmatrix}$$

- Thus, $\Phi(k) = \exp\left(\begin{bmatrix} 0 & 1 \\ -b/m & -k/m \end{bmatrix} \Delta t \right)$. We'll explain later how to derive state transition matrices from the underlying differential equations.
- Here $\Delta t = t_k - t_{k-1}$, m is the mass, k is the spring constant, and b is the damping constant

Damped Harmonic Oscillator: System Dynamics

- Number of states = 2. System model:

$$\begin{bmatrix} x_1(k) \\ x_2(k) \end{bmatrix} = \exp\left(\begin{bmatrix} 0 & 1 \\ -b/m & -k/m \end{bmatrix} \Delta t \right) \begin{bmatrix} x_1(k-1) \\ x_2(k-1) \end{bmatrix} + \begin{bmatrix} w_1(k) \\ w_2(k) \end{bmatrix}$$

- Thus, $\Phi(k) = \exp\left(\begin{bmatrix} 0 & 1 \\ -b/m & -k/m \end{bmatrix} \Delta t \right)$. We'll explain later how to derive state transition matrices from the underlying differential equations.
- Here $\Delta t = t_k - t_{k-1}$, m is the mass, k is the spring constant, and b is the damping constant
- Note that exp is a matrix exponential.

Damped Harmonic Oscillator: Process Noise

Damped Harmonic Oscillator: Process Noise

- Assume that the process noise w has the following statistics:

$$Q_k = \begin{bmatrix} 0.00001 & 0 \\ 0 & 0.000001 \end{bmatrix}$$

Damped Harmonic Oscillator: Process Noise

- Assume that the process noise w has the following statistics:

$$Q_k = \begin{bmatrix} 0.00001 & 0 \\ 0 & 0.000001 \end{bmatrix}$$

- This means that there is a small amount of position noise and an even smaller amount of velocity.

Damped Harmonic Oscillator: Process Noise

- Assume that the process noise w has the following statistics:

$$Q_k = \begin{bmatrix} 0.00001 & 0 \\ 0 & 0.000001 \end{bmatrix}$$

- This means that there is a small amount of position noise and an even smaller amount of velocity.
- It also means that the noises in position and velocity are statistically uncorrelated (i.e., no non-zero off-diagonal elements in Q_k.)

Damped Harmonic Oscillator: Process Noise

- Assume that the process noise w has the following statistics:

$$Q_k = \begin{bmatrix} 0.00001 & 0 \\ 0 & 0.000001 \end{bmatrix}$$

- This means that there is a small amount of position noise and an even smaller amount of velocity.
- It also means that the noises in position and velocity are statistically uncorrelated (i.e., no non-zero off-diagonal elements in Q_k.)
- One may think of process noise as due to unmodelled dynamic effects

Dampled Harmonic Oscillator: Measurement Model

Dampled Harmonic Oscillator: Measurement Model

- $$z(k) = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} x_1(k) \\ x_2(k) \end{bmatrix} + v(k)$$

  where the measurement noise $v(k)$ has variance σ^2

Dampled Harmonic Oscillator: Measurement Model

- $$z(k) = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} x_1(k) \\ x_2(k) \end{bmatrix} + v(k)$$

  where the measurement noise $v(k)$ has variance σ^2
- Thus $H_k = \begin{bmatrix} 1 & 0 \end{bmatrix}$

Dampled Harmonic Oscillator: Measurement Model

- $$z(k) = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} x_1(k) \\ x_2(k) \end{bmatrix} + v(k)$$

  where the measurement noise $v(k)$ has variance σ^2
- Thus $H_k = \begin{bmatrix} 1 & 0 \end{bmatrix}$
- Note that we only have a position measurement (similar to constant velocity example).

Damped Harmonic Oscillator: State Transistion Matrix Φ

Damped Harmonic Oscillator: State Transistion Matrix Φ

- $\Phi(k)$ can be approximated through series expansion of

$$\exp \begin{bmatrix} 0 & \Delta t \\ -b/m\Delta t & -k/m\Delta t \end{bmatrix}$$

Damped Harmonic Oscillator: State Transistion Matrix Φ

- $\Phi(k)$ can be approximated through series expansion of

$$\exp \begin{bmatrix} 0 & \Delta t \\ -b/m\Delta t & -k/m\Delta t \end{bmatrix}$$

- Or we can use Matlab's expm function for specific values of k,c,m, and Δt.

Damped Harmonic Oscillator: Kalman Gain

Damped Harmonic Oscillator: Kalman Gain

- Because Φ and Q are matrices, the state and covariance extrapolation equations do not simplify from their general matrix form.

Damped Harmonic Oscillator: Kalman Gain

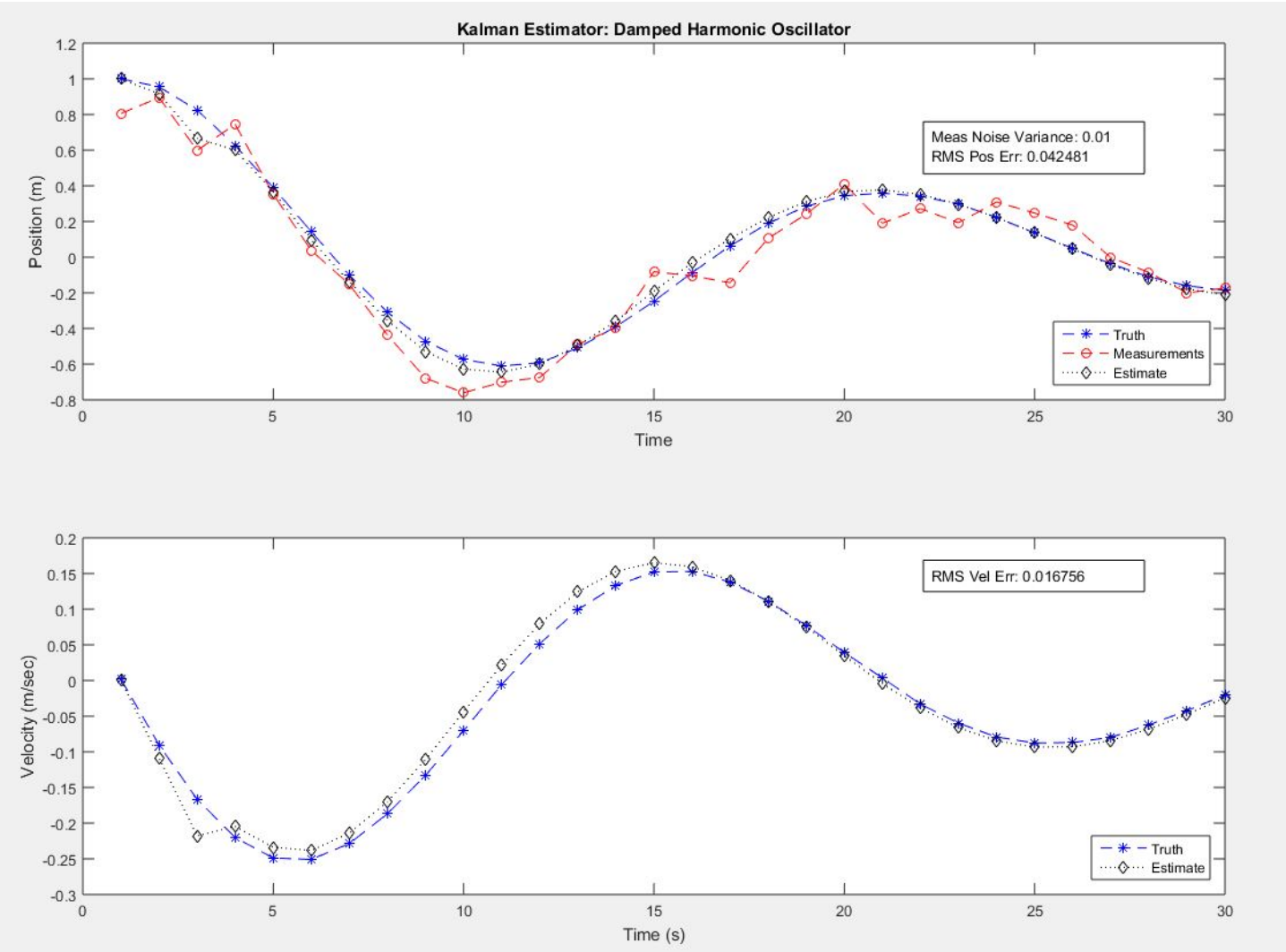
- Because Φ and Q are matrices, the state and covariance extrapolation equations do not simplify from their general matrix form.
- However, the gain equation $K(k) = P^-(k)H(k)^\top \left[H(k)P^-(k)H(k)^\top + R(k)\right]^{-1}$ looks like this:

$$K_k = \begin{bmatrix} p_{11} & p_{12} \\ p_{21} & p_{22} \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} \left(\begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} p_{11} & p_{12} \\ p_{21} & p_{22} \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \sigma^2 \right)^{-1}$$

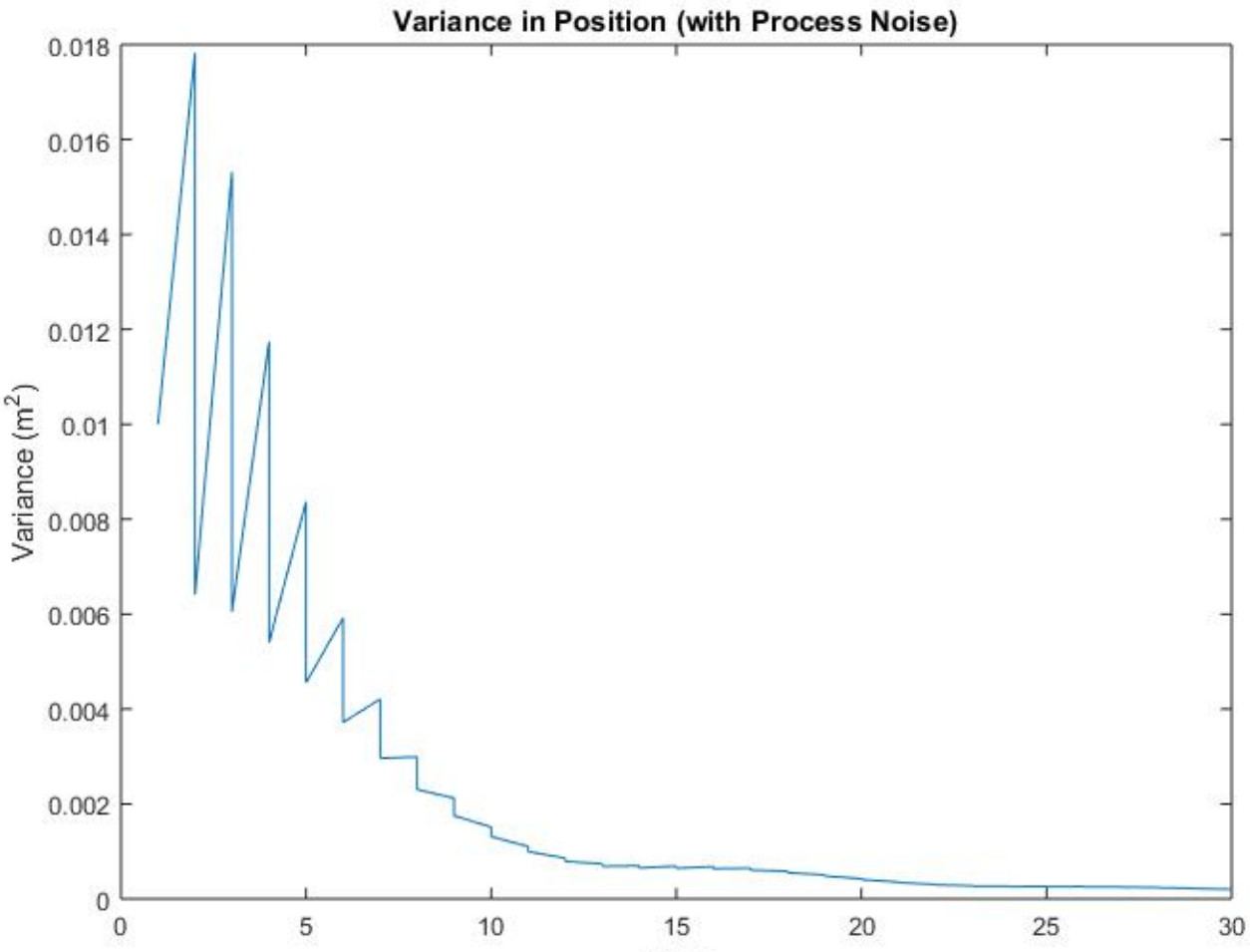
$$= \begin{bmatrix} \dfrac{p_{11}}{p_{11} + \sigma^2} \\ \dfrac{p_{21}}{p_{11} + \sigma^2} \end{bmatrix}$$

where we have once again suppresed the +/- superscripts and k's.

Damped Harmonic Oscillator Estimator: MATLAB Run



Damped Harmonic Oscillator Estimator Sawtooth: MATLAB Run



Exercises

Exercises

1. Work through the resistor example by hand using $\hat{x}_0 = 100\Omega$, $\sigma = 0.1$, and $P_0 = 0$. Start the recursive loop with the gain equation first. Run through the loop three times.

Exercises

1. Work through the resistor example by hand using $\hat{x}_0 = 100\Omega$, $\sigma = 0.1$, and $P_0 = 0$. Start the recursive loop with the gain equation first. Run through the loop three times.
2. Draw a rough plot of the error covariance, plotting both the extrapolated and updated covariance for any give k at the same time. (This will yield the classic Kalman sawtooth pattern.) Similarly, draw a rough plot of the state estimates.

Exercises

Exercises

3. Calculate the Kalman gain of the non-accelerated motion example. Hint: It looks a lot like the gain for the damped harmonic oscillator.

Exercises

3. Calculate the Kalman gain of the non-accelerated motion example. Hint: It looks a lot like the gain for the damped harmonic oscillator.
4. Write out the state update equation for the damped harmonic oscillator, using the Kalman gain derived in the example.

Exercises

3. Calculate the Kalman gain of the non-accelerated motion example. Hint: It looks a lot like the gain for the damped harmonic oscillator.
4. Write out the state update equation for the damped harmonic oscillator, using the Kalman gain derived in the example.
5. Write out the covariance update equation for the damped harmonic oscillator, using the Kalman gain dervied in the example.

Segment 4
State Space Dynamics
Topics

Segment 4
State Space Dynamics
Topics

- Dynamic Systems: Differential Equations and the Concept of State Space

Segment 4
State Space Dynamics
Topics

- Dynamic Systems: Differential Equations and the Concept of State Space
- Diagrams and State Space

Segment 4
State Space Dynamics
Topics

- Dynamic Systems: Differential Equations and the Concept of State Space
- Diagrams and State Space
- State Transition Matrix

Dynamic Systems

Dynamic Systems

- Time-dependent behavior of physical systems often modeled by differential equations

Dynamic Systems

- Time-dependent behavior of physical systems often modeled by differential equations
- Called **dynamic systems** or **dynamical systems**

Dynamic Systems

Dynamic Systems

- In Newton's law $F = ma$, the resultant force F acts on mass m to produce acceleration $a = \dfrac{dv}{dt}$

Dynamic Systems

- In Newton's law $F = ma$, the resultant force F acts on mass m to produce acceleration $a = \dfrac{dv}{dt}$
- Resultant force is vector sum of all forces acting on the body, both internal coupling forces and external input forces

Dynamic Systems

- In Newton's law $F = ma$, the resultant force F acts on mass m to produce acceleration $a = \dfrac{dv}{dt}$
- Resultant force is vector sum of all forces acting on the body, both internal coupling forces and external input forces
- Restoring forces (such as Hook's law $-ky$ force in simple oscillator problem) are often considered internal to system

Dynamic Systems

- In Newton's law $F = ma$, the resultant force F acts on mass m to produce acceleration $a = \dfrac{dv}{dt}$
- Resultant force is vector sum of all forces acting on the body, both internal coupling forces and external input forces
- Restoring forces (such as Hook's law $-ky$ force in simple oscillator problem) are often considered internal to system
- Internal forces not considered as an input *per se*

Dynamic Systems

Dynamic Systems

- Response or output of (mechanical) system is motion, accelerated motion given by a

Dynamic Systems

- Response or output of (mechanical) system is motion, accelerated motion given by a
- Simple harmonic oscillator is dynamic system, one in which there is one internal force determining system behavior: $F = -ky$
  (more complicated oscillators may also include frictional forces and internal)

Dynamic Systems

- Response or output of (mechanical) system is motion, accelerated motion given by a
- Simple harmonic oscillator is dynamic system, one in which there is one internal force determining system behavior: $F = -ky$
  (more complicated oscillators may also include frictional forces and internal)
- External forces (i.e., a time-dependent input functions) may also be present, e.g.:

$$\frac{d^2y}{dt^2} + \frac{k}{m}y = \sin \omega t$$

Dynamic Systems

Dynamic Systems

- Most general second-order equation for mechanical system of this type (according to Newton's law) is:

$$\frac{d^2y}{dt^2} = \frac{1}{m}F(y, \dot{y}, t) \tag{14}$$

Dynamic Systems

- Most general second-order equation for mechanical system of this type (according to Newton's law) is:

$$\frac{d^2y}{dt^2} = \frac{1}{m}F(y, \dot{y}, t) \tag{14}$$

- If we can express forcing function $F(y, \dot{y}, t)$ as sum of two functions $F_1(y, \dot{y})$ and $F_2(t)$, then we can rearrange into input-output form of equation (**??**):

$$\frac{d^2y}{dt^2} - \frac{1}{m}F_1(y, \dot{y}) = \frac{1}{m}F_2(t)$$

Dynamic Systems

Dynamic Systems

- In engineering applications, we are often interested not only in modeling system dynamics, but also in measuring output or resulting behavior of system in response to inputs or initial conditions

Dynamic Systems

- In engineering applications, we are often interested not only in modeling system dynamics, but also in measuring output or resulting behavior of system in response to inputs or initial conditions
- But outputs are not always directly observable!!

Dynamic Systems

- In engineering applications, we are often interested not only in modeling system dynamics, but also in measuring output or resulting behavior of system in response to inputs or initial conditions
- But outputs are not always directly observable!!
- Thus, in specifying dynamic systems, we often add second equation, usually algebraic or functional in nature, that relates output y (and possibly the input u) to some measured or observed quantities

$$z = h(y) + g(u)$$

The Concept of State

The Concept of State

- To determine behavior of deterministic system at any future time requires knowledge of two things:
  1. Equations governing system dynamics
  2. Initial conditions

The Concept of State

- To determine behavior of deterministic system at any future time requires knowledge of two things:
  1. Equations governing system dynamics
  2. Initial conditions
- But initial conditions for n^{th}-order continuous dynamic system are simply values of first $n-1$ derivatives at initial time

The Concept of State

The Concept of State

- At any point in time, we can think of instantaneous values of first $n-1$ derivatives as initial conditions for all future dynamic evolution

The Concept of State

- At any point in time, we can think of instantaneous values of first $n-1$ derivatives as initial conditions for all future dynamic evolution
- These values capture the "state" of the dynamic system at that time

The Concept of State

The Concept of State

- This concept of "state" is reinforced by considering the Taylor series expansion of a function about some point in time.

The Concept of State

- This concept of "state" is reinforced by considering the Taylor series expansion of a function about some point in time.
- Taylor's theorem says that if a function $y(t)$ is sufficiently well-behaved in a neighborhood about a point t_0, then the value of y at any other point t in that neighborhood can be determined from the values of all of its derivatives at the given point t_0.

The Concept of State

- This concept of "state" is reinforced by considering the Taylor series expansion of a function about some point in time.
- Taylor's theorem says that if a function $y(t)$ is sufficiently well-behaved in a neighborhood about a point t_0, then the value of y at any other point t in that neighborhood can be determined from the values of all of its derivatives at the given point t_0.

$$y(t) = y(t_0) + \dot{y}(t_0)(t - t_0) + \frac{1}{2!}\ddot{y}(t_0)(t - t_0)^2 + \cdots$$

The Concept of State

- This concept of "state" is reinforced by considering the Taylor series expansion of a function about some point in time.
- Taylor's theorem says that if a function $y(t)$ is sufficiently well-behaved in a neighborhood about a point t_0, then the value of y at any other point t in that neighborhood can be determined from the values of all of its derivatives at the given point t_0.

$$y(t) = y(t_0) + \dot{y}(t_0)(t - t_0) + \frac{1}{2!}\ddot{y}(t_0)(t - t_0)^2 + \cdots$$

- Note that this works for time points in the past as well as those in the future, provided they are sufficiently close to t_0.

The Concept of State

The Concept of State

- In the language of dynamic systems, the quantity $t - t_0$ (if > 0) is simply the time elapsed since the initial time t_0 (otherwise, it is the negative duration from t_0 to a past time point t).

The Concept of State

- In the language of dynamic systems, the quantity $t - t_0$ (if > 0) is simply the time elapsed since the initial time t_0 (otherwise, it is the negative duration from t_0 to a past time point t).
- Thus, if you have knowledge of the state at time t_0, i.e., if you know the values of all the (infinitely many) derivatives, then you can determine the value of the function at any later time (sufficiently close to t_0).

The Concept of State

- In the language of dynamic systems, the quantity $t - t_0$ (if > 0) is simply the time elapsed since the initial time t_0 (otherwise, it is the negative duration from t_0 to a past time point t).
- Thus, if you have knowledge of the state at time t_0, i.e., if you know the values of all the (infinitely many) derivatives, then you can determine the value of the function at any later time (sufficiently close to t_0).
- This is because all of the dynamic information is contained in the derivatives.

The Concept of State

- In the language of dynamic systems, the quantity $t - t_0$ (if > 0) is simply the time elapsed since the initial time t_0 (otherwise, it is the negative duration from t_0 to a past time point t).
- Thus, if you have knowledge of the state at time t_0, i.e., if you know the values of all the (infinitely many) derivatives, then you can determine the value of the function at any later time (sufficiently close to t_0).
- This is because all of the dynamic information is contained in the derivatives.
- Of course, if $y(t)$ is infinitely differentiable, then you can also express each of the derivatives of y at time t in terms of their values at time t_0 by means of their Taylor series, and thus from the state at t_0 you can compute the state at time t.

Concept of State

Concept of State

$$\begin{bmatrix} y(t) \\ D_t[y](t) \\ \vdots \\ D_t^n y(t) \end{bmatrix} \approx \begin{bmatrix} \frac{1}{0!} & \frac{(t-t_0)}{1!} & \cdots & \cdots & \frac{(t-t_0)^n}{(n)!} \\ 0 & \frac{1}{0!} & \frac{(t-t_0)}{1!} & \cdots & \frac{(t-t_0)^{(n-1)}}{(n-1)!} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & \cdots & \cdots & 1 \end{bmatrix} \begin{bmatrix} y(t_0) \\ D_t[y](t_0) \\ \vdots \\ D_t^y n(t_0) \end{bmatrix} \tag{15}$$

The Concept of State

The Concept of State

- In keeping with the linear spirit of things, we will restrict ourselves for now to linear functions h.

The Concept of State

- In keeping with the linear spirit of things, we will restrict ourselves for now to linear functions h.
- Also, we will now introduce the state space concept and define z as a function of the state.

The Concept of State

- In keeping with the linear spirit of things, we will restrict ourselves for now to linear functions h.
- Also, we will now introduce the state space concept and define z as a function of the state.
- Intuitively, the concept of state involves in some sense the instantaneous internal arrangement of a system. In practical situations, the state variables may represent attributes of the system that we wish to know something about.

The Concept of State

- In keeping with the linear spirit of things, we will restrict ourselves for now to linear functions h.
- Also, we will now introduce the state space concept and define z as a function of the state.
- Intuitively, the concept of state involves in some sense the instantaneous internal arrangement of a system. In practical situations, the state variables may represent attributes of the system that we wish to know something about.
- Also intuitively, state variables are usually naturally coupled in some sense; this is obvious when they are derived from a single differential equation in which all quantities are derivatives of a single variable (the output variable)

The Concept of State

The Concept of State

- Mechanical systems like a moving particle have three simultaneous attributes: position, velocity, and acceleration.

The Concept of State

- Mechanical systems like a moving particle have three simultaneous attributes: position, velocity, and acceleration.
- Newton tells us that acceleration is often a function of position, velocity and perhaps time.

The Concept of State

- Mechanical systems like a moving particle have three simultaneous attributes: position, velocity, and acceleration.
- Newton tells us that acceleration is often a function of position, velocity and perhaps time.
- However, as we just saw, a force that acts on a system as a function of time only should be considered an external forcing function, i.e., an input to the system.

The Concept of State

- Mechanical systems like a moving particle have three simultaneous attributes: position, velocity, and acceleration.
- Newton tells us that acceleration is often a function of position, velocity and perhaps time.
- However, as we just saw, a force that acts on a system as a function of time only should be considered an external forcing function, i.e., an input to the system.
- Thus we are led to think of position and velocity as the primary state variables of interest.

State Space Representation

State Space Representation

- A general n-th order linear differential equation can be expressed as a system of first-order differential equations.

State Space Representation

- A general n-th order linear differential equation can be expressed as a system of first-order differential equations.
- Ignoring the input u for the moment, consider:

$$p(D)y = D^n y + a_{n-1}D^{n-1}y + \cdots + a_1 Dy + a_0 y,$$

where we have put the polynomial in normal form, i.e., such that its leading coefficient is 1.

State Space Representation

State Space Representation

- If we consider y and its first $n-1$ derivatives as the states of this system, then we have n first order equations. Setting $x_1 = y, x_2 = Dy, \ldots$, we can write

$$
\begin{aligned}
x_1(t) &= y(t) \\
x_2(t) &= Dy(t) \\
&\ldots \\
x_n(t) &= D^{n-1}y(t).
\end{aligned}
$$

State Space Representation

- If we consider y and its first $n-1$ derivatives as the states of this system, then we have n first order equations. Setting $x_1 = y, x_2 = Dy, \ldots$, we can write

$$\begin{aligned} x_1(t) &= y(t) \\ x_2(t) &= Dy(t) \\ &\cdots \\ x_n(t) &= D^{n-1}y(t). \end{aligned}$$

- The functions $x_i(t)$ are called the **state variables**.

State Space Representation

State Space Representation

- Taking the derivative of these n state variables and making use of the original differential equation yields:

$$
\begin{aligned}
Dx_1(t) &= x_2(t) \\
Dx_2(t) &= x_3(t) \\
&\dots \\
Dx_n(t) &= -a_0 x_1(t) - a_1 x_2(t) - \dots - a_{n-1} x_n(t).
\end{aligned}
$$

State Space Representation

- Taking the derivative of these n state variables and making use of the original differential equation yields:

$$\begin{aligned} Dx_1(t) &= x_2(t) \\ Dx_2(t) &= x_3(t) \\ &\dots \\ Dx_n(t) &= -a_0 x_1(t) - a_1 x_2(t) - \cdots - a_{n-1} x_n(t). \end{aligned}$$

- These n equations can be written more compactly in vector matrix form: $Dx(t) = Ax(t)$ or $\dot{x}(t) = Ax(t)$.

State Space Representation

State Space Representation

- In the equation $\dot{x}(t) = Ax(t)$, we have:

$$x(t) = \begin{bmatrix} x_1(t) \\ x_2(t) \\ \vdots \\ x_n(t) \end{bmatrix}$$

and

$$A = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & & \ddots & & \vdots \\ 0 & 0 & 0 & 0 & 1 \\ -a_0 & -a_1 & \cdots & -a_{n-2} & -a_{n-1} \end{bmatrix},$$

the **companion matrix** of the differential equation.

State Space Representation

State Space Representation

- If there is an input function u and it is present via its derivatives as in equation (**??**), then we can write:

$$\dot{x} = Ax + Bu \tag{16}$$

forming B in much the same way we formed A, and suppressing the time variable t.

State Space Representation

- If there is an input function u and it is present via its derivatives as in equation (**??**), then we can write:

$$\dot{x} = Ax + Bu \tag{16}$$

  forming B in much the same way we formed A, and suppressing the time variable t.
- Equation (16) expresses the system's **state dynamics**.
- Later we will include white noise as an input. However, that will make the differential equations stochastic, and we are not yet ready for that.

State Space Representation

State Space Representation

- To obtain the original output y as a function of the internal state of the system, we require one additional vector-matrix equation:

$$y = Cx + Du \tag{17}$$

where the matrix D represents a **direct feedthrough** of the input to the output.

State Space Representation

- To obtain the original output y as a function of the internal state of the system, we require one additional vector-matrix equation:

$$y = Cx + Du \tag{17}$$

  where the matrix D represents a **direct feedthrough** of the input to the output.
- The output equation is derived directly from the b_i coefficients in the original matrix differential equation.

State Space Representation

State Space Representation

- Kalman filter theory generally assumes $D = 0$ in the output equation and uses a measurement model instead:

$$z = Hx \tag{18}$$

State Space Representation

- Kalman filter theory generally assumes $D = 0$ in the output equation and uses a measurement model instead:

$$z = Hx \tag{18}$$

- Note that H implicitly includes C as a factor.

State Space Representation

- Kalman filter theory generally assumes $D = 0$ in the output equation and uses a measurement model instead:

$$z = Hx \tag{18}$$

- Note that H implicitly includes C as a factor.
- Note that we have not placed any restrictions on the size of the measurement column vector z. As long as H has n columns it can have as many rows as desired.

State Space Representation: Example

State Space Representation: Example

- Let's consider once again the simple harmonic oscillator:

$$m\frac{d^2y}{dt^2} + ky = 0$$

and convert this to state space form:

$$x_1 = y$$
$$x_2 = Dy$$

and $\dot{x}_2 = -\frac{k}{m}x_1 + 0 \cdot x_2$.

State Space Representation: Example

- Let's consider once again the simple harmonic oscillator:

$$m\frac{d^2y}{dt^2} + ky = 0$$

and convert this to state space form:

$$x_1 = y$$
$$x_2 = Dy$$

and $\dot{x}_2 = -\frac{k}{m}x_1 + 0 \cdot x_2$.

- Therefore, in matrix and vector form:

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -k/m & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}.$$

State Space Representation: Example

State Space Representation: Example

- For a measurement model, we could consider a camera with a strobe flash that periodically records the instantaneous position of the sliding mass.

State Space Representation: Example

- For a measurement model, we could consider a camera with a strobe flash that periodically records the instantaneous position of the sliding mass.
- In this case, only the position would be observable and we would write the measurement or observation model as:

$$z = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}.$$

(Remember that we are ignoring both process and measurement noise.)

State Space Representation: Matlab / Simulink Example

State Space Representation: Matlab / Simulink Example

- Four parameters A, B, C, D from Matlab workspace

State Space Representation: Matlab / Simulink Example

- Four parameters A, B, C, D from Matlab workspace
- Measurement model added external to Simulink's state space block

State Space Representation

State Space Representation

- A single n^{th} order ordinary differential equation presents only one way of deriving a state space representation for a dynamic system

State Space Representation

- A single n^{th} order ordinary differential equation presents only one way of deriving a state space representation for a dynamic system
- Can also derive state space models directly from systems of coupled first order ODEs

State Space Representation

- A single n^{th} order ordinary differential equation presents only one way of deriving a state space representation for a dynamic system
- Can also derive state space models directly from systems of coupled first order ODEs
  - Real world systems consisting of electrical, mechanical, thermal, hydraulic components are coupled in this way: e.g., a DC motor driving an inertial load

State Space Representation

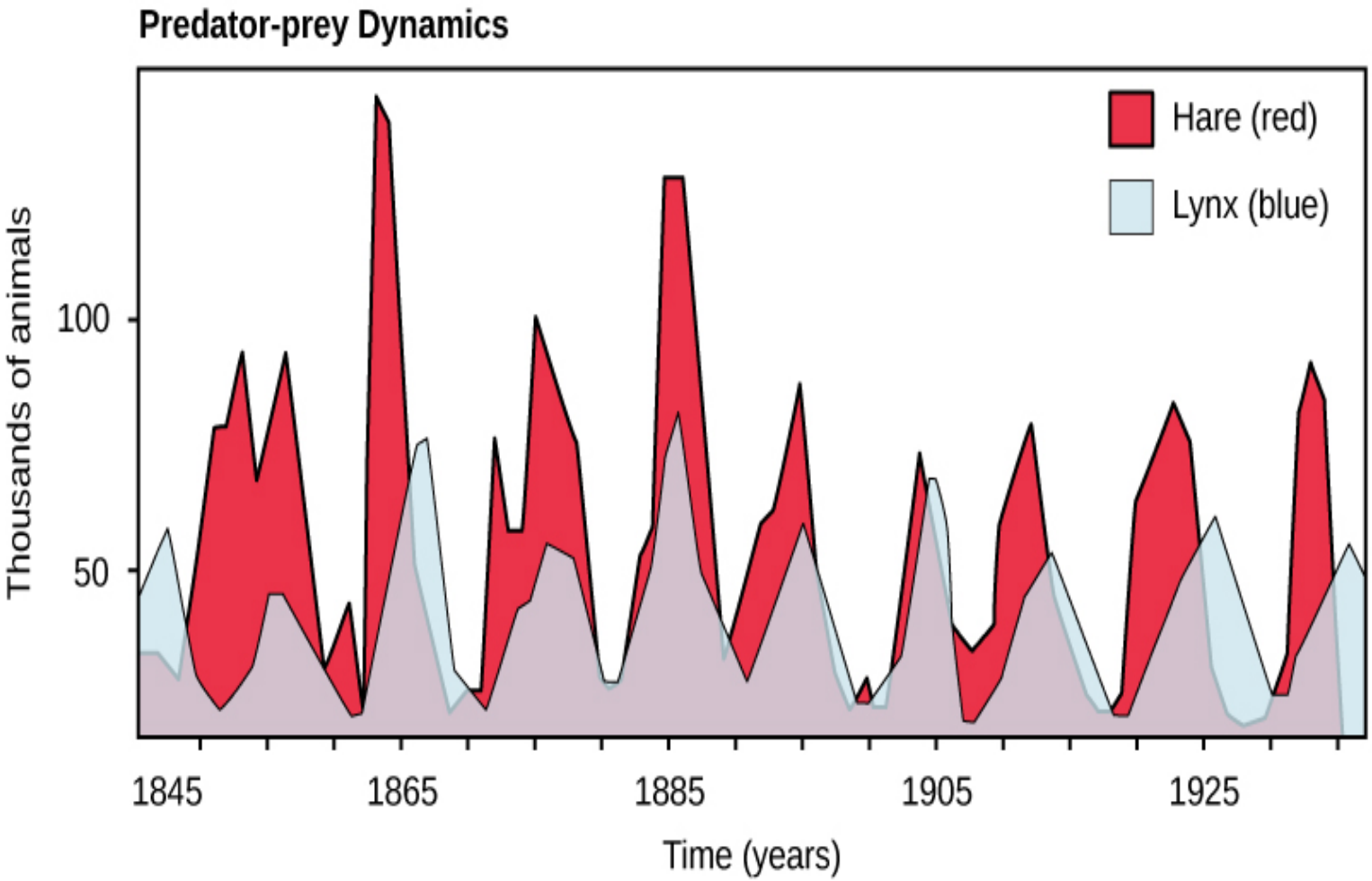
- A single n^{th} order ordinary differential equation presents only one way of deriving a state space representation for a dynamic system
- Can also derive state space models directly from systems of coupled first order ODEs
  - Real world systems consisting of electrical, mechanical, thermal, hydraulic components are coupled in this way: e.g., a DC motor driving an inertial load
- In discrete time ordinary difference equations also lead to state space models

State Space Representation

Examples: Predator-Prey Equations, Real World

State Space Representation

Examples: Predator-Prey Equations, Real World



State Space Representation

Example: Predator-Prey Equations

State Space Representation

Example: Predator-Prey Equations

- Sufficient food is always available for the prey population

State Space Representation

Example: Predator-Prey Equations

- Sufficient food is always available for the prey population
- Food supply of predator population depends only on size of prey population (predators eat only one kind of thing)

State Space Representation

Example: Predator-Prey Equations

- Sufficient food is always available for the prey population
- Food supply of predator population depends only on size of prey population (predators eat only one kind of thing)
- Current rate of change of population is proportional to its current size

State Space Representation

Example: Predator-Prey Equations

- Sufficient food is always available for the prey population
- Food supply of predator population depends only on size of prey population (predators eat only one kind of thing)
- Current rate of change of population is proportional to its current size
- Environment does not change to favor one species over another

State Space Representation

Example: Predator-Prey Equations

- Sufficient food is always available for the prey population
- Food supply of predator population depends only on size of prey population (predators eat only one kind of thing)
- Current rate of change of population is proportional to its current size
- Environment does not change to favor one species over another
- Genetic adaptation is negligible

State Space Representation

Example: Predator-Prey Equations

- Sufficient food is always available for the prey population
- Food supply of predator population depends only on size of prey population (predators eat only one kind of thing)
- Current rate of change of population is proportional to its current size
- Environment does not change to favor one species over another
- Genetic adaptation is negligible
- Predators are insatiable

State Space Representation

Example: Predator-Prey Equations

State Space Representation

Example: Predator-Prey Equations

- Lotka-Volterra Equations

State Space Representation

Example: Predator-Prey Equations

- Lotka-Volterra Equations
- Predator-Prey Population Dynamics

  Prey: $\dot{x}_1(t) = \alpha x_1(t) - \beta x_2(t)$

  Predators: $\dot{x}_2(t) = \delta x_1(t) - \gamma x_2(t)$

State Space Representation

Example: Predator-Prey Equations

- Lotka-Volterra Equations
- Predator-Prey Population Dynamics
  Prey: $\dot{x}_1(t) = \alpha x_1(t) - \beta x_2(t)$
  Predators: $\dot{x}_2(t) = \delta x_1(t) - \gamma x_2(t)$
- i.e., in state space form:

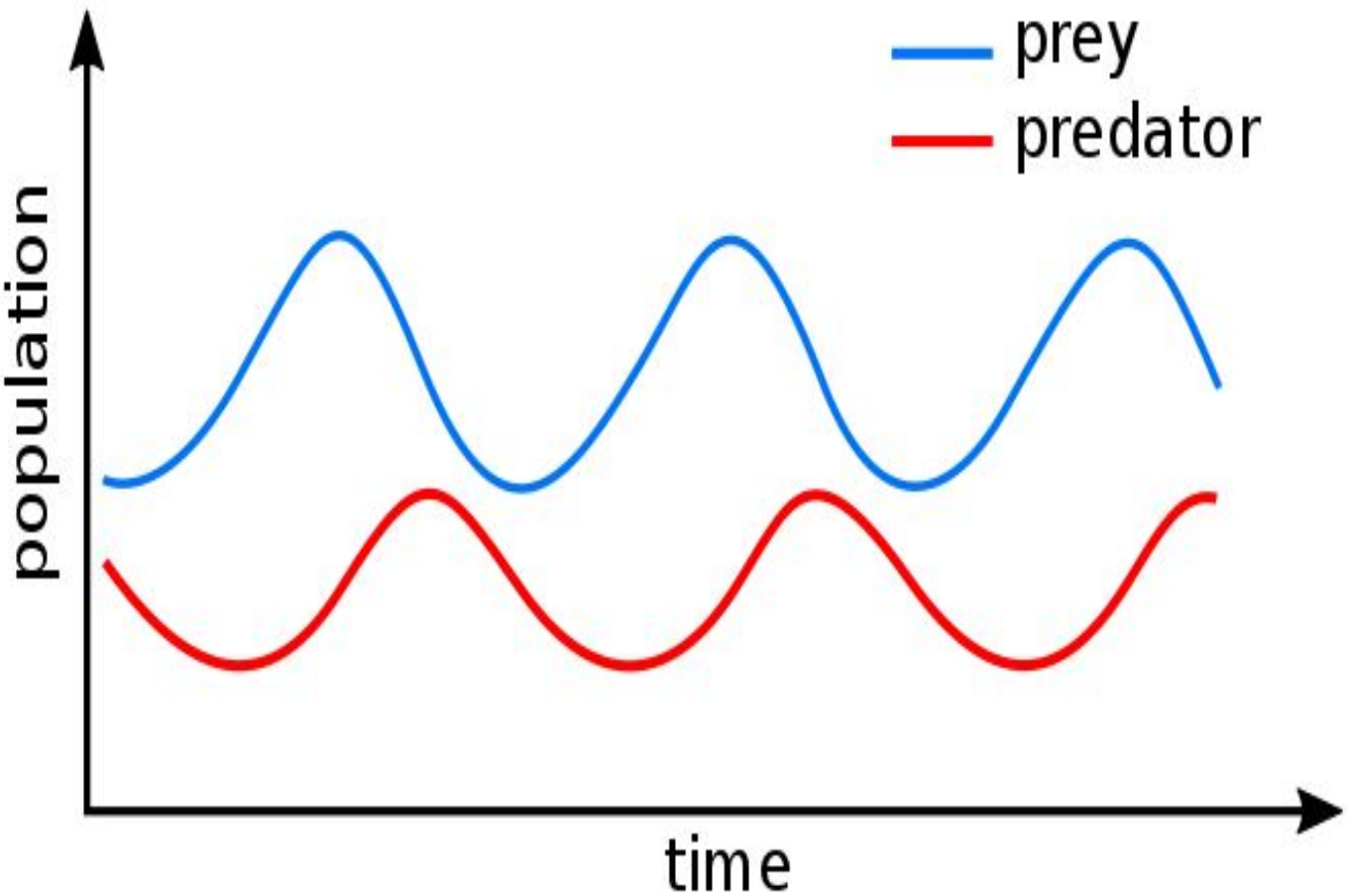
$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} \alpha & -\beta \\ \delta & -\gamma \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

State Space Representation

Examples: Predator-Prey Equations, Simulated

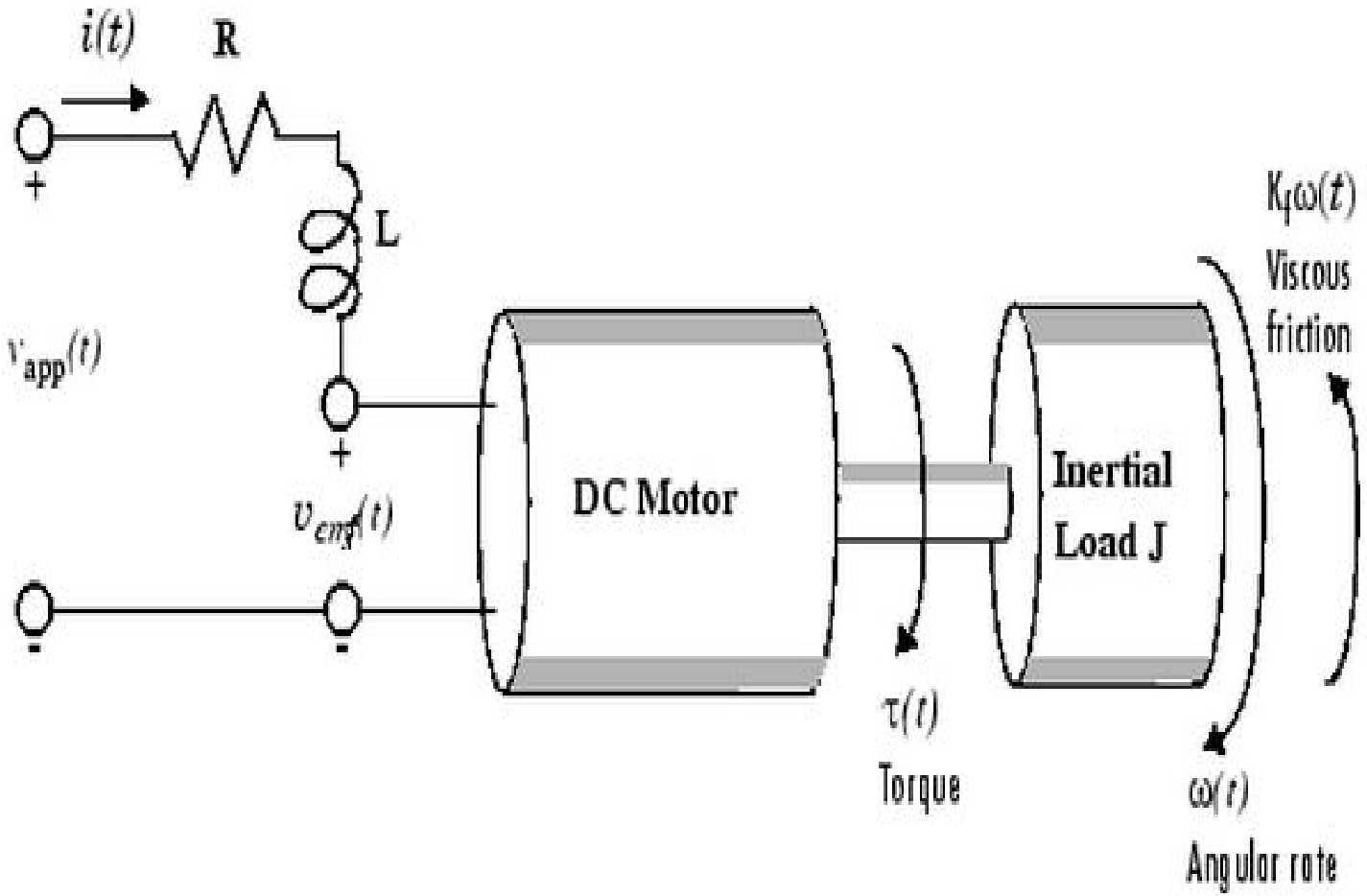
State Space Representation

Examples: Predator-Prey Equations, Simulated



State Space Representation

Examples: DC Motor with Inertial Load



Source: MATLAB Tutorial

State Space Representation

Examples: DC Motor with Inertial Load

State Space Representation

Examples: DC Motor with Inertial Load

- Electromechanical: Torque τ proportional to current i induced by applied voltage: $\tau(t) = K_m i(t)$,
  whereK_m is armature constant (related to magnetic field strength, number of turns of wire, etc.)

State Space Representation

Examples: DC Motor with Inertial Load

- Electromechanical: Torque τ proportional to current i induced by applied voltage: $\tau(t) = K_m i(t)$,
  whereK_m is armature constant (related to magnetic field strength, number of turns of wire, etc.)
- Electomechanical Back EMF: $v_{emf}(t) = K_b \omega(t)$,
  where K_b is EMF constant

State Space Representation

Examples: DC Motor with Inertial Load

- Electromechanical: Torque τ proportional to current i induced by applied voltage: $\tau(t) = K_m i(t)$,
  whereK_m is armature constant (related to magnetic field strength, number of turns of wire, etc.)
- Electomechanical Back EMF: $v_{emf}(t) = K_b \omega(t)$,
  where K_b is EMF constant
- Mechanical (from Newton's laws):
  $J\frac{d\omega}{dt} = -K_v \omega(t) + K_m i(t)$,
  where $K_f \omega$ is linear approximation of viscouous friction

State Space Representation

Examples: DC Motor with Inertial Load

- Electromechanical: Torque τ proportional to current i induced by applied voltage: $\tau(t) = K_m i(t)$,
  whereK_m is armature constant (related to magnetic field strength, number of turns of wire, etc.)
- Electomechanical Back EMF: $v_{emf}(t) = K_b \omega(t)$,
  where K_b is EMF constant
- Mechanical (from Newton's laws):
  $J\frac{d\omega}{dt} = -K_v\omega(t) + K_m i(t)$,
  where $K_f\omega$ is linear approximation of viscouous friction
- Electrical:
  $v_{app}(t) = L\frac{di}{dt} + Ri(t) + v_{emf}(t) = L\frac{di}{dt} + Ri(t) + K_b\omega(t)$,
  where v_{app} is applied voltage, L is inductance, and R is resistance

State Space Representation

Examples: DC Motor with Inertial Load

State Space Representation

Examples: DC Motor with Inertial Load

- Equations for $\frac{di}{dt}$ and $\frac{d\omega}{dt}$ lead to state space form

State Space Representation

Examples: DC Motor with Inertial Load

- Equations for $\frac{di}{dt}$ and $\frac{d\omega}{dt}$ lead to state space form
- $$\frac{d}{dt}\begin{bmatrix} i \\ \omega \end{bmatrix} = \begin{bmatrix} -\frac{R}{L} & -\frac{K_b}{L} \\ \frac{K_m}{J} & -\frac{K_f}{J} \end{bmatrix} \begin{bmatrix} i \\ \omega \end{bmatrix} + \begin{bmatrix} 1/L \\ 0 \end{bmatrix} v_{app}(t)$$

State Space Representation

Examples: DC Motor with Inertial Load

- Equations for $\frac{di}{dt}$ and $\frac{d\omega}{dt}$ lead to state space form
- $$\frac{d}{dt}\begin{bmatrix} i \\ \omega \end{bmatrix} = \begin{bmatrix} -\frac{R}{L} & -\frac{K_b}{L} \\ \frac{K_m}{J} & -\frac{K_f}{J} \end{bmatrix} \begin{bmatrix} i \\ \omega \end{bmatrix} + \begin{bmatrix} 1/L \\ 0 \end{bmatrix} v_{app}(t)$$
- Note the presence of control input $v_{app}(t)$

Ordinary Difference Equations

Ordinary Difference Equations

- Auto-Regressive Moving Average (ARMA)

Ordinary Difference Equations

- Auto-Regressive Moving Average (ARMA)
- n^{th} derivative corresponds to n-step advance:

Ordinary Difference Equations

- Auto-Regressive Moving Average (ARMA)
- n^{th} derivative corresponds to n-step advance:

$$\sum_{i=0}^{n} a_{n-i} y(k+n-i) = \sum_{i=0}^{m} b_{m-i} u(k+m-i) \qquad (19)$$

Ordinary Difference Equations

- Auto-Regressive Moving Average (ARMA)
- n^{th} derivative corresponds to n-step advance:

$$\sum_{i=0}^{n} a_{n-i} y(k+n-i) = \sum_{i=0}^{m} b_{m-i} u(k+m-i) \qquad (19)$$

- The left side of Eq. (19) is the **Auto-Regressive** part

Ordinary Difference Equations

- Auto-Regressive Moving Average (ARMA)
- n^{th} derivative corresponds to n-step advance:

$$\sum_{i=0}^{n} a_{n-i} y(k+n-i) = \sum_{i=0}^{m} b_{m-i} u(k+m-i) \qquad (19)$$

- The left side of Eq. (19) is the **Auto-Regressive** part
- The right side of Eq. (19) is the Moving Average part

Ordinary Difference Equations

Example

Ordinary Difference Equations

Example

- Let $n = 3$, $m = 2$ and consider the difference equation:

Ordinary Difference Equations

Example

- Let $n = 3$, $m = 2$ and consider the difference equation:

$$y(k+3) + 1.5y(k+2) + 7y(k+1) - 3.2y(k) = 23u(k+2) - 5u(k+1) + 0.5u(k) \quad (20)$$

Ordinary Difference Equations

Example

- Let $n = 3$, $m = 2$ and consider the difference equation:

$$y(k+3) + 1.5y(k+2) + 7y(k+1) - 3.2y(k) = 23u(k+2) - 5u(k+1) + 0.5u(k) \quad (20)$$

- A state space model can now be derived using the companion matrix method exactly as for differential equations

Ordinary Difference Equations

Example

Ordinary Difference Equations

Example

- The homgenerous version of the difference equation (20) can be converted as follows:

Ordinary Difference Equations

Example

- The homgenerous version of the difference equation (20) can be converted as follows:
- First put all but the highest order y's on the right side:
$y(k+3) = 3.2y(k) - 7y(k+1) - 1.5(k+2)$

Ordinary Difference Equations

Example

- The homgenerous version of the difference equation (20) can be converted as follows:
- First put all but the highest order y's on the right side:
  $y(k+3) = 3.2y(k) - 7y(k+1) - 1.5(k+2)$
- Define the x's:
  $x_1(k) = y(k), x_2(k) = y(k+1), x_3(k) = y(k+2)$

Ordinary Difference Equations

Example

- The homgenerous version of the difference equation (20) can be converted as follows:
- First put all but the highest order y's on the right side:
  $y(k+3) = 3.2y(k) - 7y(k+1) - 1.5(k+2)$
- Define the x's:
  $x_1(k) = y(k), x_2(k) = y(k+1), x_3(k) = y(k+2)$
- Then the state space model is:

$$\begin{bmatrix} x_1(k+1) \\ x_2(k+1) \\ x_3(k+1) \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 3.2 & -7 & -1.5 \end{bmatrix} \begin{bmatrix} x_1(k) \\ x_2(k) \\ x_3(k) \end{bmatrix}$$

Ordinary Difference Equations

Example

- The homgenerous version of the difference equation (20) can be converted as follows:
- First put all but the highest order y's on the right side:
  $y(k+3) = 3.2y(k) - 7y(k+1) - 1.5(k+2)$
- Define the x's:
  $x_1(k) = y(k), x_2(k) = y(k+1), x_3(k) = y(k+2)$
- Then the state space model is:

$$\begin{bmatrix} x_1(k+1) \\ x_2(k+1) \\ x_3(k+1) \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 3.2 & -7 & -1.5 \end{bmatrix} \begin{bmatrix} x_1(k) \\ x_2(k) \\ x_3(k) \end{bmatrix}$$

The insertion of the u's as a matrix-vector product is left as an exercise.

State Space Representation: Summary

State Space Representation: Summary

- State variables sometimes provided by 0th through $n - 1^{\text{st}}$ derivatives of single n^{th}-order differential equation plus internal dynamic coupling

State Space Representation: Summary

- State variables sometimes provided by 0th through $n - 1^{\text{st}}$ derivatives of single n^{th}-order differential equation plus internal dynamic coupling
- Equivalently, outputs of n integrators of system

State Space Representation: Summary

- State variables sometimes provided by 0th through $n - 1^{\text{st}}$ derivatives of single n^{th}-order differential equation plus internal dynamic coupling
- Equivalently, outputs of n integrators of system
- They represent implicit or explicit internal coupling of the system

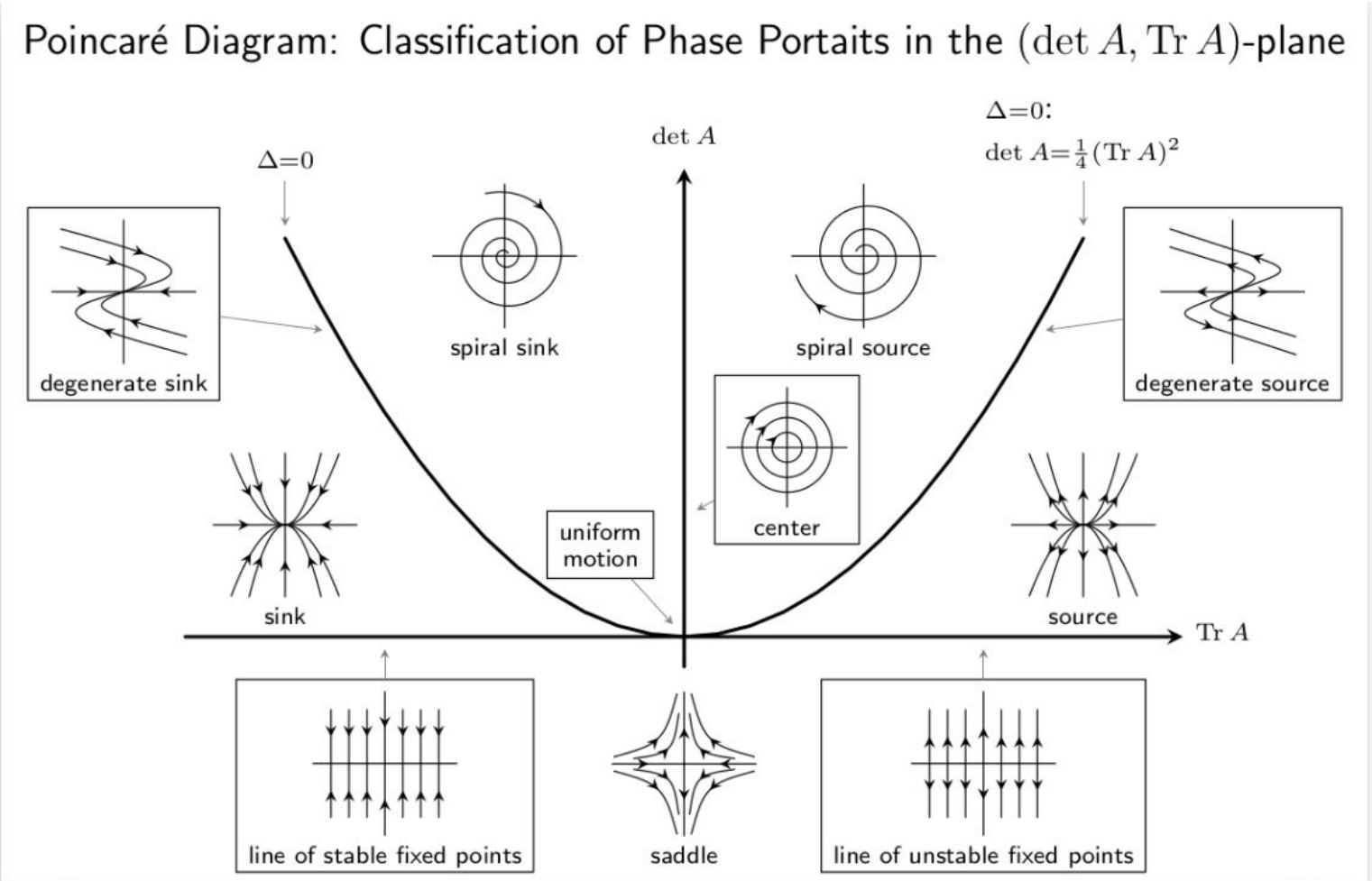
State Space Representation: Summary

- State variables sometimes provided by 0th through $n - 1^{\text{st}}$ derivatives of single n^{th}-order differential equation plus internal dynamic coupling
- Equivalently, outputs of n integrators of system
- They represent implicit or explicit internal coupling of the system
- In both natural and artificial systems, state space representations are often derived directly from observed or designed couplings

State Space Representation: Summary

- State variables sometimes provided by 0th through $n - 1^{\text{st}}$ derivatives of single n^{th}-order differential equation plus internal dynamic coupling
- Equivalently, outputs of n integrators of system
- They represent implicit or explicit internal coupling of the system
- In both natural and artificial systems, state space representations are often derived directly from observed or designed couplings
- MATLAB Control Systems Toolbox and MATLAB / Simulink have many functions and blocks facilitating state space components and interconnections

Stability



Source: By Freesodas - Gimp, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=67990034

Graphical Representations

Block Diagrams

Graphical Representations

Block Diagrams

- Edges represent signals

Graphical Representations

Block Diagrams

- Edges represent signals
- Nodes represent sources, sinks, and transformations

Graphical Representations

Block Diagrams

- Edges represent signals
- Nodes represent sources, sinks, and transformations
  - Sources: Signal generators

Graphical Representations

Block Diagrams

- Edges represent signals
- Nodes represent sources, sinks, and transformations
  - Sources: Signal generators
  - Sinks: Scopes and displays

Graphical Representations

Block Diagrams

- Edges represent signals
- Nodes represent sources, sinks, and transformations
  - Sources: Signal generators
  - Sinks: Scopes and displays
  - Various transformation blocks

Graphical Representations

Block Diagrams

- Edges represent signals
- Nodes represent sources, sinks, and transformations
  - Sources: Signal generators
  - Sinks: Scopes and displays
  - Various transformation blocks
    - Summation and takeoff points

Graphical Representations

Block Diagrams

- Edges represent signals
- Nodes represent sources, sinks, and transformations
  - Sources: Signal generators
  - Sinks: Scopes and displays
  - Various transformation blocks
    - Summation and takeoff points
    - integrators (continuous) and delayors (discrete)

Graphical Representations

Block Diagrams

- Edges represent signals
- Nodes represent sources, sinks, and transformations
  - Sources: Signal generators
  - Sinks: Scopes and displays
  - Various transformation blocks
    - Summation and takeoff points
    - integrators (continuous) and delayors (discrete)
    - differentiators

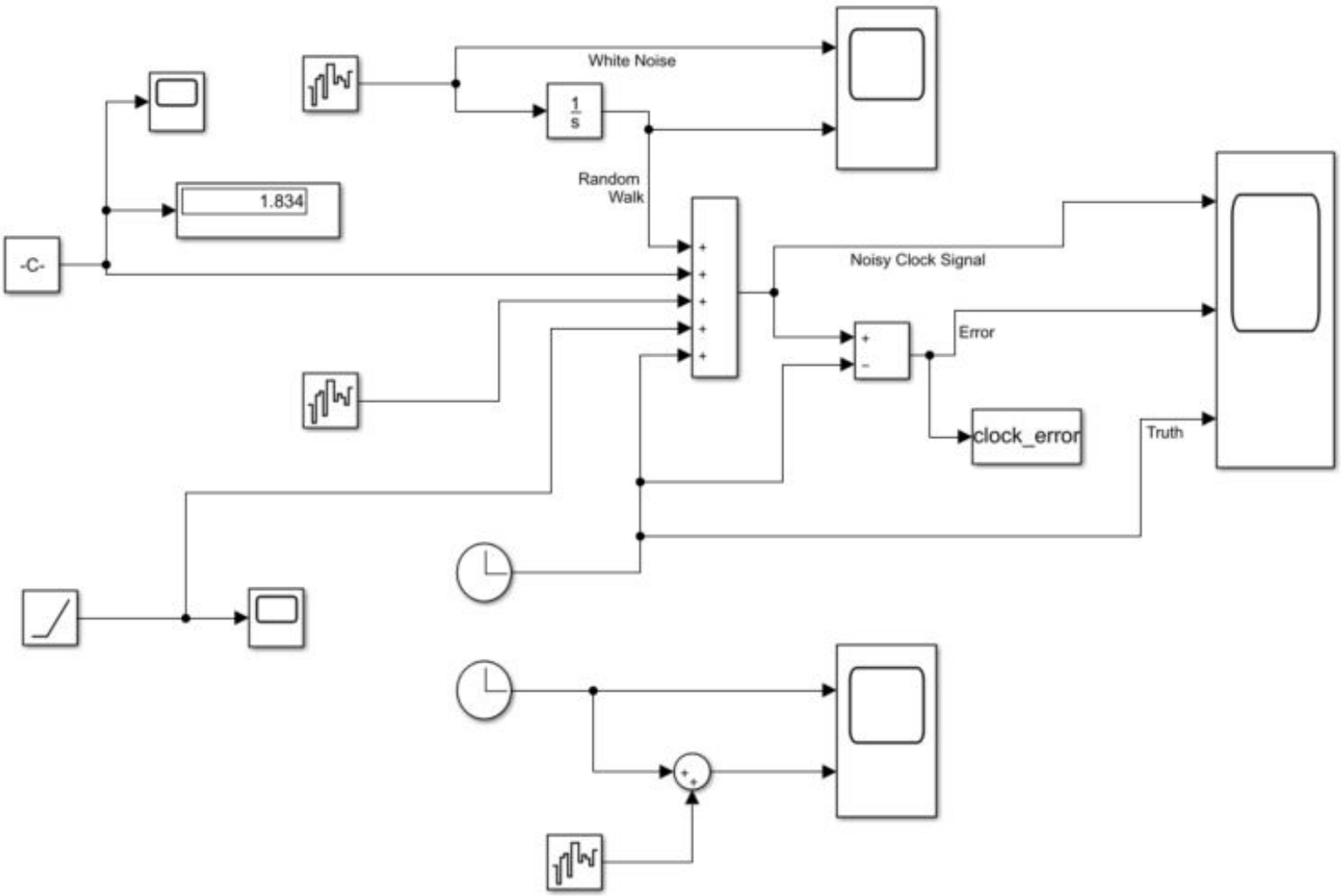
Graphical Representations

Block Diagrams

- Edges represent signals
- Nodes represent sources, sinks, and transformations
  - Sources: Signal generators
  - Sinks: Scopes and displays
  - Various transformation blocks
    - Summation and takeoff points
    - integrators (continuous) and delayors (discrete)
    - differentiators
    - scalors or gain multipliers

Block Diagrams

Example: MATLAB/Simulink Clock Model



Graphical Representations: Block Diagrams

Deriving State Equations from Block Diagrams

Graphical Representations: Block Diagrams

Deriving State Equations from Block Diagrams

- Continuous states can be identified as outputs of integrators

Graphical Representations: Block Diagrams

Deriving State Equations from Block Diagrams

- Continuous states can be identified as outputs of integrators
  - First derive differential equations

Graphical Representations: Block Diagrams

Deriving State Equations from Block Diagrams

- Continuous states can be identified as outputs of integrators
  - First derive differential equations
  - Then apply companion matrix approach to identify state dynamics matrix A

Graphical Representations: Block Diagrams

Deriving State Equations from Block Diagrams

- Continuous states can be identified as outputs of integrators
  - First derive differential equations
  - Then apply companion matrix approach to identify state dynamics matrix A
- Discrete states can be identified either as sampled continous states or as outputs of delayors

Examples

Examples

- Scalar Constant Dynamics

Examples

- Scalar Constant Dynamics
- Scalar Constant Dynamics with Recursive Estimator

Examples

- Scalar Constant Dynamics
- Scalar Constant Dynamics with Recursive Estimator
- Scalar Ramp Dynamics with Recursive Estimator
- Damped Harmonic Oscillator

Scalar Constant Dynamics

Scalar Constant Dynamics

- MATLAB/Simulink Model

Scalar Constant Dynamics

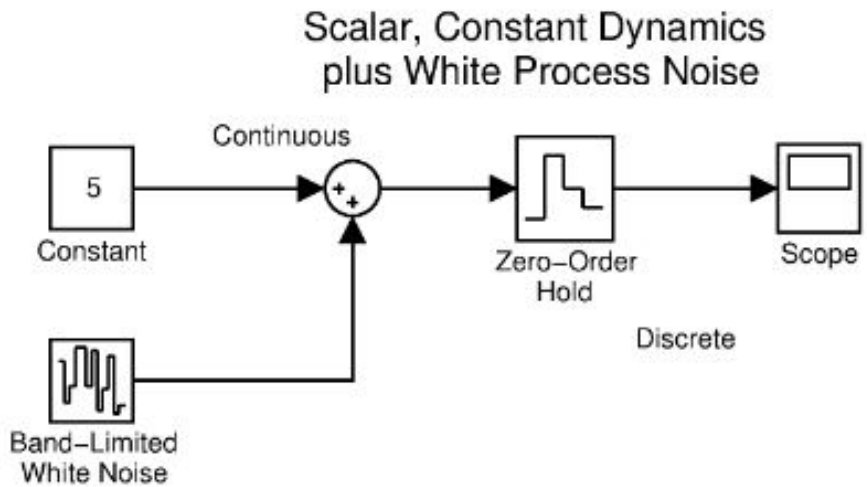
- MATLAB/Simulink Model
- Very Simple

Scalar Constant Dynamics

- MATLAB/Simulink Model
- Very Simple
- Additive Noise

Scalar Constant Dynamics

- MATLAB/Simulink Model
- Very Simple
- Additive Noise
- No feedback loops


Scalar, Constant Dynamics
plus White Process Noise
Continuous
5
Constant
Zero–Order
Hold
Scope
Discrete
Band–Limited
White Noise

Scalar Constant Dynamics with Recursive Estimator

Scalar Constant Dynamics with Recursive Estimator

- MATLAB/Simulink Model

Scalar Constant Dynamics with Recursive Estimator

- MATLAB/Simulink Model
- Very Simple Dynamics

Scalar Constant Dynamics with Recursive Estimator

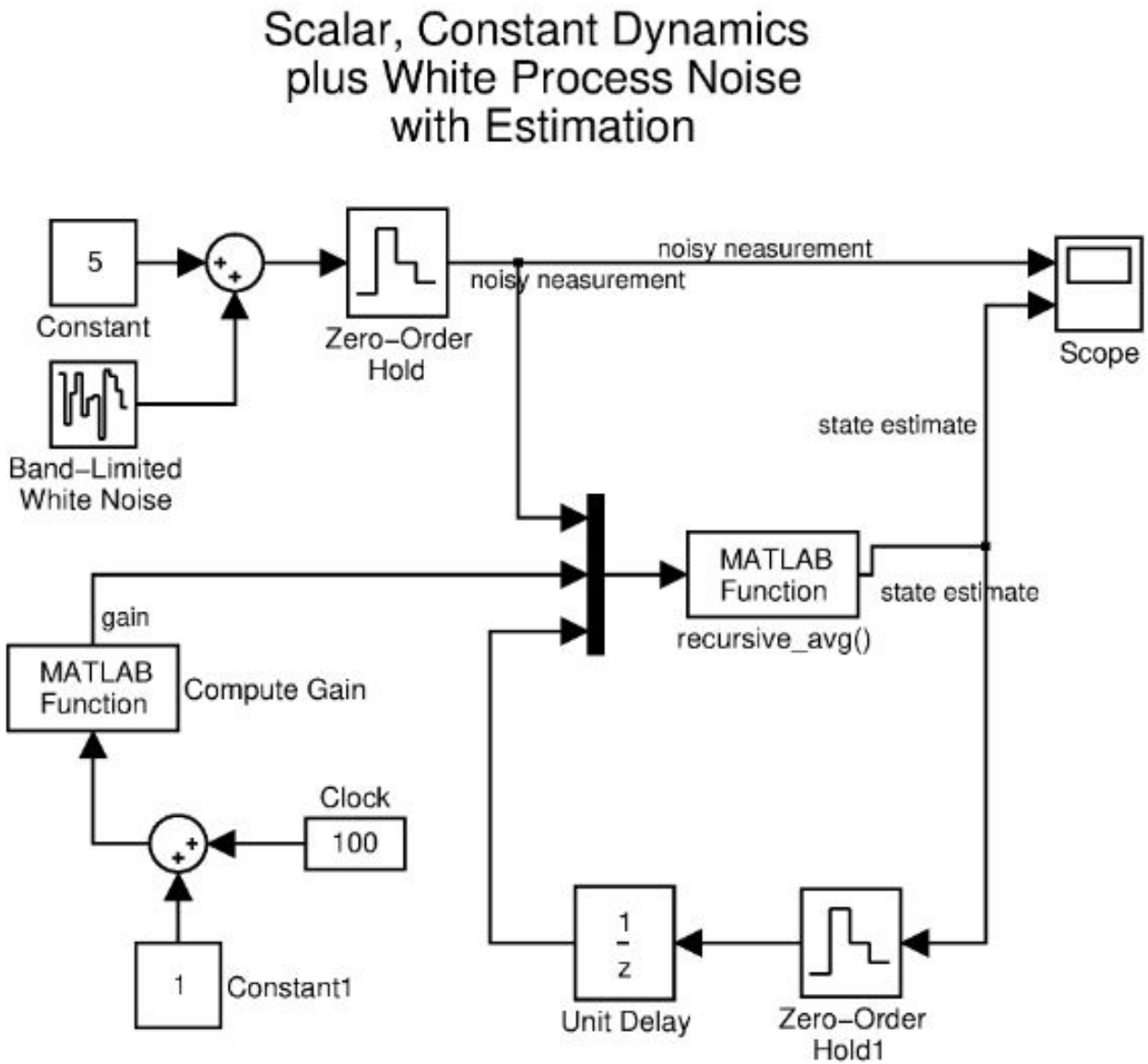
- MATLAB/Simulink Model
- Very Simple Dynamics
- Additive Noise

Scalar Constant Dynamics with Recursive Estimator

- MATLAB/Simulink Model
- Very Simple Dynamics
- Additive Noise
- Estimation loop implementing recursive simple averaging

Scalar Constant Dynamics with Recursive Estimator

- MATLAB/Simulink Model
- Very Simple Dynamics
- Additive Noise
- Estimation loop implementing recursive simple averaging
- Simple open loop gain input


Scalar, Constant Dynamics
plus White Process Noise
with Estimation
5
Constant
Band−Limited
White Noise
Zero−Order
Hold
noisy neasurement
noisy neasurement
Scope
state estimate
MATLAB
Function
recursive_avg()
state estimate
gain
MATLAB
Function
Compute Gain
Clock
100
1
Constant1
1
z
Unit Delay
Zero−Order
Hold1

Scalar Ramp Dynamics with Recursive Estimator

Scalar Ramp Dynamics with Recursive Estimator

- MATLAB/Simulink Model

Scalar Ramp Dynamics with Recursive Estimator

- MATLAB/Simulink Model
- Scalar Dynamics with known ramp

Scalar Ramp Dynamics with Recursive Estimator

- MATLAB/Simulink Model
- Scalar Dynamics with known ramp
- Additive Noise

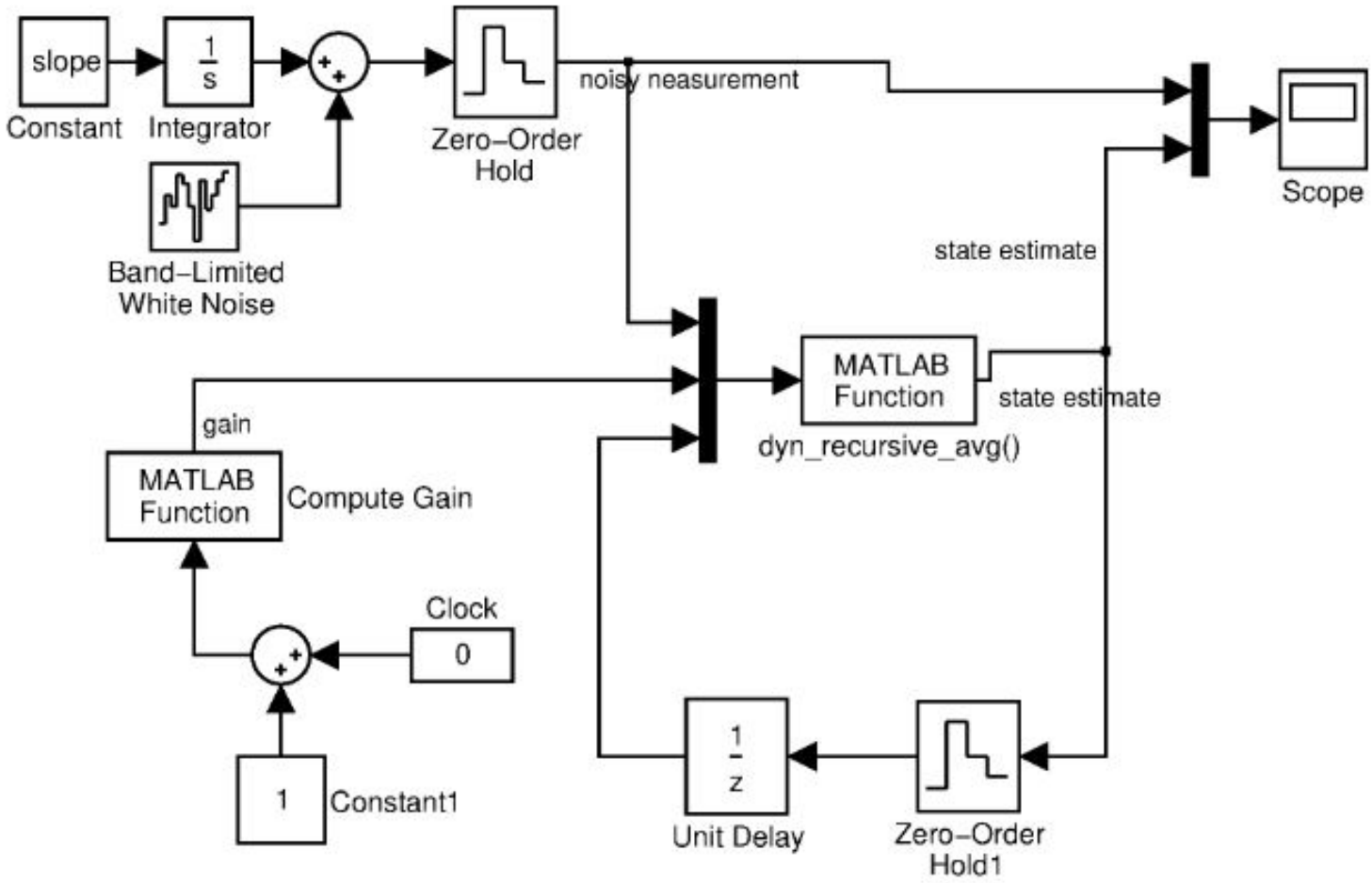
Scalar Ramp Dynamics with Recursive Estimator

- MATLAB/Simulink Model
- Scalar Dynamics with known ramp
- Additive Noise
- Estimation loop implementing recursive simple averaging

Scalar Ramp Dynamics with Recursive Estimator

- MATLAB/Simulink Model
- Scalar Dynamics with known ramp
- Additive Noise
- Estimation loop implementing recursive simple averaging
- Simple open loop gain input

Scalar, Ramp Dynamics
plus White Process Noise
with Dynamic Average Estimation



Observe how the sampling rate affects the ability of the filter to follow the input

Vector Dynamics: Damped Harmonic Oscillator

Vector Dynamics: Damped Harmonic Oscillator

- MATLAB/Simulink Model

Vector Dynamics: Damped Harmonic Oscillator

- MATLAB/Simulink Model
- 2-dimensional Vector Dynamics

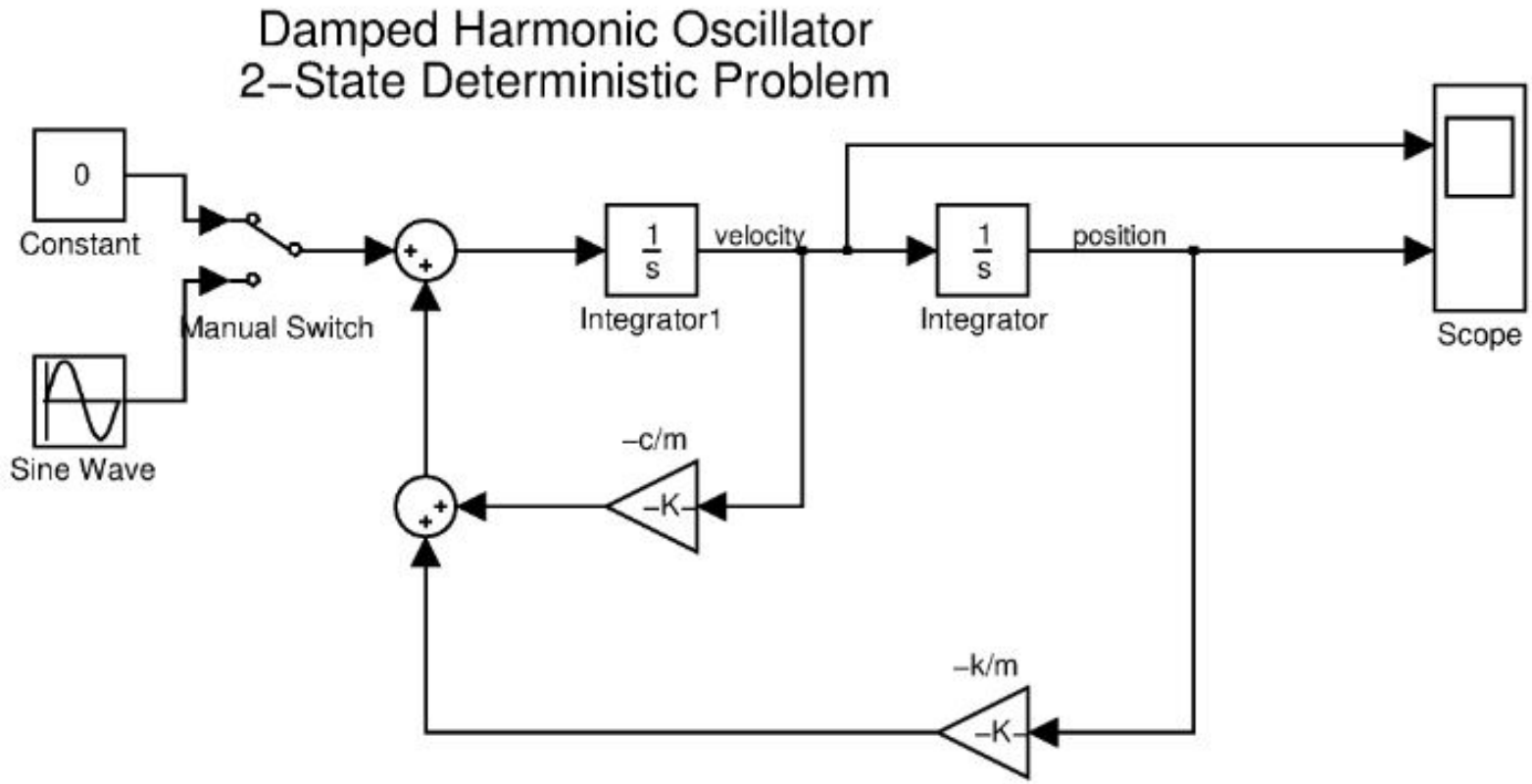
Vector Dynamics: Damped Harmonic Oscillator

- MATLAB/Simulink Model
- 2-dimensional Vector Dynamics
- Deterministic, Continuous Model

Vector Dynamics: Damped Harmonic Oscillator

- MATLAB/Simulink Model
- 2-dimensional Vector Dynamics
- Deterministic, Continuous Model

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -k/m & -c/m \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$


Damped Harmonic Oscillator
2–State Deterministic Problem
0
Constant
Manual Switch
Sine Wave
1
s
Integrator1
velocity
1
s
Integrator
position
Scope
–c/m
–K
–k/m
–K

Vector Dynamics: Damped Harmonic Oscillator

Vector Dynamics: Damped Harmonic Oscillator

- Discrete model would look like this

Vector Dynamics: Damped Harmonic Oscillator

- Discrete model would look like this

$$\begin{bmatrix} x_1(k) \\ x_2(k) \end{bmatrix} = \exp\left(\begin{bmatrix} 0 & 1 \\ -k/m & -c/m \end{bmatrix} \Delta t \right) \begin{bmatrix} x_1(k-1) \\ x_2(k-1) \end{bmatrix}$$

- Here $\Delta t = t_k - t_{k-1}$

Vector Dynamics: Damped Harmonic Oscillator

- Discrete model would look like this

$$\begin{bmatrix} x_1(k) \\ x_2(k) \end{bmatrix} = \exp\left(\begin{bmatrix} 0 & 1 \\ -k/m & -c/m \end{bmatrix} \Delta t \right) \begin{bmatrix} x_1(k-1) \\ x_2(k-1) \end{bmatrix}$$

- Here $\Delta t = t_k - t_{k-1}$
- Note that exp is a matrix exponential

Vector Dynamics: Damped Harmonic Oscillator

- Discrete model would look like this

$$\begin{bmatrix} x_1(k) \\ x_2(k) \end{bmatrix} = \exp\left(\begin{bmatrix} 0 & 1 \\ -k/m & -c/m \end{bmatrix} \Delta t \right) \begin{bmatrix} x_1(k-1) \\ x_2(k-1) \end{bmatrix}$$

- Here $\Delta t = t_k - t_{k-1}$
- Note that exp is a matrix exponential
- In the continuous case, the state transition matrix is a matrix exponential whenever the system dynamics matrix is a constant (i.e., time invariant) matrix

State Transition Matrix

State Transition Matrix

- $\Phi(t, t_0)$

State Transition Matrix

- $\Phi(t, t_0)$
- Solution of homogeneous (zero-input) equation for continuous-time systems

State Transition Matrix

- $\Phi(t, t_0)$
- Solution of homogeneous (zero-input) equation for continuous-time systems
- If discrete-time system is derived from continuous-time system, then $\Phi(k) = \Phi(k, k-1)$ satisfies:

$$x_k = \Phi(k) x_{k-1}$$

State Transition Matrix

State Transition Matrix

- Φ satisfies matrix differential equation: $\dot{\Phi}(t, t_0) = A(t)\Phi(t, t_0)$ with initial condition: $\Phi(t_0, t_0) = I$

State Transition Matrix

- Φ satisfies matrix differential equation: $\dot{\Phi}(t, t_0) = A(t)\Phi(t, t_0)$ with initial condition: $\Phi(t_0, t_0) = I$
- For a time-invariant system (constant coefficients): $\dot{x}(t) = Ax(t)$, solution is $\Phi(t_2, t_1) = \exp(A(t_2 - t_1))$

State Transition Matrix

- Φ satisfies matrix differential equation: $\dot{\Phi}(t, t_0) = A(t)\Phi(t, t_0)$ with initial condition: $\Phi(t_0, t_0) = I$
- For a time-invariant system (constant coefficients): $\dot{x}(t) = Ax(t)$, solution is $\Phi(t_2, t_1) = \exp(A(t_2 - t_1))$
- For a time-varying system: $\dot{x}(t) = A(t)x(t)$, there is NO general solution.
  But solutions can be found in special circumstances and several series solutions are available.
  E.g., If $A(t)A(s) = A(s)A(t)$, then
  $\Phi(t_2, t_1) = \exp\left(\int_{t_1}^{t_2} A(\tau)d\tau\right)$

State Transition Matrix

Properties of Φ

State Transition Matrix

Properties of Φ

- Transition Property: $\Phi(t_3, t_1) = \Phi(t_3, t_2)\Phi(t_2, t_1)$

State Transition Matrix

Properties of Φ

- Transition Property: $\Phi(t_3, t_1) = \Phi(t_3, t_2)\Phi(t_2, t_1)$
- Inversion Property: $\Phi^{-1}(t_2, t_1) = \Phi(t_1, t_2)$

State Transition Matrix

Properties of Φ

- Transition Property: $\Phi(t_3, t_1) = \Phi(t_3, t_2)\Phi(t_2, t_1)$
- Inversion Property: $\Phi^{-1}(t_2, t_1) = \Phi(t_1, t_2)$
- Separation Property: Φ can be written as $\Phi(t_2, t_1) = \theta(t_2)\theta^{-1}(t_1)$, where θ is the solution to the matrix differential equation: $\dot{\theta}(t) = A(t)\theta(t)$

State Transition Matrix

Properties of Φ

- Transition Property: $\Phi(t_3, t_1) = \Phi(t_3, t_2)\Phi(t_2, t_1)$
- Inversion Property: $\Phi^{-1}(t_2, t_1) = \Phi(t_1, t_2)$
- Separation Property: Φ can be written as $\Phi(t_2, t_1) = \theta(t_2)\theta^{-1}(t_1)$, where θ is the solution to the matrix differential equation: $\dot{\theta}(t) = A(t)\theta(t)$
- Determinant Property: $\det \Phi(t_2, t_1) = \exp\left(\int_{t_1}^{t_2} \mathrm{tr}\left[A(\tau)\right] d\tau\right)$

State Transition Matrix

Solving for Φ

State Transition Matrix

Solving for Φ

- Peano-Baker Series (valid even for time-varying $A(t)$):
  $\Phi(t_2, t_1) = I + \int_{t_1}^{t_2} A(s_1)ds_1 + \int_{t_1}^{t_2} A(s_1) \int_{t_1}^{s_1} A(s_2)ds_2 ds_1 + \cdots$

State Transition Matrix

Solving for Φ

- Peano-Baker Series (valid even for time-varying $A(t)$):
  $\Phi(t_2, t_1) = I + \int_{t_1}^{t_2} A(s_1)ds_1 + \int_{t_1}^{t_2} A(s_1) \int_{t_1}^{s_1} A(s_2)ds_2 ds_1 + \cdots$
- Another series solution is given by the Magnus series expansion involving matrix commutators
  - Truncated Magnus expansions exhibit many of the properties of Φ

Exercises

1. Convert the differential equation

$$5\frac{d^3y}{dt^3} - 0.3\frac{dy}{dt} + 7y = u(t)$$

   into a state space system model

2. Convert the difference equation

$$100y(k+9) = \sum_{i=0}^{2} b_i u(k+2-i)$$

3. Write the output equations for the above systems as well!